Control Is Motion

A New Framework for Nonlinear Systems
That Move Through the World

Contents

Nomenclature

General Conventions

- Scalars are lowercase or Greek letters (e.g., m, t, θ).
- Vectors are bold lowercase (e.g., $\mathbf{x}, \mathbf{u}, \mathbf{q}$).
- Matrices are bold uppercase (e.g., $\mathbf{A}, \mathbf{B}, \mathbf{M}, \mathbf{J}$).
- Expected values and sets are denoted by blackboard bold or script (e.g., $\mathbb{E}, \mathcal{S}$).

State and Kinematics

$\mathbf{q} \in \mathbb{R}^n$ Joint configuration vector (generalized coordinates).

$\dot{\mathbf{q}} \in \mathbb{R}^n$ Joint velocity vector (generalized velocities).

$\ddot{\mathbf{q}} \in \mathbb{R}^n$ Joint acceleration vector.

$\mathbf{x} \in \mathbb{R}^{nx}$ System state vector; commonly $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$ for mechanical systems.

$\mathbf{u} \in \mathbb{R}^m$ Control input vector (e.g., joint torques, muscle activations).

$\boldsymbol{\tau} \in \mathbb{R}^n$ Generalized forces/torques acting on the joints.

$t \in \mathbb{R}$ Time.

Inertia and Dynamics

$\mathbf{M}(\mathbf{q})$ Mass (inertia) matrix, symmetric positive definite.

$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ Coriolis and centrifugal force matrix.

$\mathbf{g}(\mathbf{q})$ Gravity vector.

$\mathbf{J}(\mathbf{q})$ Jacobian matrix relating joint velocities to task-space velocities ($\dot{\mathbf{p}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$).

Control and Optimization

$\mathbf{A}_k = \frac{\partial f}{\partial \mathbf{x}}$ Local Jacobian matrix of the state dynamics.

$\mathbf{B}_k = \frac{\partial f}{\partial \mathbf{u}}$ Local Jacobian matrix of the control input.

$V(\mathbf{x})$ **or** $\mathcal{V}$ Value function or Lyapunov function.

$\mathbf{Q}$ State penalty matrix in LQR/DDP.

$\mathbf{R}$ Control penalty matrix in LQR/DDP.

$\mathbf{K}$ Feedback gain matrix ($\delta\mathbf{u} = -\mathbf{K}\delta\mathbf{x}$).

$\boldsymbol{\gamma}$ A trajectory or flow, mapping time and initial conditions to states.

Biomechanics and Motor Control

$a_i \in [0, 1]$ Muscle activation level.

ℓ_i^M Muscle fiber length.

v_i^M Muscle fiber contraction velocity.

ℓ_i^{MT} Musculotendon unit length.

$\mathbf{F}_{\mathbf{muscle}}$ Force generated by muscles.

$\mathbf{W}$ Muscle synergy matrix (weights).

$\mathbf{c}(t)$ Muscle synergy activation vector over time.

Geometry and Manifolds

$SE(3)$ Special Euclidean group of rigid body motions.

$SO(3)$ Special Orthogonal group of 3D rotations.

$\mathfrak{se}(3)$ Lie algebra of $SE(3)$, space of spatial twists (velocities).

$\mathcal{M}$ A smooth manifold.

$T_{\mathbf{x}}\mathcal{M}$ Tangent space at point $\mathbf{x}$.

Chapter 1

Throwing Away the Target

> A river does not flow toward the ocean because it knows where the ocean is. It flows because flowing is what rivers do.
>
> — Adapted from Lao Tzu

In Plain Language 1.1. Setting the Stage: Motion, Not Destination

Classical control theory often focuses on getting a system to a specific point and keeping it there—like a thermostat holding a room at 72 degrees. However, for complex motions like a golf swing, a gymnast's routine, or a robot walking, there is no single "destination"; the motion itself is the goal. This chapter shifts the perspective from "reaching a target" to "following a trajectory." We will see how focusing on the path a system takes, rather than a single endpoint, fundamentally changes how we think about stability, optimization, and control in high-dimensional spaces. Let's explore why throwing away the target is the first step toward true mastery of motion.

1.1 The Wrong Question

Open any classical control textbook and you will find, within the first few pages, a picture like this: a system, an error signal, a reference point, and a controller whose entire purpose is to drive that error to zero. The implicit promise is seductive—*give me a destination and I will take you there.*

But consider the golf swing.

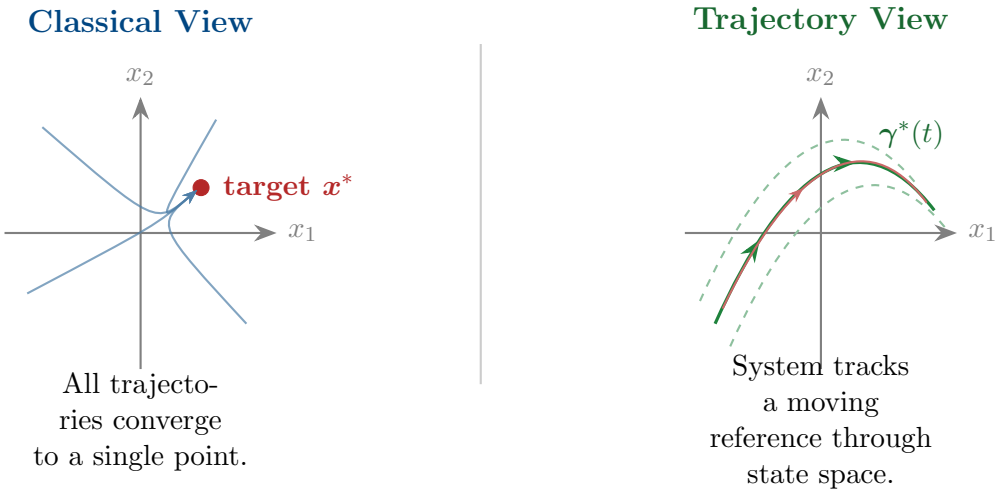
A skilled golfer does not steer the clubhead toward the ball the way a thermostat drives a room toward 72°F. There is no "setpoint" floating in space that the club is chasing. Instead, the golfer's body executes a *motion*—a coordinated, whole-body trajectory through a high-dimensional configuration space. The clubhead arrives at the ball not because it was pointed at the ball, but because the entire motion was shaped so that, somewhere along its arc, the club happens to pass through the ball's location at enormous speed.

Key Idea 1.1. The Central Thesis

Control is motion, not destination. The fundamental object of nonlinear control is not a setpoint, not an equilibrium, and not a fixed target. It is a *trajectory*—a curve through state space that the system is asked to follow, and around which disturbances are rejected. The quality of control is measured not by how close the system gets to a point, but by how faithfully it reproduces a desired motion.

This is not merely a philosophical reframing. It changes the mathematics. It changes what we optimize. It changes how we define stability. And it changes what "success" means in a controlled system.

The purpose of this book is to develop control theory from this trajectory-first perspective, to show that the classical setpoint framework is a special (and often misleading) case of a much richer theory, and to build the mathematical tools needed to design controllers for systems whose purpose is to *move*.



1.2 Why Setpoints Are a Special Case

Let us be precise about what we mean. Consider a nonlinear system

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u}), \qquad \boldsymbol{x} \in \mathcal{X} \subseteq \mathbb{R}^n, \quad \boldsymbol{u} \in \mathcal{U} \subseteq \mathbb{R}^m, \tag{1.1}$$

where $\boldsymbol{f}$ is smooth and the state space $\mathcal{X}$ may carry additional geometric structure (it may be a manifold, as we will see in Chapter 3).

In the classical framework, the control objective is:

Find $\boldsymbol{u}(t)$ *such that* $\boldsymbol{x}(t) \to \boldsymbol{x}^*$ *as* $t \to \infty$.

The entire apparatus of Lyapunov stability, LQR, pole placement, and PID control is built around this question. But notice what this objective assumes:

A1. There exists a meaningful equilibrium $\boldsymbol{x}^*$ where $\boldsymbol{f}(\boldsymbol{x}^*, \boldsymbol{u}^*) = \boldsymbol{0}$.

A2. The system's purpose is to *reach and remain at* this equilibrium.

A3. Performance is measured by how quickly and smoothly the error $\boldsymbol{e}(t) = \boldsymbol{x}(t) - \boldsymbol{x}^*$ decays.

Now consider the systems this book is about:

- A golfer executing a swing—the body passes through a continuum of configurations and never "arrives" anywhere.
- A gymnast performing a floor routine—the motion *is* the performance.
- A biped walking—there is no single posture that constitutes "walking"; walking is a *limit cycle*, a periodic orbit through state space.
- A robotic arm tracing a weld seam—the task is defined by the path, not the endpoint.

For all of these systems, assumptions **A1**–**A3** fail. The correct control objective is fundamentally different:

Principle 1.1. The Trajectory Tracking Objective

Given a reference trajectory $\boldsymbol{\gamma}^* : [0, T] \to \mathcal{X}$, find a feedback control law $\boldsymbol{u} = \boldsymbol{k}(\boldsymbol{x}, t)$ such that

$$\|\boldsymbol{x}(t) - \boldsymbol{\gamma}^*(t)\| \leq \varepsilon(t) \qquad \text{for all } t \in [0, T], \tag{1.2}$$

where $\varepsilon(t)$ is a prescribed tracking tolerance (the "tube width"), and the system is robust to disturbances $\boldsymbol{d}(t)$ satisfying $\|\boldsymbol{d}(t)\| \leq \delta$.

A setpoint is simply the degenerate case where $\boldsymbol{\gamma}^*(t) = \boldsymbol{x}^*$ for all t and $T \to \infty$. The trajectory-tracking framework *contains* the setpoint framework, but not vice versa.

Definition 1.1. Trajectory Tube

Given a reference trajectory $\boldsymbol{\gamma}^* : [0, T] \to \mathcal{X}$ and a tube radius function $\varepsilon : [0, T] \to \mathbb{R}_{>0}$, the **trajectory tube** is the time-varying set

$$\mathcal{T}(t) = \left\{ \boldsymbol{x} \in \mathcal{X} : \|\boldsymbol{x} - \boldsymbol{\gamma}^*(t)\| \leq \varepsilon(t) \right\}. \tag{1.3}$$

A controller achieves **tube-invariance** if every solution starting in $\mathcal{T}(0)$ remains in $\mathcal{T}(t)$ for all $t \in [0, T]$.

This definition is the trajectory-centric replacement for Lyapunov stability. Instead of asking "does the system converge to a point?" we ask "does the system stay inside a moving tube?" The tube can be tight where precision matters (e.g., at ball impact in the golf swing) and loose where it does not (e.g., during the backswing).

Remark 1.1. Tubes as Reachable Sets

The trajectory tube $\mathcal{T}(t)$ is intimately related to the concept of **reachable sets** and **attainable sets** from geometric control theory. These concepts form the mathematical foundation of trajectory-first control and are rigorously developed in the modern treatment by Agrachev and Sachkov [1]. A reachable set at time t is the collection of all states that can be reached from the initial condition $\boldsymbol{x}(0)$ in exactly time t, using admissible controls. The funnel-shaped tube narrows as time progresses precisely because the reachable sets, when constrained by the terminal manifold $\psi(\boldsymbol{x}(t_f)) = \mathbf{0}$, force convergence toward the target. Agrachev and Sachkov's framework for characterizing these sets through the orbit of vector fields generated by the control Lie algebra provides the theoretical underpinning for understanding why certain motions are "accessible" and others are not.

1.3 Anatomy of a Motion: The Golf Swing as Prototype

We will use the golf swing throughout this book as a running example—not because golf is inherently more interesting than other motions, but because it compresses an extraordinary number of control-theoretic challenges into roughly 1.2 seconds:

(i) **High dimensionality.** The human body has over 200 degrees of freedom. Even a simplified musculoskeletal model of the golf swing uses 20–40 generalized coordinates.

(ii) **Extreme nonlinearity.** Joint torques couple through the full rigid-body dynamics, including Coriolis and centrifugal terms that dominate at high angular velocities.

(iii) **Underactuation.** Once the club is released in the downswing, the wrist hinge "freewheels"—the golfer cannot directly control the club angle. The clubhead speed at impact comes from *passive dynamics*, not active torque.

(iv) **Time-criticality with flexible timing.** The swing must produce the right state (clubhead position, velocity, and orientation) at impact, but the *exact* time of impact is not prescribed. The golfer is free to take the backswing slowly or quickly.

(v) **Repeatability over optimality.** A touring professional does not execute the theoretically optimal swing. They execute a swing they can *repeat* 300 times in a tournament, under pressure, with minimal variance.

Example 1.1. From Setpoint Thinking to Trajectory Thinking

Suppose we model a simplified golf swing with generalized coordinates $\boldsymbol{q} = (\theta_{\text{torso}}, \theta_{\text{shoulder}}, \theta_{\text{arm}}, \theta_{\text{wrist}})^\top$ and their velocities $\dot{\boldsymbol{q}}$, giving state $\boldsymbol{x} = (\boldsymbol{q}, \dot{\boldsymbol{q}}) \in \mathbb{R}^8$.
Setpoint approach (wrong): Define a target state $\boldsymbol{x}^* = (\boldsymbol{q}_{\text{impact}}, \dot{\boldsymbol{q}}_{\text{impact}})$ and design a controller to drive $\boldsymbol{x}(t) \to \boldsymbol{x}^*$. This fails immediately: the system should not *stop* at

$\boldsymbol{x}^*$. Impact is a waypoint, not a destination. The club must be accelerating *through* the ball, not decelerating toward it.
Trajectory approach (right): Define a reference motion $\boldsymbol{\gamma}^*(s) : [0,1] \to \mathbb{R}^8$ parameterized by a phase variable s (not clock time t). The controller's job is to ensure that the system traverses $\boldsymbol{\gamma}^*$ with the right sequencing and coordination, with a tight tube near the impact phase ($s \approx 0.7$) and a loose tube during setup ($s \approx 0$).

Point (v) deserves special emphasis. Classical optimal control asks: "What is the minimum-time or minimum-energy trajectory?" But this is the wrong question for biological and athletic systems. The right question is:

Principle 1.2. The Repeatability Principle

The best trajectory is not the one that maximizes performance on a single trial, but the one that maximizes *expected performance over many trials* in the presence of noise, fatigue, and uncertainty. Formally, we seek

$$\boldsymbol{\gamma}^* = \arg\min_{\boldsymbol{\gamma}} \; \mathbb{E}\left[\int_0^T \ell\big(\boldsymbol{x}(t), \boldsymbol{u}(t)\big)\, \mathrm{d}t \right] + \lambda \,\mathrm{Var}\big[\, J(\boldsymbol{\gamma}, \boldsymbol{w}) \,\big], \tag{1.4}$$

where J is the performance cost, $\boldsymbol{w}$ represents stochastic disturbances (motor noise, perceptual error), and $\lambda > 0$ penalizes variance.

This is why elite golfers do not all swing the same way. Each golfer finds a trajectory that is locally optimal *for their body and their noise characteristics*. Robustness is not an afterthought bolted onto an optimal controller—it is the primary design objective.

1.4 The Mathematical Landscape

To make the trajectory-first perspective rigorous, we need mathematical machinery that most introductory control courses skip entirely. This section provides a roadmap of the key concepts we will develop in subsequent chapters.

1.4.1 Phase Variables and Time Reparameterization

A crucial distinction separates *what* the system does from *when* it does it. Consider two executions of the same golf swing: one with a slow backswing and one with a fast backswing. The *spatial path* through configuration space may be identical, but the *time parameterization* differs.

We formalize this by introducing a **phase variable** $s(t) \in [0,1]$ that monotonically increases from 0 (start of motion) to 1 (end of motion), with dynamics

$$\dot{s} = \alpha(s, \boldsymbol{x}), \tag{1.5}$$

where $\alpha > 0$ is the *phase velocity*. The reference trajectory is defined as a function of s, not

t:

$$\boldsymbol{\gamma}^* = \boldsymbol{\gamma}^*(s), \qquad s \in [0,1]. \tag{1.6}$$

This decouples the *shape* of the motion from its *temporal execution*. The controller has two separate tasks:

(a) **Path tracking**: keep $\boldsymbol{x}(t)$ close to the reference path $\boldsymbol{\gamma}^*(s(t))$, and

(b) **Timing control**: regulate $\dot{s}$ to achieve the desired speed profile along the path.

This decomposition is impossible in the setpoint framework, where time and space are fused together.

1.4.2 Transverse Dynamics and Orbital Stability

When we shift from point stability to trajectory stability, the relevant notion of "closeness" changes. We do not care about the distance from $\boldsymbol{x}(t)$ to $\boldsymbol{\gamma}^*(t)$ at the same clock time—we care about the distance from $\boldsymbol{x}(t)$ to the *nearest point on the trajectory*.

Definition 1.2. Transverse Distance

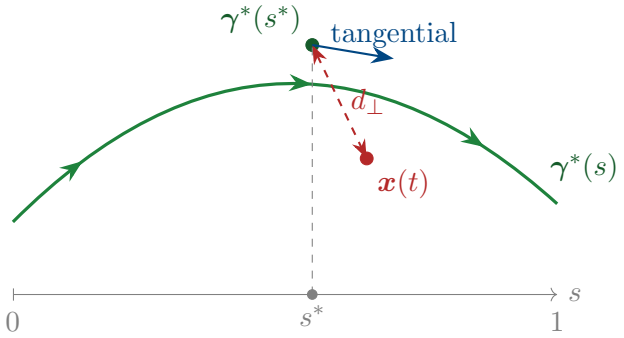
Let $\boldsymbol{\gamma}^* : [0,1] \to \mathcal{X}$ be a smooth reference trajectory. The **transverse distance** from a state $\boldsymbol{x}$ to $\boldsymbol{\gamma}^*$ is

$$d_\perp(\boldsymbol{x}, \boldsymbol{\gamma}^*) = \min_{s \in [0,1]} \|\boldsymbol{x} - \boldsymbol{\gamma}^*(s)\| . \tag{1.7}$$

The minimizing argument $s^*(\boldsymbol{x}) = \arg\min_s \|\boldsymbol{x} - \boldsymbol{\gamma}^*(s)\|$ is the **projection phase**, giving the "closest point on the reference."

This leads to a decomposition of the state into two components:

- The **tangential component**: where the system is *along* the trajectory (measured by s).
- The **transverse component**: how far the system is *from* the trajectory (measured by $d_\perp$).



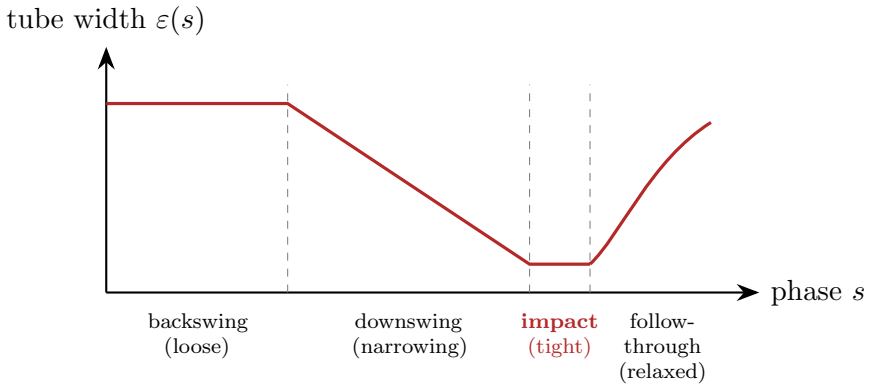
A system is **orbitally stable** around $\boldsymbol{\gamma}^*$ if the transverse distance $d_\perp(\boldsymbol{x}(t), \boldsymbol{\gamma}^*)$ remains small (or decays) even if the tangential phase $s(t)$ drifts relative to the nominal timing. This is exactly the right notion for the golf swing: we want the motion to follow the right *path* even if the golfer is slightly faster or slower than planned.

1.4.3 Funnel Control and Time-Varying Tubes

Combining the trajectory tube (Definition 1.1) with the transverse/tangential decomposition leads to what we call **funnel control**: the design of controllers that keep the system inside a prescribed time-varying tube around the reference.

The tube width $\varepsilon(s)$ is a design parameter. For the golf swing:

$$\varepsilon(s) = \begin{cases} \varepsilon_{\text{loose}} & s \in [0,\, 0.3] \quad \text{(address \& backswing)}, \\ \varepsilon_{\text{loose}} - (\varepsilon_{\text{loose}} - \varepsilon_{\text{tight}})\dfrac{s - 0.3}{0.4} & s \in [0.3,\, 0.7] \quad \text{(transition \& downswing)}, \\ \varepsilon_{\text{tight}} & s \in [0.7,\, 0.8] \quad \text{(impact zone)}, \\ \text{relaxed} & s \in [0.8,\, 1.0] \quad \text{(follow-through)}. \end{cases} \tag{1.8}$$



This funnel profile encodes the physical insight that precision matters most at impact and least during setup. The controller "spends" its control authority where it matters, rather than trying to maintain uniform precision everywhere—a lesson that classical setpoint control cannot teach.

1.5 What Changes When Control Is Motion

Adopting the trajectory-first perspective changes nearly every aspect of control system design. Here we preview the major consequences, each of which will be developed in detail in subsequent chapters.

1.5.1 Stability Is Not Convergence—It Is Confinement

In the setpoint framework, stability means convergence: the state approaches the equilibrium and stays near it. In the trajectory framework, stability means *confinement*: the state remains inside the moving tube.

Definition 1.3. Trajectory Stability

A reference trajectory $\boldsymbol{\gamma}^*$ is **stable** under the closed-loop dynamics $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{k}(\boldsymbol{x}, t))$ if for every $\varepsilon > 0$ there exists $\delta > 0$ such that

$$d_\perp(\boldsymbol{x}(0), \boldsymbol{\gamma}^*) < \delta \implies d_\perp(\boldsymbol{x}(t), \boldsymbol{\gamma}^*) < \varepsilon \quad \forall\, t \in [0, T]. \tag{1.9}$$

It is **asymptotically trajectory-stable** if, additionally, $d_\perp(\boldsymbol{x}(t), \boldsymbol{\gamma}^*) \to 0$ as the system evolves.

Note the subtle but critical difference: we use the transverse distance $d_\perp$, not the time-indexed error $\|\boldsymbol{x}(t) - \boldsymbol{\gamma}^*(t)\|$. This allows the system to be ahead of or behind schedule without being considered "unstable."

1.5.2 The Cost Function Is a Functional, Not a Function

In setpoint control, the cost is typically a function of the error $\boldsymbol{e}(t)$:

$$J_{\text{setpoint}} = \int_0^\infty \left(\boldsymbol{e}^\top \boldsymbol{Q}\, \boldsymbol{e} + \boldsymbol{u}^\top \boldsymbol{R}\, \boldsymbol{u}\right) \mathrm{d}t. \tag{1.10}$$

In trajectory control, the cost is a **functional** that operates on the entire motion:

$$J_{\text{traj}}[\boldsymbol{x}(\cdot), \boldsymbol{u}(\cdot)] = \underbrace{\int_0^T w_\perp(s)\, d_\perp^2\left(\boldsymbol{x}(t), \boldsymbol{\gamma}^*\right) \mathrm{d}t}_{\text{tracking fidelity (transverse)}} + \underbrace{\int_0^T w_\parallel(s)\, (\dot{s} - \dot{s}_{\text{ref}})^2 \, \mathrm{d}t}_{\text{timing fidelity (tangential)}} + \underbrace{\int_0^T \boldsymbol{u}^\top \boldsymbol{R}(s)\, \boldsymbol{u} \, \mathrm{d}t}_{\text{effort}}, \tag{1.11}$$

where the weighting matrices $w_\perp(s)$, $w_\parallel(s)$, and $\boldsymbol{R}(s)$ are *phase-dependent*. This is what allows us to demand high precision at impact and low effort during the backswing.

Remark 1.2. The Pontryagin Maximum Principle

The mathematical foundation for solving trajectory optimization problems with terminal constraints (such as the moving goal in the golf swing) comes from the **Pontryagin Maximum Principle**, a cornerstone of geometric control theory. As developed rigorously in Agrachev and Sachkov [1], this principle transforms the infinite-dimensional problem of optimizing $J[\boldsymbol{x}(\cdot), \boldsymbol{u}(\cdot)]$ into a two-point boundary value problem via the introduction of costate variables (adjoint variables) and a Hamiltonian function. The principle states that along an optimal trajectory, the Hamiltonian $\mathcal{H} = L(\boldsymbol{x}, \boldsymbol{u}) + \boldsymbol{\lambda}^\top \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$ is maximized with respect to $\boldsymbol{u}$ at each instant, where $\boldsymbol{\lambda}(t)$ evolves backward in time according to adjoint dynamics. This formalism unifies our treatment of both the path optimization (determining $\boldsymbol{\gamma}^*$) and the feedback stabilization (computing funnels around $\boldsymbol{\gamma}^*$) within a single geometric framework. Chapter 6 will apply these principles explicitly to synthesize the moving-target golf swing trajectory.

1.5.3 Feedforward Is Not Optional—It Is Primary

In the setpoint framework, feedforward is a "nice to have" that reduces transient error. In the trajectory framework, feedforward is the *primary* control action. The reference trajectory $\boldsymbol{\gamma}^*(t)$ implicitly defines a feedforward control signal

$$\boldsymbol{u}_{\text{ff}}(t) = \boldsymbol{f}^{-1}(\dot{\boldsymbol{\gamma}}^*(t), \boldsymbol{\gamma}^*(t)), \tag{1.12}$$

where $\boldsymbol{f}^{-1}$ denotes the inverse dynamics. The feedback controller handles only the *deviations* from this feedforward signal:

$$\boldsymbol{u}(t) = \underbrace{\boldsymbol{u}_{\text{ff}}(t)}_{\text{executes the motion}} + \underbrace{\boldsymbol{u}_{\text{fb}}(\boldsymbol{x}(t), t)}_{\text{corrects deviations}}. \tag{1.13}$$

For the golf swing, the feedforward component accounts for approximately 90% of the total joint torques. The feedback controller provides the remaining 10% of correction. A control design that treats everything as feedback is asking the feedback loop to do ten times more work than necessary.

1.5.4 Underactuation Creates Geometry, Not Just Constraints

Many moving systems are underactuated: they have fewer control inputs than degrees of freedom. In the setpoint framework, underactuation is an obstacle—it means we cannot independently control all states. In the trajectory framework, underactuation creates *geometric structure* that we can exploit.

The set of states reachable from a given configuration through passive dynamics forms a submanifold of the state space. A well-designed trajectory *rides along* this manifold, using the passive dynamics to generate motion that active torques alone could not produce. This is how the golf swing works: the wrist hinge releases passively, and the centrifugal acceleration of the downswing *automatically* squares the clubface through the double pendulum effect.

Proposition 1.1. *For a mechanical system with n degrees of freedom and $m < n$ actuators, the set of dynamically feasible trajectories lies on a submanifold of dimension at most $2m$ in the $2n$-dimensional state space. Effective trajectory design requires understanding the geometry of this submanifold.*

We will make this precise in Chapter 5 using the language of constraint manifolds and their null-space structures.

1.6 A Roadmap for the Book

The remainder of this book develops the trajectory-first theory in a sequence of increasingly sophisticated layers:

Ch. 2 Curves in State Space. The geometry of trajectories: parameterization, arc length, curvature, and torsion in high-dimensional state spaces. Frenet–Serret frames for nonlinear systems.

Ch. 3 **Configuration Manifolds.** Why state space is often not $\mathbb{R}^n$: Lie groups, joint spaces, and the geometry of articulated bodies. The golf swing lives on $SO(3)^k$, not $\mathbb{R}^k$.

Ch. 4 **Orbital Stability and Transverse Linearization.** Rigorous definitions of trajectory stability. Linearization transverse to the orbit. Floquet theory for periodic motions. Moving Poincaré sections.

Ch. 5 **Underactuation and Passive Dynamics.** Constraint Jacobians, null-space control, and the exploitation of passive mechanical coupling. The double pendulum and the physics of the wrist release.

Ch. 6 **Trajectory Optimization.** Direct collocation, shooting methods, and pseudospectral methods for finding optimal reference trajectories. The interplay between optimality and repeatability.

Ch. 7 **Funnel Synthesis.** Designing time-varying tubes with guaranteed invariance. Sums-of-squares programming for funnel computation. Robust funnels under model uncertainty.

Ch. 8 **Phase-Variable Control** [17]. Decoupling path shape from timing. Virtual constraints and hybrid zero dynamics for walking and running. Central pattern generators as phase oscillators.

Ch. 9 **Stochastic Trajectories and Motor Variability.** Signal-dependent noise in biological actuators. The minimum-variance theory of human movement [16]. Why optimal trajectories are smooth [6].

Ch. 10 **Learning to Move.** Iterative learning control, policy search, and trajectory libraries. How to improve a motion over repeated trials without losing stability.

Ch. 11 **Case Study: The Complete Golf Swing.** Full application of the entire theory to a 15-DOF musculoskeletal model of the golf swing, from trajectory optimization through funnel synthesis to stochastic robustness analysis.

1.7 A New Language for Control

Before proceeding, let us establish the conceptual vocabulary that will replace the classical language throughout this book:

Classical term	$\longrightarrow$	**Trajectory term**
Setpoint / reference	$\longrightarrow$	Reference trajectory $\boldsymbol{\gamma}^*(s)$
Error $\boldsymbol{e}(t) = \boldsymbol{x} - \boldsymbol{x}^*$	$\longrightarrow$	Transverse deviation $d_\perp(\boldsymbol{x}, \boldsymbol{\gamma}^*)$
Lyapunov stability	$\longrightarrow$	Orbital / trajectory stability
Convergence to equilibrium	$\longrightarrow$	Confinement to tube
Regulation	$\longrightarrow$	Path tracking + timing control
Step response	$\longrightarrow$	Trajectory tracking response
Steady-state error	$\longrightarrow$	Persistent transverse offset
Gain margin	$\longrightarrow$	Funnel robustness margin
Pole placement	$\longrightarrow$	Transverse eigenvalue assignment

1.8 Exercises

1.1. (Conceptual.) Consider a simple pendulum $\ddot{\theta} + \sin\theta = u$.

(a) Identify the equilibria and classify their stability.

(b) Now consider the task of making the pendulum execute a full revolution (pumping up from rest to continuous spinning). Explain why this task *cannot* be formulated as setpoint regulation.

(c) Sketch the reference trajectory in the $(\theta, \dot{\theta})$ phase plane for one complete revolution. What is the natural phase variable?

1.2. (Mathematical.) For the system $\dot{x}_1 = x_2,\ \dot{x}_2 = -x_1 + u$, consider the reference trajectory $\boldsymbol{\gamma}^*(t) = (\cos t,\ -\sin t)^\top$ (a unit circle in state space).

(a) Verify that $u_{\text{ff}}(t) = 0$ produces exactly this trajectory.

(b) Compute the transverse linearization about $\boldsymbol{\gamma}^*$.

(c) Design a feedback law u_{fb} that achieves asymptotic orbital stability.

1.3. (Design.) A simplified 2-DOF golf swing model has coordinates $\boldsymbol{q} = (\theta_{\text{arm}}, \theta_{\text{club}})$ with dynamics

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) = \boldsymbol{B}\,\boldsymbol{u}, \qquad \boldsymbol{B} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

This system is underactuated: only θ_{arm} is directly actuated.

(a) Explain qualitatively how the club can accelerate through passive dynamics even though no torque is applied at the wrist.

(b) Define a phase variable s and sketch a funnel profile $\varepsilon(s)$ appropriate for this system.

(c) Discuss why a setpoint controller applied at impact ($s = 0.7$) would produce the wrong motion.

1.4. (Philosophical.) A thermostat, an autopilot, and a dancer's body are all "controlled systems." Classify each according to whether its primary control objective is (i) setpoint regulation, (ii) trajectory tracking, or (iii) something else entirely. Justify your classification.

The river does not arrive at the ocean and stop.
It becomes the ocean.
The swing does not arrive at impact and stop.
It becomes the follow-through.

Control is motion.

Chapter 2

Curves in State Space

The shortest distance between two points is a straight line, but no interesting motion has ever been straight.

In Plain Language 2.1. State Space and Motion

If you want to describe a golf swing, you don't just need to know where the club is; you need to know how fast it's moving, where the arms are, and how the hips are rotating. "State space" is a mathematical way to keep track of all these physical variables at once. A single point in state space represents a complete snapshot of the system at one instant. As the system moves, that point traces out a path, or a "curve." In this chapter, we explore how to mathematically describe these curves. We'll find that taking apart a motion into its geometric path (its shape) and its timing (how fast it moves along that shape) makes complex motions much easier to understand and control.

2.1 Why Geometry Comes First

In Chapter 1 we argued that control is fundamentally about *motion*—the faithful execution of a trajectory through state space. Before we can stabilize a trajectory, optimize a trajectory, or confine the system to a tube around a trajectory, we need to understand what a trajectory *is* as a mathematical object.

This chapter develops the differential geometry of curves in $\mathbb{R}^n$, adapted to the needs of control theory. The central insight is that a trajectory has intrinsic geometric properties—curvature, torsion, and a natural moving frame—that exist independently of how fast the system moves along the curve. These intrinsic properties determine how difficult the trajectory is to track, how much control authority is needed to stay on course, and where the system is most vulnerable to disturbances.

The reader may wonder: *why not simply work in coordinates?* The answer is that coordinate-based descriptions mix up properties of the *curve* with properties of the *parameterization*. A golf swing executed slowly and the same swing executed quickly trace the same curve in configuration space, but their coordinate descriptions $\boldsymbol{q}(t)$ are entirely different

functions. The geometric tools we develop here allow us to separate what is intrinsic to the motion from what is merely a choice of clock.

Key Idea 2.1. Intrinsic vs. Extrinsic

A trajectory has two kinds of properties:

- **Intrinsic**: properties of the curve itself—its shape, curvature, how it bends and twists through state space. These are independent of parameterization.
- **Extrinsic**: properties that depend on how the curve is traversed—speed, acceleration, timing. These depend on the parameterization.

Good control design separates these cleanly. The *trajectory planner* designs the intrinsic shape. The *timing law* specifies the extrinsic speed profile. The *tracking controller* maintains both.

2.2 Parameterized Curves

Definition 2.1. Parameterized Curve

A **parameterized curve** in $\mathbb{R}^n$ is a smooth map

$$\boldsymbol{\gamma} : I \to \mathbb{R}^n, \qquad \lambda \mapsto \boldsymbol{\gamma}(\lambda), \tag{2.1}$$

where $I \subseteq \mathbb{R}$ is an interval. The parameter λ may be time t, arc length s, a phase variable σ, or any other monotone scalar. The curve is **regular** if $\boldsymbol{\gamma}'(\lambda) \neq \mathbf{0}$ for all $\lambda \in I$, i.e., it never "stops" (though the system traversing it may slow down).

The critical point is that many different parameterizations describe the same geometric curve. If $\boldsymbol{\gamma}(\lambda)$ is a parameterized curve and $\lambda = \phi(\mu)$ is a smooth, monotonically increasing reparameterization, then $\tilde{\boldsymbol{\gamma}}(\mu) = \boldsymbol{\gamma}(\phi(\mu))$ traces the same set of points in $\mathbb{R}^n$, but at different "speeds."

Example 2.1. The Circle: Three Parameterizations

Consider the unit circle in $\mathbb{R}^2$. Three natural parameterizations:

$$\boldsymbol{\gamma}_1(t) = (\cos t,\ \sin t), \quad t \in [0, 2\pi] \qquad \text{(constant angular speed)}, \tag{2.2}$$

$$\boldsymbol{\gamma}_2(t) = (\cos t^2,\ \sin t^2), \quad t \in [0, \sqrt{2\pi}] \qquad \text{(accelerating)}, \tag{2.3}$$

$$\boldsymbol{\gamma}_3(s) = (\cos s,\ \sin s), \quad s \in [0, 2\pi] \qquad \text{(arc-length, same as } \boldsymbol{\gamma}_1 \text{ here)}. \tag{2.4}$$

All three trace the same circle. The *curve* is the same; the *parameterizations* differ. Quantities like curvature are properties of the curve, not the parameterization.

2.2.1 Velocity, Speed, and the Tangent Vector

Given a parameterized curve $\boldsymbol{\gamma}(\lambda)$, the **velocity vector** is

$$\boldsymbol{v}(\lambda) = \frac{\mathrm{d}\boldsymbol{\gamma}}{\mathrm{d}\lambda} = \boldsymbol{\gamma}'(\lambda). \tag{2.5}$$

The **speed** is its magnitude:

$$v(\lambda) = \|\boldsymbol{\gamma}'(\lambda)\|\,. \tag{2.6}$$

The **unit tangent vector** is the velocity normalized to unit length:

$$\boldsymbol{T}(\lambda) = \frac{\boldsymbol{\gamma}'(\lambda)}{\|\boldsymbol{\gamma}'(\lambda)\|} = \frac{\boldsymbol{\gamma}'(\lambda)}{v(\lambda)}. \tag{2.7}$$

The tangent vector $\boldsymbol{T}$ tells us the *direction* of the curve at each point—a purely geometric quantity. The speed v tells us how *fast* we traverse it—a parameterization-dependent quantity. This separation is the first instance of the intrinsic/extrinsic decomposition.

2.3 Arc Length: The Natural Parameter

Among all possible parameterizations, one is geometrically privileged: the **arc-length parameterization**, in which the parameter measures distance traveled along the curve.

Definition 2.2. Arc Length

The **arc length** of a curve $\boldsymbol{\gamma}(\lambda)$ from $\lambda = a$ to $\lambda = b$ is

$$L = \int_a^b \|\boldsymbol{\gamma}'(\lambda)\|\,\mathrm{d}\lambda = \int_a^b v(\lambda)\,\mathrm{d}\lambda. \tag{2.8}$$

The **arc-length function** $s(\lambda)$ measures the distance from the starting point:

$$s(\lambda) = \int_a^{\lambda} \|\boldsymbol{\gamma}'(\mu)\|\,\mathrm{d}\mu, \qquad \text{so that } \frac{\mathrm{d}s}{\mathrm{d}\lambda} = v(\lambda). \tag{2.9}$$

When a curve is parameterized by arc length, the speed is identically 1: $\|\boldsymbol{\gamma}'(s)\| = 1$ for all s. This means the tangent vector and the velocity vector coincide: $\boldsymbol{T}(s) = \boldsymbol{\gamma}'(s)$. All geometric information is in the direction of the derivative; none is hidden in its magnitude.

Principle 2.1. Arc Length as the Canonical Clock

Arc-length parameterization strips away all timing information and reveals the pure geometry of the curve. In control terms, it answers the question: *what does the motion look like if we forget how fast it goes?*

For a golf swing, the arc-length parameterized trajectory $\boldsymbol{\gamma}(s)$ describes the spatial *path* of the body through configuration space. The timing law $s(t)$—how the golfer moves along this path in real time—is a separate design choice. A slow practice swing and a full-speed competition swing follow the same $\boldsymbol{\gamma}(s)$ with different $s(t)$.

Remark 2.1. Arc Length in High Dimensions

In $\mathbb{R}^n$ with the Euclidean metric, arc length is defined exactly as above. When the state space carries a non-Euclidean metric (as it will when we work on configuration manifolds in Chapter 3), the formula (2.8) generalizes to

$$L = \int_a^b \sqrt{\boldsymbol{\gamma}'(\lambda)^\top \boldsymbol{G}(\boldsymbol{\gamma}(\lambda))\, \boldsymbol{\gamma}'(\lambda)}\, \mathrm{d}\lambda, \tag{2.10}$$

where $\boldsymbol{G}(\boldsymbol{x})$ is the Riemannian metric tensor (for mechanical systems, this is the mass-inertia matrix). This means "distance in configuration space" naturally accounts for how massive different parts of the body are. A degree of freedom connected to a heavy segment contributes more to arc length than one connected to a light segment—exactly as physical intuition suggests.

2.4 Curvature: How Curves Bend

The most important intrinsic property of a curve is its **curvature**—a measure of how rapidly the tangent direction changes as we move along the curve.

Definition 2.3. Curvature

Let $\boldsymbol{\gamma}(s)$ be a curve parameterized by arc length in $\mathbb{R}^n$. The **curvature** at each point is

$$\kappa(s) = \|\boldsymbol{\gamma}''(s)\| = \left\| \frac{\mathrm{d}\boldsymbol{T}}{\mathrm{d}s} \right\|. \tag{2.11}$$

Since $\|\boldsymbol{T}\| = 1$, the derivative $\boldsymbol{T}'(s)$ is orthogonal to $\boldsymbol{T}(s)$, and κ measures the rate of turning per unit distance. The curvature is always non-negative and is zero if and only if the curve is locally a straight line.

The reciprocal $R(s) = 1/\kappa(s)$ is the **radius of curvature**—the radius of the circle that best approximates the curve at that point. High curvature means a tight bend; low curvature means a gentle sweep.

2.4.1 Curvature in Arbitrary Parameterization

In practice, we rarely work in arc-length parameterization. Given a curve $\boldsymbol{\gamma}(t)$ parameterized by time (or any other parameter), the curvature can be computed directly:

Theorem 2.1 (Curvature Formula)**.** *For a regular curve $\boldsymbol{\gamma}(t)$ in $\mathbb{R}^n$, the curvature is*

$$\kappa(t) = \frac{\|\boldsymbol{\gamma}'(t) \times_n \boldsymbol{\gamma}''(t)\|}{\|\boldsymbol{\gamma}'(t)\|^3}, \tag{2.12}$$

where for $n = 2$, $\|\boldsymbol{\gamma}' \times_2 \boldsymbol{\gamma}''\| = |x'y'' - y'x''|$, and for $n = 3$, $\times_3$ is the usual cross product. For general n, this generalizes via

$$\kappa(t) = \frac{\sqrt{\|\boldsymbol{\gamma}'\|^2 \|\boldsymbol{\gamma}''\|^2 - (\boldsymbol{\gamma}' \cdot \boldsymbol{\gamma}'')^2}}{\|\boldsymbol{\gamma}'\|^3}. \tag{2.13}$$

Proof. Write $\boldsymbol{\gamma}(t) = \boldsymbol{\gamma}(s(t))$ where s is arc length. Then $\boldsymbol{\gamma}' = \dot{s}\,\boldsymbol{T}$ and $\boldsymbol{\gamma}'' = \ddot{s}\,\boldsymbol{T} + \dot{s}^2\,\kappa\,\boldsymbol{N}$, where $\boldsymbol{N}$ is the unit normal (defined below). Since $\boldsymbol{T} \perp \boldsymbol{N}$, we have $\|\boldsymbol{\gamma}'\|^2 \|\boldsymbol{\gamma}''\|^2 - (\boldsymbol{\gamma}' \cdot \boldsymbol{\gamma}'')^2 = \dot{s}^2(\ddot{s}^2 + \dot{s}^4\kappa^2) - \dot{s}^2\ddot{s}^2 = \dot{s}^6\kappa^2$. Dividing by $\|\boldsymbol{\gamma}'\|^6 = \dot{s}^6$ and taking the square root yields the result. □

Example 2.2. Curvature of an Elliptical Trajectory

Consider the ellipse $\boldsymbol{\gamma}(t) = (a\cos t,\ b\sin t)$ with $a > b > 0$. Then

$$\boldsymbol{\gamma}'(t) = (-a\sin t,\ b\cos t), \tag{2.14}$$
$$\boldsymbol{\gamma}''(t) = (-a\cos t,\ -b\sin t). \tag{2.15}$$

Applying the 2D curvature formula:

$$\kappa(t) = \frac{|(-a\sin t)(-b\sin t) - (b\cos t)(-a\cos t)|}{(a^2\sin^2 t + b^2\cos^2 t)^{3/2}} = \frac{ab}{(a^2\sin^2 t + b^2\cos^2 t)^{3/2}}. \tag{2.16}$$

The curvature is *highest at the ends of the minor axis* ($t = 0, \pi$, where $\kappa = a/b^2$) and *lowest at the ends of the major axis* ($t = \pi/2, 3\pi/2$, where $\kappa = b/a^2$). The tighter the bend, the higher the curvature—and, as we will see, the more control authority is needed to track it.

2.4.2 Why Curvature Matters for Control

Curvature is not merely a geometric curiosity. It has direct implications for tracking difficulty.

Principle 2.2. The Curvature–Authority Principle

Consider a system tracking a reference trajectory $\boldsymbol{\gamma}^*(s)$ at speed v. The centripetal acceleration required to follow the curve is

$$a_\perp = \kappa\, v^2. \tag{2.17}$$

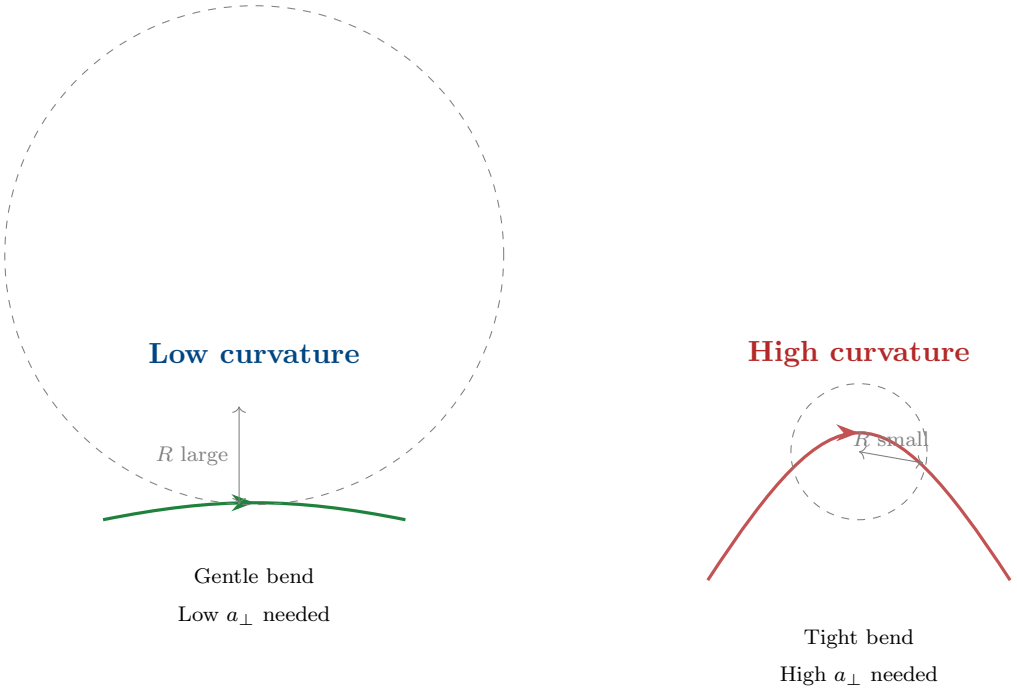
This means:

(i) High curvature demands high lateral control authority.

(ii) The required authority grows as the *square* of the speed.

(iii) A trajectory that is easy to track slowly may be impossible to track at high speed.

For the golf swing, this explains why the transition from backswing to downswing is so critical: the clubhead path has high curvature precisely where the speed is increasing rapidly.



2.5 The Frenet–Serret Frame

The tangent vector $\boldsymbol{T}$ is only the first element of a natural coordinate frame that moves with the curve. This **Frenet–Serret frame** (or simply **moving frame**) provides a complete local coordinate system at every point of the trajectory.

2.5.1 Construction in $\mathbb{R}^n$

Definition 2.4. Frenet–Serret Frame

Let $\boldsymbol{\gamma}(s)$ be a curve parameterized by arc length in $\mathbb{R}^n$ with $\boldsymbol{\gamma}^{(k)}(s)$ denoting the k-th derivative with respect to s. Assume the first $p \leq n$ derivatives $\{\boldsymbol{\gamma}'(s), \boldsymbol{\gamma}''(s), \ldots, \boldsymbol{\gamma}^{(p)}(s)\}$ are linearly independent at each point. The **Frenet–Serret frame** $\{\boldsymbol{e}_1(s), \boldsymbol{e}_2(s), \ldots, \boldsymbol{e}_p(s)\}$ is defined by applying the Gram–Schmidt process to these derivatives:

$$\boldsymbol{e}_1 = \boldsymbol{\gamma}' = \boldsymbol{T}, \tag{2.18}$$

$$\tilde{\boldsymbol{e}}_k = \boldsymbol{\gamma}^{(k)} - \sum_{j=1}^{k-1} \langle \boldsymbol{\gamma}^{(k)}, \boldsymbol{e}_j \rangle \, \boldsymbol{e}_j, \qquad \boldsymbol{e}_k = \frac{\tilde{\boldsymbol{e}}_k}{\|\tilde{\boldsymbol{e}}_k\|}, \quad k = 2, \ldots, p. \tag{2.19}$$

The frame vectors $\boldsymbol{e}_1, \ldots, \boldsymbol{e}_p$ form an orthonormal set at each point. If $p < n$, the frame can be extended to a full orthonormal basis by appending $n - p$ vectors, but these additional vectors have no intrinsic geometric meaning for the curve.

For the familiar case $n = 3$:

Vector	Name	Interpretation
$\boldsymbol{e}_1 = \boldsymbol{T}$	Unit tangent	Direction of motion
$\boldsymbol{e}_2 = \boldsymbol{N}$	Principal normal	Direction the curve is bending toward
$\boldsymbol{e}_3 = \boldsymbol{B}$	Binormal	$\boldsymbol{T} \times \boldsymbol{N}$; normal to the osculating plane

2.5.2 The Frenet–Serret Equations

The power of the moving frame lies in how it evolves along the curve. The derivatives of the frame vectors with respect to arc length are expressed entirely in terms of the frame itself and a set of scalar functions called **generalized curvatures**.

Theorem 2.2 (Frenet–Serret Equations)**.** *The Frenet–Serret frame satisfies*

$$\frac{\mathrm{d}}{\mathrm{d}s}\begin{pmatrix} \boldsymbol{e}_1 \\ \boldsymbol{e}_2 \\ \boldsymbol{e}_3 \\ \vdots \\ \boldsymbol{e}_p \end{pmatrix} = \begin{pmatrix} 0 & \kappa_1 & 0 & \cdots & 0 \\ -\kappa_1 & 0 & \kappa_2 & \cdots & 0 \\ 0 & -\kappa_2 & 0 & \cdots & 0 \\ \vdots & & \ddots & \ddots & \kappa_{p-1} \\ 0 & \cdots & 0 & -\kappa_{p-1} & 0 \end{pmatrix} \begin{pmatrix} \boldsymbol{e}_1 \\ \boldsymbol{e}_2 \\ \boldsymbol{e}_3 \\ \vdots \\ \boldsymbol{e}_p \end{pmatrix}, \tag{2.20}$$

*where $\kappa_1(s) = \kappa(s) > 0$ is the curvature, $\kappa_2(s)$ is the **torsion** (often denoted τ), and $\kappa_3, \ldots, \kappa_{p-1}$ are the **higher-order curvatures**. The matrix is skew-symmetric, which guarantees that the frame remains orthonormal as it evolves.*

The skew-symmetry of this matrix is not a coincidence—it is the hallmark of a *rotation*. As the frame moves along the curve, it rotates without stretching. The curvatures $\kappa_1, \ldots, \kappa_{p-1}$ are the "angular velocities" of this rotation projected onto the various planes of the frame.

Example 2.3. Frenet–Serret for a 3D Helix

The helix $\boldsymbol{\gamma}(t) = (a\cos t,\ a\sin t,\ bt)$ with speed $v = \sqrt{a^2 + b^2}$ has arc-length parameter

$s = vt$. The Frenet–Serret quantities are:

$$\boldsymbol{T} = \frac{1}{v}(-a\sin t,\ a\cos t,\ b), \tag{2.21}$$

$$\boldsymbol{N} = (-\cos t,\ -\sin t,\ 0), \tag{2.22}$$

$$\boldsymbol{B} = \frac{1}{v}(b\sin t,\ -b\cos t,\ a), \tag{2.23}$$

$$\kappa = \frac{a}{a^2+b^2}, \qquad \tau = \frac{b}{a^2+b^2}. \tag{2.24}$$

Both curvature and torsion are *constant*, which makes the helix a natural reference trajectory for testing tracking controllers—analogous to the role of step inputs in classical control. The helix is to trajectory control what the step function is to setpoint control.

2.5.3 The Moving Frame in State Space

For control applications, the state space typically has dimension $n = 2N$ where N is the number of generalized coordinates (positions and velocities). A golf swing model with 4 DOF has $n = 8$. The Frenet–Serret frame in $\mathbb{R}^8$ has up to 7 generalized curvatures, κ_1 through κ_7.

Not all of these are physically meaningful for every system. In practice, the effective dimensionality of the curve—the number of linearly independent derivatives—is determined by the system dynamics:

Principle 2.3. Dynamic Rank of a Trajectory

For a controlled mechanical system with N DOF and m independent actuators, a generic trajectory has at most $p = 2m$ linearly independent arc-length derivatives. The Frenet–Serret frame therefore has at most $2m$ intrinsically meaningful vectors, and the curve has at most $2m - 1$ independent curvatures.

For a fully actuated system ($m = N$), the trajectory can bend freely in all $2N$ state-space directions. For an underactuated system ($m < N$), the trajectory is geometrically constrained: it lives on a submanifold, and the curvatures in the "forbidden" directions are determined by the passive dynamics.

2.6 Curvature and Torsion in the Language of Control

We now connect the geometric quantities to the dynamics and control of nonlinear systems.

2.6.1 Curvature as a Demand on the Controller

Consider the system $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$ tracking a reference trajectory $\boldsymbol{\gamma}^*(s)$ at instantaneous speed $v(t) = \dot{s}(t)$. Using the chain rule:

$$\dot{\boldsymbol{x}} = v\,\boldsymbol{T}, \qquad \ddot{\boldsymbol{x}} = \dot{v}\,\boldsymbol{T} + v^2\,\kappa\,\boldsymbol{N}. \tag{2.25}$$

The acceleration decomposes into:

- A **tangential component** $\dot{v}\,\boldsymbol{T}$ that changes the speed along the curve.
- A **normal component** $v^2\kappa\,\boldsymbol{N}$ that "steers" the system around the bend.

Definition 2.5. Control Demand Decomposition

Along a reference trajectory $\boldsymbol{\gamma}^*(s)$ traversed at speed v, the instantaneous control demand decomposes as

$$\boldsymbol{u}_{\text{demand}} = \underbrace{\boldsymbol{u}_{\parallel}}_{\text{tangential}} + \underbrace{\boldsymbol{u}_{\perp}}_{\text{normal}}, \tag{2.26}$$

where:

$$\boldsymbol{u}_{\parallel} \propto \dot{v} \qquad \text{(accelerate/decelerate along path)}, \tag{2.27}$$

$$\boldsymbol{u}_{\perp} \propto v^2\,\kappa \qquad \text{(centripetal force to follow curvature)}. \tag{2.28}$$

The normal demand $\boldsymbol{u}_{\perp}$ grows quadratically with speed and linearly with curvature. This is the **curvature cost** of the trajectory.

Example 2.4. Curvature Cost in the Golf Downswing

In the transition from backswing to downswing, a professional golfer reverses the direction of the club in roughly 0.15 seconds. Let us estimate the curvature cost.

At the top of the backswing, the clubhead speed is approximately zero. At impact (0.3 s later), it reaches $\sim 50\,\text{m/s}$. The path through configuration space has a tight reversal—a region of high curvature κ.

Using $a_{\perp} = \kappa v^2$, and noting that mid-downswing $v \approx 25\,\text{m/s}$ with the reversal curvature $\kappa \approx 2\,\text{rad/m}$ (estimated from motion capture data), the centripetal acceleration demand is

$$a_{\perp} \approx 2 \times 25^2 = 1250\ \text{m/s}^2 \approx 127\ g. \tag{2.29}$$

This enormous demand is met not by muscular torque alone, but by the *passive coupling* in the kinematic chain—the very underactuation that the setpoint framework treats as a constraint. The geometry of the trajectory creates the forces needed to execute it.

Remark 2.2. The Orbit Theorem and Achievable Curves

A profound connection exists between the curvature of a trajectory and the *control vector fields* that generate it. The **Orbit Theorem** (also known as **Chow's Theorem**), a central result in geometric control theory as presented by Agrachev and Sachkov [1], states that the orbit generated by iteratively flowing along the control vector fields $\boldsymbol{f}_i$ and their Lie brackets $[\boldsymbol{f}_i, \boldsymbol{f}_j] = \frac{\partial \boldsymbol{f}_i}{\partial \boldsymbol{x}} \boldsymbol{f}_j - \frac{\partial \boldsymbol{f}_j}{\partial \boldsymbol{x}} \boldsymbol{f}_i$ determines the space of achievable curves. For control-affine systems $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}) + \sum u_i \boldsymbol{g}_i(\boldsymbol{x})$, higher-order Lie brackets correspond to higher derivatives of the trajectory in the direction perpendicular to the primary control inputs. The curvature of a trajectory encodes which Lie bracket relationships are being exploited: a highly curved trajectory requires the activation of higher-order commutators in the control Lie algebra. This is the geometric signature of how underactuated systems achieve complex motions despite having fewer inputs than degrees of freedom.

2.6.2 Torsion as Out-of-Plane Complexity

If curvature measures how a trajectory bends *within* its osculating plane, **torsion** measures how the osculating plane itself rotates along the curve. A planar curve has zero torsion everywhere. A helix has constant nonzero torsion. A golf swing—which involves simultaneous rotation in the transverse, sagittal, and frontal planes—has torsion that varies dramatically through the motion.

Definition 2.6. Torsion

For a curve in $\mathbb{R}^n$ with $n \geq 3$, the **torsion** $\tau(s) = \kappa_2(s)$ is the second generalized curvature in the Frenet–Serret equations:

$$\frac{\mathrm{d}\boldsymbol{N}}{\mathrm{d}s} = -\kappa \boldsymbol{T} + \tau \boldsymbol{B}. \tag{2.30}$$

Torsion measures the rate at which the osculating plane rotates about the tangent direction, per unit arc length.

In high-dimensional state spaces ($n \gg 3$), the higher curvatures $\kappa_3, \ldots, \kappa_{p-1}$ measure increasingly subtle geometric twisting. For control design, the first two curvatures (κ and τ) capture the dominant geometric effects, with higher curvatures becoming relevant primarily in highly articulated systems.

Remark 2.3. The Curvature Signature

The ordered set of generalized curvatures $(\kappa_1(s), \kappa_2(s), \ldots, \kappa_{p-1}(s))$ as functions of arc length constitutes the **curvature signature** of the trajectory. By the fundamental theorem of curves, this signature (up to rigid motions) *uniquely determines* the trajectory. Two motions with identical curvature signatures are geometrically congruent—they are the same shape, possibly translated and rotated.

This has a striking implication for motor control: if a golfer can reproduce the curvature signature of their swing, the spatial path is automatically reproduced, regardless of timing.

2.7 Reparameterization and Timing Laws

We now formalize the separation between the *shape* of a trajectory and the *timing* of its execution.

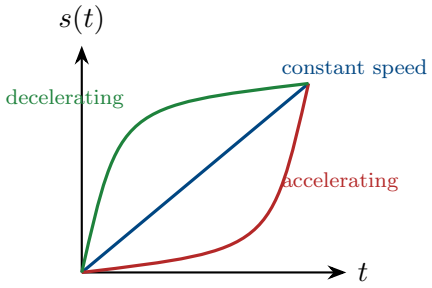
Definition 2.7. Timing Law

Given a reference trajectory $\boldsymbol{\gamma}^*(s)$ parameterized by arc length, a **timing law** is a smooth, monotonically increasing function $s : [0, T] \to [0, L]$ with $s(0) = 0$ and $s(T) = L$. The time-parameterized trajectory is

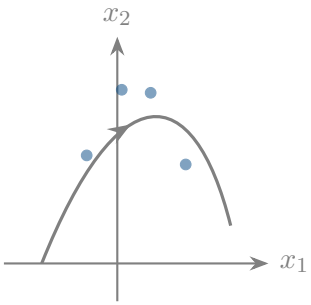
$$\boldsymbol{x}^*(t) = \boldsymbol{\gamma}^*(s(t)). \tag{2.31}$$

The timing law is equivalently specified by the speed profile $v(t) = \dot{s}(t)$ or the phase velocity $v(s) = \mathrm{d}s/\mathrm{d}t$ expressed as a function of arc length.

Different timing laws produce different motions through the same geometric path:



Three timing laws



Same spatial path

2.7.1 The Path–Timing Decomposition of Control

The reparameterization framework leads to a clean decomposition of the control problem:

Principle 2.4. Path–Timing Separation

The trajectory control problem decomposes into two subproblems:

(A) **Path design**: Choose the geometric curve $\boldsymbol{\gamma}^*(s)$ in state space that achieves the task (e.g., delivers the clubhead to the ball with the correct velocity vector).

(B) **Timing design**: Choose the timing law $s(t)$ that balances speed against tracking difficulty, control effort, and robustness.

These two problems have different objective functions and constraints:

	Path design	**Timing design**
Objective	Task performance	Trackability & effort
Key quantity	Curvature $\kappa(s)$	Speed profile $v(s)$
Constraint	Dynamic feasibility	Actuator limits

The interaction between the two is captured by the curvature cost $\kappa(s)\,v(s)^2$, which the timing law must keep within the controller's authority at every point.

Example 2.5. Timing the Downswing

Suppose the downswing path has a curvature profile $\kappa(s)$ with a peak of $\kappa_{\max}$ at the transition point. The maximum centripetal acceleration the controller can provide is $a_{\max}$. Then the speed at the transition is bounded by

$$v_{\text{transition}} \leq \sqrt{\frac{a_{\max}}{\kappa_{\max}}}. \tag{2.32}$$

This is a *geometric speed limit*: no matter how powerful the actuators, the system cannot traverse a tight bend faster than its control authority allows. The timing law must respect this limit, typically by slowing down through high-curvature regions and accelerating through straight sections.

This is precisely what golfers do instinctively. The "pause at the top"—the brief deceleration between backswing and downswing—is not aesthetic affectation. It is the timing law respecting the curvature speed limit at the point of maximum path curvature.

2.8 The Osculating Objects

The Frenet–Serret frame defines a sequence of "best-fit" geometric objects at each point of the curve. These osculating objects provide increasingly refined local approximations that are useful for controller design.

Definition 2.8. Osculating Objects

At a point $\boldsymbol{\gamma}^*(s_0)$ on a curve in $\mathbb{R}^n$:

(i) The **osculating line** is the tangent line: the best linear approximation.

(ii) The **osculating circle** lies in the $(\boldsymbol{T}, \boldsymbol{N})$ plane, has radius $R = 1/\kappa$, and center at $\boldsymbol{\gamma}^*(s_0) + R\,\boldsymbol{N}$. It is the best second-order (circular) approximation.

(iii) The **osculating plane** is the plane spanned by $\boldsymbol{T}$ and $\boldsymbol{N}$. Torsion measures

how fast the curve leaves this plane.

(iv) The **osculating sphere** is the best third-order approximation and accounts for both curvature and torsion.

For trajectory control, the osculating circle is the most immediately useful. It tells us: *if the system were constrained to follow a circle at this point, what circle would it be?* The radius of that circle is the radius of curvature $R = 1/\kappa$, and the center indicates the direction toward which the trajectory bends.

2.9 The Geometry of Trajectory Deviations

We now use the Frenet–Serret frame to analyze what happens when the system deviates from the reference trajectory. This provides the geometric foundation for the transverse linearization of Chapter 4.

2.9.1 Frenet–Serret Coordinates

Given a reference trajectory $\boldsymbol{\gamma}^*(s)$ and the associated Frenet–Serret frame $\{\boldsymbol{e}_1(s), \ldots, \boldsymbol{e}_p(s)\}$, any state $\boldsymbol{x}$ near the trajectory can be decomposed as:

$$\boldsymbol{x} = \boldsymbol{\gamma}^*(s^*) + \sum_{k=2}^{n} \xi_k \, \boldsymbol{e}_k(s^*), \tag{2.33}$$

where $s^* = s^*(\boldsymbol{x})$ is the projection phase (the arc-length parameter of the nearest point on the reference), and $\xi_2, \ldots, \xi_n$ are the **transverse coordinates** measuring deviation in each normal direction.

Definition 2.9. Frenet–Serret Coordinates

The **Frenet–Serret coordinate representation** of a state $\boldsymbol{x}$ relative to a reference trajectory $\boldsymbol{\gamma}^*$ is the tuple

$$(s^*,\ \xi_2,\ \xi_3,\ \ldots,\ \xi_n), \tag{2.34}$$

where s^* is the tangential (along-trajectory) coordinate and $\boldsymbol{\xi} = (\xi_2, \ldots, \xi_n)$ are the transverse (cross-trajectory) coordinates. The transverse distance from Chapter 1 is

$$d_\perp = \|\boldsymbol{\xi}\| = \sqrt{\xi_2^2 + \cdots + \xi_n^2}. \tag{2.35}$$

These coordinates have a powerful property: the reference trajectory itself corresponds to $\boldsymbol{\xi} = \mathbf{0}$, and the trajectory-tracking problem becomes the problem of *regulating* $\boldsymbol{\xi}$ *to zero while* s^* *advances*. This converts the tracking problem into a regulation problem, but in the moving frame rather than a fixed frame.

2.9.2 Curvature-Induced Coupling

There is a subtlety: the Frenet–Serret coordinates are not inertial. Because the frame rotates as it moves along the curve (governed by the Frenet–Serret equations (2.20)), fictitious forces appear in the transverse dynamics. The dominant effect is a **curvature-induced coupling**:

Proposition 2.1. *In Frenet–Serret coordinates, the linearized transverse dynamics about the reference trajectory have the form*

$$\dot{\boldsymbol{\xi}} = \big[\boldsymbol{A}(s) - v^2\boldsymbol{K}(s)\big]\,\boldsymbol{\xi} + \boldsymbol{B}(s)\,\boldsymbol{u}_{fb}, \tag{2.36}$$

where $\boldsymbol{A}(s)$ comes from linearizing the original dynamics, and $\boldsymbol{K}(s)$ is a matrix that depends on the generalized curvatures $\kappa_1(s), \ldots, \kappa_{p-1}(s)$. The term $v^2\boldsymbol{K}(s)$ represents the geometric coupling between the curve's shape and the transverse stability.

This result has a profound implication: *the stability of trajectory tracking depends on the geometry of the trajectory.* A trajectory with low curvature is inherently easier to stabilize than one with high curvature, even if the underlying system dynamics are identical. The curvature modifies the effective dynamics transverse to the path.

2.10 Fundamental Theorem and Uniqueness

We close this chapter with a theorem that is central to the trajectory-first philosophy.

Theorem 2.3 (Fundamental Theorem of Curves)**.** *Let $\kappa_1(s), \ldots, \kappa_{p-1}(s)$ be smooth functions on $[0, L]$ with $\kappa_k(s) > 0$ for $k = 1, \ldots, p-2$. Then there exists a curve $\boldsymbol{\gamma} : [0, L] \to \mathbb{R}^n$, parameterized by arc length, whose generalized curvatures are exactly $\kappa_1, \ldots, \kappa_{p-1}$. Moreover, this curve is unique up to rigid motions (translations and rotations) of $\mathbb{R}^n$.*

This theorem says: **the curvature signature is the trajectory.** Two trajectories with the same curvature functions are identical in shape. This has a practical corollary for control:

Corollary 2.4. *To specify a reference trajectory, it suffices to specify the generalized curvatures as functions of arc length, together with an initial position and orientation. This representation is:*

(i) Compact*: $p-1$ scalar functions instead of n coordinate functions.*

(ii) Intrinsic*: independent of coordinate choice.*

(iii) Timing-independent*: the trajectory shape is fully determined without specifying when each point is reached.*

Example 2.6. Curvature Signature of a Golf Swing

A simplified 2-DOF model of the golf swing (arm + club) has a 4-dimensional state space $(\theta_{\text{arm}}, \theta_{\text{club}}, \dot{\theta}_{\text{arm}}, \dot{\theta}_{\text{club}})$. The trajectory through this space has up to 3 generalized curvatures: $\kappa_1(s)$, $\kappa_2(s)$, and $\kappa_3(s)$.
The curvature signature reveals the structure of the swing:

- $\kappa_1(s)$ peaks at the transition (the "pause at the top").
- $\kappa_2(s)$ peaks during the release, where the swing plane rotates.
- $\kappa_3(s)$ is small for a well-executed swing (the motion is approximately confined to a 3D subspace of $\mathbb{R}^4$).

A coach's instruction to "flatten your swing plane" is, in geometric terms, a request to reduce $\kappa_2(s)$ during a specific phase of the motion—a precise modification to the curvature signature.

2.11 Summary

This chapter has established the geometric language we will use throughout the book:

Concept	**Control significance**
Arc length s	The natural "distance along the path"—separates shape from timing
Curvature $\kappa(s)$	Determines the centripetal control demand $a_\perp = \kappa v^2$
Torsion $\tau(s)$	Measures out-of-plane complexity; relates to multi-joint coordination
Frenet–Serret frame	Provides the natural moving coordinate system for decomposing deviations into tangential and transverse components
Curvature signature	Uniquely specifies the trajectory shape, independent of timing and coordinates
Path–timing separation	Decomposes control into geometric path design and temporal speed planning

The key takeaway is that a trajectory is not just "a function of time." It is a geometric object with intrinsic structure, and this structure directly determines the difficulty and requirements of controlling it. In Chapter 3, we will see that the state spaces of mechanical

systems are not flat $\mathbb{R}^n$—they are curved manifolds with their own geometry. The interplay between the geometry of the *curve* and the geometry of the *space* will produce the richest and most practically important results of this book.

2.12 Exercises

2.1. (Computation.) Compute the curvature and torsion of the curve $\boldsymbol{\gamma}(t) = (t, t^2, t^3)$ at $t = 0$ and $t = 1$. At which point is the curve "harder to track" at constant speed?

2.2. (Arc length.) The logarithmic spiral $\boldsymbol{\gamma}(t) = e^{at}(\cos t,\ \sin t)$ with $a > 0$ is a model for certain biological growth patterns.

(a) Find the arc-length function $s(t)$.

(b) Show that the curvature is $\kappa = 1/(e^{at}\sqrt{1+a^2})$ and interpret what this means about tracking difficulty as the spiral grows.

(c) What timing law $s(t)$ would maintain constant centripetal acceleration?

2.3. (Frenet–Serret.) For the curve $\boldsymbol{\gamma}(s) = (\cos(s/\sqrt{2}),\ \sin(s/\sqrt{2}),\ s/\sqrt{2})$ (a helix parameterized by arc length):

(a) Verify that $\|\boldsymbol{\gamma}'(s)\| = 1$.

(b) Compute the Frenet–Serret frame $\{\boldsymbol{T}, \boldsymbol{N}, \boldsymbol{B}\}$.

(c) Compute κ and τ and verify the Frenet–Serret equations.

2.4. (Path–timing separation.) A robot end-effector must trace the path $\boldsymbol{p}(s) = (s,\ \sin(\pi s),\ 0)$ for $s \in [0, 2]$ (one full sine wave).

(a) Compute $\kappa(s)$ and identify the points of maximum curvature.

(b) If the maximum centripetal acceleration is $a_{\max} = 10\ \mathrm{m/s^2}$, find the speed limit $v_{\max}(s)$ along the path.

(c) Design a timing law $s(t)$ that traverses the path in minimum time while respecting the speed limit. Sketch $v(s)$ and $s(t)$.

2.5. (Conceptual.) Consider two trajectories in $\mathbb{R}^4$ with identical curvature signatures $(\kappa_1(s), \kappa_2(s), \kappa_3(s))$ but different initial conditions.

(a) By the fundamental theorem, how are these trajectories related?

(b) If you designed a tracking controller for one, would it work for the other? Under what conditions?

(c) Relate this to the idea that a well-learned golf swing can be started from slightly different address positions.

2.6. (Numerical project.) Using motion capture data (or simulated data from a double pendulum), compute the curvature signature of a swing-like motion:

(a) Numerically estimate $s(t)$ by integrating $\|\dot{\boldsymbol{x}}(t)\|$.

(b) Compute $\kappa_1(s)$ using Eq. (2.13).

(c) Identify the phases of the motion (backswing, transition, downswing, impact) from features of the curvature profile.

(d) Plot the curvature cost $\kappa(s)\, v(s)^2$ and identify the most demanding phase of the motion.

The shape of a river is written in its curvature.
The story of a motion is written the same way.

Learn to read the curvature, and you read the motion.

Chapter 3

Configuration Manifolds

The map is not the territory. The coordinate chart is not the manifold. Every time you write θ for an angle, you are lying—just a little.

In Plain Language 3.1. Angles Are Not Numbers

When we measure the position of a car on a straight road, we use a regular number: 0 miles, 1 mile, 2 miles, stretching on forever. But think about a spinning wheel or a robotic joint. If you rotate it by 360 degrees, it ends up in the exact same physical position as 0 degrees. Regular numbers don't work like this ($0 \neq 360$). If we force angles to act like regular numbers, our math gets tangled up in "singularities" or sudden jumps, confusing the control system. To fix this, we have to admit that the space of possible rotations is more like the surface of a sphere or a donut than a flat sheet of paper. This chapter explores how to do math on these curved spaces, known as "manifolds."

3.1 The Lie That Coordinates Tell

In Chapter 2 we developed the geometry of curves in $\mathbb{R}^n$. This was a necessary starting point, but it was also a simplification. The state spaces of real mechanical systems are *not* $\mathbb{R}^n$.

Consider the simplest possible rotating joint: a hinge. Its configuration is an angle θ, which we habitually write as a real number. But an angle is not a real number. The configuration $\theta = 0$ and the configuration $\theta = 2\pi$ are *the same physical state*. No point in $\mathbb{R}$ has this property. The true configuration space of a hinge is the *circle* S^1—a one-dimensional manifold that is locally like $\mathbb{R}$ but globally has a completely different topology.

This is not pedantry. It is the source of real engineering failures.

Warning 3.1. The Gimbal Lock Catastrophe

If we represent orientation using three Euler angles $(\phi, \theta, \psi) \in \mathbb{R}^3$, we encounter **gimbal lock**: configurations where the representation degenerates and loses a degree of

freedom. At $\theta = \pm\pi/2$, the axes for ϕ and ψ align, making the system appear to have only 2 DOF when it actually has 3.
This is not a hardware problem—it is a *coordinate singularity*, an artifact of forcing a non-Euclidean space (SO(3)) into Euclidean coordinates ($\mathbb{R}^3$). Apollo 13's inertial measurement unit experienced gimbal lock operationally. Every modern spacecraft avoids Euler angles for precisely this reason.
The lesson for control: **if you use the wrong coordinates, your controller will have singularities that the physical system does not.**

Key Idea 3.1. The Manifold Imperative

The configuration space of a mechanical system is a *smooth manifold* $\mathcal{Q}$—a space that is locally like $\mathbb{R}^n$ but may have nontrivial global topology. Control theory built on $\mathbb{R}^n$ must be replaced by control theory built on $\mathcal{Q}$ whenever:

(i) Joints allow full rotation (wrapping around S^1 or SO(3)).

(ii) The trajectory traverses a significant fraction of the configuration space.

(iii) Singularity-free representation is required for robustness.

For the golf swing—which involves large rotations in multiple joints simultaneously—all three conditions are met.

3.2 Smooth Manifolds: The Minimum You Need

We develop only as much manifold theory as the control engineer needs. The reader seeking a comprehensive treatment should consult Lee (2012) or Abraham & Marsden (1978).

In Plain Language 3.2. Mathematical Reminder: The Foundations Re-applied

In Volume 0, we introduced Manifolds and Tangent Spaces as abstract geometric concepts to explain configuration and rotation. Here in Volume II, we mathematically formalize them to build robust control algorithms. Remember the core definitions:

1. **Manifold ($\mathcal{Q}$):** The curved, global "position" space of the robot.
2. **Tangent Space ($T_q\mathcal{Q}$):** The perfectly flat, linear "velocity" space attached to one specific position.
3. **Tangent Bundle ($T\mathcal{Q}$):** The total **State Space** combining all positions and all their velocities.

Our motor torques compute inside the flat tangent space. We rely on differential geometry to project those flat velocities back onto the curved physical reality.

Definition 3.1. Smooth Manifold

A **smooth manifold** $\mathcal{M}$ of dimension n is a set together with a collection of **coordinate charts** $\{(U_\alpha, \varphi_\alpha)\}$ such that:

(i) Each $U_\alpha \subseteq \mathcal{M}$ is an open set, and the U_α cover $\mathcal{M}$.

(ii) Each $\varphi_\alpha : U_\alpha \to \mathbb{R}^n$ is a homeomorphism onto an open subset of $\mathbb{R}^n$.

(iii) Where two charts overlap, the **transition maps** $\varphi_\beta \circ \varphi_\alpha^{-1}$ are smooth (C^∞) functions from $\mathbb{R}^n$ to $\mathbb{R}^n$.

The key intuition is: a manifold is a space where you can *locally* use coordinates, but no single coordinate system works *globally*. You need an atlas of overlapping charts, like a road atlas needs multiple pages to cover a country.

Example 3.1. The Circle S^1

The circle $S^1 = \{(\cos\theta, \sin\theta) : \theta \in \mathbb{R}\}$ is a 1-dimensional manifold. It cannot be covered by a single coordinate chart: any angle parameterization $\theta \in [0, 2\pi)$ has a discontinuity at $\theta = 0 = 2\pi$. We need at least two charts:

$$U_1 = S^1 \setminus \{(-1, 0)\}, \qquad \varphi_1(p) = \theta \in (-\pi, \pi), \tag{3.1}$$

$$U_2 = S^1 \setminus \{(1, 0)\}, \qquad \varphi_2(p) = \theta \in (0, 2\pi). \tag{3.2}$$

On the overlap $U_1 \cap U_2$, the transition map is either the identity or a shift by 2π—both smooth.
For a hinge joint, this means: no single angle variable θ can represent all configurations without a discontinuity. If the joint rotates past the discontinuity, any controller using that angle will experience a "jump"—a coordinate artifact, not a physical event.

3.2.1 Tangent Spaces and Tangent Vectors

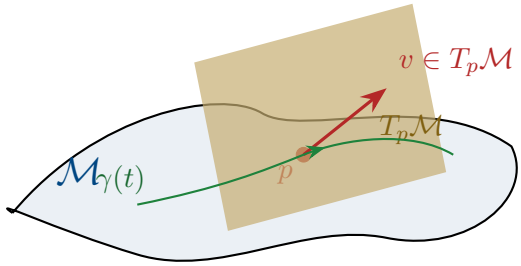
On a manifold, there is no global notion of "direction." Velocity is a *local* concept, defined at each point.

Definition 3.2. Tangent Space

The **tangent space** $T_p\mathcal{M}$ at a point $p \in \mathcal{M}$ is the vector space of all possible velocity vectors at p. If $\mathcal{M}$ has dimension n, then $T_p\mathcal{M} \cong \mathbb{R}^n$ as a vector space, but each point has its *own* tangent space.
Given a curve $\gamma : (-\varepsilon, \varepsilon) \to \mathcal{M}$ with $\gamma(0) = p$, the tangent vector $\dot{\gamma}(0)$ is an element of $T_p\mathcal{M}$.

Definition 3.3. Tangent Bundle

The **tangent bundle** $T\mathcal{M} = \bigsqcup_{p\in\mathcal{M}} T_p\mathcal{M}$ is the collection of all tangent spaces, one for each point. It is itself a smooth manifold of dimension $2n$. A point in $T\mathcal{M}$ is a pair (p, v) where $p \in \mathcal{M}$ and $v \in T_p\mathcal{M}$.
For a mechanical system with configuration manifold $\mathcal{Q}$, the **state space is the tangent bundle** $\mathcal{X} = T\mathcal{M}$. A state is a pair $(\boldsymbol{q}, \dot{\boldsymbol{q}})$—a configuration and a velocity.



3.3 The Configuration Manifolds of Joints

Every mechanical joint constrains relative motion between two rigid bodies. The set of allowed relative configurations forms a manifold whose topology depends on the joint type.

Joint type	**DOF**	**Manifold**	**Topology**
Revolute (hinge)	1	S^1	Circle
Prismatic (slider)	1	$\mathbb{R}$	Line (or interval)
Universal	2	S^2	2-sphere
Ball-and-socket	3	$\mathrm{SO}(3)$	Rotation group
Planar	3	$\mathrm{SE}(3)\vert_{\mathrm{2D}} = \mathbb{R}^2 \times S^1$	Plane + rotation
Free body	6	$\mathrm{SE}(3) = \mathbb{R}^3 \rtimes \mathrm{SO}(3)$	Rigid motions of 3-space

The configuration space of a multi-body system is the *product* of individual joint manifolds (modulo kinematic constraints):

Definition 3.4. Product Configuration Space

For a serial chain of k joints with individual configuration manifolds $\mathcal{M}_1, \ldots, \mathcal{M}_k$, the total configuration space is

$$\mathcal{Q} = \mathcal{M}_1 \times \mathcal{M}_2 \times \cdots \times \mathcal{M}_k. \tag{3.3}$$

For example, a robot arm with three revolute joints has $\mathcal{Q} = S^1 \times S^1 \times S^1 = \mathbb{T}^3$, the 3-dimensional torus.

Example 3.2. The Human Shoulder

The glenohumeral (shoulder) joint is approximately a ball-and-socket joint with configuration manifold SO(3). Combined with the 1-DOF hinge at the elbow (S^1) and the 1-DOF pronation/supination of the forearm (S^1), the arm has configuration space

$$\mathcal{Q}_{\text{arm}} = \text{SO}(3) \times S^1 \times S^1. \tag{3.4}$$

This is a 5-dimensional manifold with the topology of $\text{SO}(3) \times \mathbb{T}^2$. No set of 5 angle variables can parameterize this space without singularities.

3.4 The Rotation Group SO(3)

The rotation group SO(3) appears wherever a joint allows full 3D rotation. It is the single most important non-Euclidean manifold in mechanical control, and its properties drive many of the challenges we will encounter.

Definition 3.5. The Special Orthogonal Group

SO(3) is the group of 3×3 real orthogonal matrices with determinant $+1$:

$$\text{SO}(3) = \{\boldsymbol{R} \in \mathbb{R}^{3\times 3} : \boldsymbol{R}^\top \boldsymbol{R} = \boldsymbol{I},\ \det(\boldsymbol{R}) = 1\}. \tag{3.5}$$

It is a 3-dimensional compact Lie group. As a manifold, it is diffeomorphic to the real projective space $\mathbb{R}P^3$.

3.4.1 Why Euler Angles Fail

The most common parameterization of SO(3) uses three Euler angles (ϕ, θ, ψ), defining the rotation matrix as a product of three elemental rotations. This parameterization has two fundamental problems:

P1. Singularities. Every 3-parameter representation of SO(3) has at least one singularity. For the ZYX convention, gimbal lock occurs at $\theta = \pm\pi/2$. This is a topological necessity, not a flaw in the choice of convention: SO(3) is not homeomorphic to any open subset of $\mathbb{R}^3$.

P2. Non-uniqueness. Euler angles are periodic: (ϕ, θ, ψ) and $(\phi + 2\pi, \theta, \psi)$ represent the same rotation. Near the singularity, drastically different angle triples can represent nearly identical orientations, creating numerical chaos in controllers.

Principle 3.1. Representation Principle for Rotations

For trajectory control on SO(3), use one of:

(a) **Rotation matrices** $\boldsymbol{R} \in \mathbb{R}^{3\times 3}$: 9 parameters with 6 orthogonality constraints. Singularity-free but redundant.

(b) **Unit quaternions** $\boldsymbol{q} \in S^3 \subset \mathbb{R}^4$: 4 parameters with 1 unit-norm constraint. Singularity-free, minimal redundancy, excellent for interpolation and computation.

(c) **Exponential coordinates** (axis-angle or Lie algebra): 3 parameters with a singularity at $\|\boldsymbol{\omega}\| = \pi$, but natural for small rotations and linearization.

Never use Euler angles for control of large-rotation trajectories.

3.4.2 The Lie Algebra $\mathfrak{so}(3)$

The tangent space of SO(3) at the identity is the **Lie algebra** $\mathfrak{so}(3)$—the space of 3×3 skew-symmetric matrices:

$$\mathfrak{so}(3) = \{\boldsymbol{\Omega} \in \mathbb{R}^{3\times 3} : \boldsymbol{\Omega}^\top = -\boldsymbol{\Omega}\} = \left\{ \widehat{\boldsymbol{\omega}} := \begin{pmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{pmatrix} : \boldsymbol{\omega} \in \mathbb{R}^3 \right\}. \tag{3.6}$$

The "hat map" $(\cdot)^\wedge : \mathbb{R}^3 \to \mathfrak{so}(3)$ and its inverse "vee map" $(\cdot)^\vee : \mathfrak{so}(3) \to \mathbb{R}^3$ identify the Lie algebra with $\mathbb{R}^3$ (the space of angular velocity vectors).

Definition 3.6. Exponential Map

The **exponential map** $\exp : \mathfrak{so}(3) \to \mathrm{SO}(3)$ connects the Lie algebra to the Lie group:

$$\boldsymbol{R} = \exp(\widehat{\boldsymbol{\omega}}) = \boldsymbol{I} + \frac{\sin\theta}{\theta}\widehat{\boldsymbol{\omega}} + \frac{1-\cos\theta}{\theta^2}\widehat{\boldsymbol{\omega}}^2, \tag{3.7}$$

where $\theta = \|\boldsymbol{\omega}\|$. This is the **Rodrigues formula**. Geometrically, $\boldsymbol{R}$ is a rotation by angle θ about the axis $\boldsymbol{\omega}/\theta$.
The inverse map $\log : \mathrm{SO}(3) \to \mathfrak{so}(3)$ is well-defined for rotations with angle $\theta < \pi$ and gives the **rotation vector** representation.

The exponential map is the bridge between the linear world (the Lie algebra, where we can add and scale) and the nonlinear world (the Lie group, where the actual dynamics live). This bridge is the central tool for linearization on manifolds.

Remark 3.1. Lie Algebras and Controllability on Manifolds

The Lie algebra of a Lie group is far more than an abstract algebraic structure; it is the fundamental object in geometric control theory. Agrachev and Sachkov [1] show

that the controllability of a nonlinear system on a manifold is determined entirely by properties of the Lie algebra generated by the control vector fields and their iterated Lie brackets. For the golf swing, the control vector fields correspond to muscular torques (shoulder, elbow, hip). Their Lie bracket [shoulder control, arm control] corresponds to the inertial coupling between links—couplings the golfer cannot directly command, but can exploit. Higher-order brackets unlock access to otherwise unreachable configurations in the presence of underactuation. Thus, the geometric structure of a configuration manifold (e.g., $SO(3)$ for torso rotations) directly constrains which motions are achievable via feedback control. The Lie algebra is both the space of directions we can push the system, and the complete generator of everything we can ultimately reach.

3.5 The Golf Swing Configuration Space

We now apply the manifold framework to our running example. The result will show exactly why Euclidean control theory is insufficient for the golf swing.

3.5.1 A Simplified Kinematic Model

Consider a 4-segment model of the golf swing:

(i) **Torso**: rotates relative to ground via the spine. Modeled as a 3-DOF ball joint: $\mathcal{M}_1 = \mathrm{SO}(3)$.

(ii) **Lead arm**: rotates relative to the torso via the shoulder. Modeled as a 3-DOF ball joint: $\mathcal{M}_2 = \mathrm{SO}(3)$.

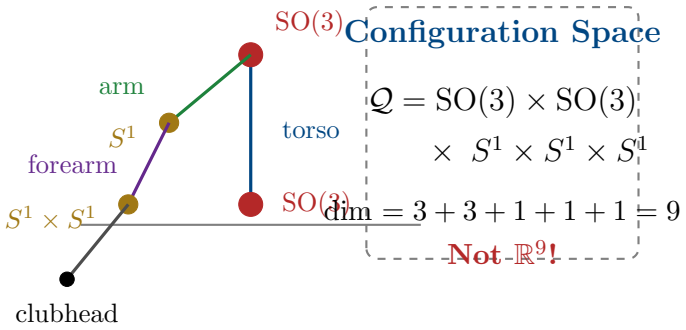
(iii) **Forearm/hands**: rotates relative to the upper arm via the elbow (1 DOF) and wrist flexion (1 DOF): $\mathcal{M}_3 = S^1 \times S^1$.

(iv) **Club**: rotates relative to the hands via wrist cock/uncock (1 DOF): $\mathcal{M}_4 = S^1$.

The total configuration space is:

$$\mathcal{Q}_{\text{golf}} = \mathrm{SO}(3) \times \mathrm{SO}(3) \times S^1 \times S^1 \times S^1. \tag{3.8}$$

This is a **9-dimensional manifold**. Its topology is $\mathrm{SO}(3) \times \mathrm{SO}(3) \times \mathbb{T}^3$—decidedly not $\mathbb{R}^9$.



3.5.2 Why This Matters for Control

The state space (tangent bundle) has dimension $2 \times 9 = 18$. A trajectory through this space is a curve on an 18-dimensional manifold. The consequences for control are immediate:

C1. No global linearization. You cannot write $\boldsymbol{x} = \boldsymbol{x}^* + \delta\boldsymbol{x}$ with $\delta\boldsymbol{x} \in \mathbb{R}^{18}$, because the "+" operation is not defined globally on the manifold. Linearization must be done in the *tangent space*, using the exponential map.

C2. Error is not a vector. The "error" between two configurations $\boldsymbol{q}_1, \boldsymbol{q}_2 \in \mathcal{Q}$ is not $\boldsymbol{q}_2 - \boldsymbol{q}_1$ (subtraction is undefined on manifolds). The correct notion of error is the *logarithmic map*: $\boldsymbol{e} = \log(\boldsymbol{q}_1^{-1}\boldsymbol{q}_2)$, which lives in the Lie algebra.

C3. Geodesics replace straight lines. The "shortest path" between two configurations is not a straight line in any coordinate system—it is a *geodesic* on the manifold, determined by the Riemannian metric (mass-inertia matrix).

C4. Parallel transport replaces vector addition. To compare tangent vectors (velocities, errors) at different points on the trajectory, we must *transport* them along the manifold using the connection. Simply subtracting velocity vectors at different times is mathematically meaningless.

Warning 3.2. Euler Angle Catastrophe in Golf Modeling

Many biomechanics studies parameterize the golf swing using Euler angles for each joint. For the shoulder joint (SO(3)), this means three angles $(\phi_s, \theta_s, \psi_s)$. But during the backswing, the shoulder passes through configurations near gimbal lock ($\theta_s \approx \pm\pi/2$).
In these regions:

- The Jacobian mapping angular velocities to Euler angle rates becomes singular.
- Small physical rotations produce large jumps in angle coordinates.
- Any controller computing torques from angle errors will produce *infinite* commanded torques at the singularity.

The fix is not to "choose a better Euler angle convention" (all conventions have singularities somewhere). The fix is to work on SO(3) directly, using rotation matrices or quaternions.

3.6 Riemannian Metrics and the Mass Matrix

A manifold by itself has no notion of distance, angle, or length. To measure these, we need a **Riemannian metric**—a smooth choice of inner product on each tangent space.

Definition 3.7. Riemannian Metric

A **Riemannian metric** on a manifold $\mathcal{M}$ is a smooth assignment of an inner product $\langle \cdot, \cdot \rangle_p$ on each tangent space $T_p\mathcal{M}$. In local coordinates $\boldsymbol{q} = (q_1, \ldots, q_n)$, the metric is represented by a positive-definite matrix $\boldsymbol{G}(\boldsymbol{q})$:

$$\langle \boldsymbol{v}, \boldsymbol{w} \rangle_{\boldsymbol{q}} = \boldsymbol{v}^\top \boldsymbol{G}(\boldsymbol{q})\, \boldsymbol{w}, \qquad \boldsymbol{v}, \boldsymbol{w} \in T_{\boldsymbol{q}}\mathcal{M}. \tag{3.9}$$

For mechanical systems, nature provides a canonical Riemannian metric: the **kinetic energy metric**.

Principle 3.2. The Mass Matrix as Riemannian Metric

For a mechanical system with generalized coordinates $\boldsymbol{q}$ and mass-inertia matrix $\boldsymbol{M}(\boldsymbol{q})$, the kinetic energy

$$T = \frac{1}{2} \dot{\boldsymbol{q}}^\top \boldsymbol{M}(\boldsymbol{q})\, \dot{\boldsymbol{q}} \tag{3.10}$$

defines a Riemannian metric $\boldsymbol{G} = \boldsymbol{M}(\boldsymbol{q})$ on $\mathcal{Q}$. This metric has deep physical significance:

(i) **Distance** between configurations is measured in "units of kinetic energy"—configurations connected by high-inertia motions are "far apart."

(ii) **Geodesics** (shortest paths) under this metric are the trajectories of the *unforced* system—free motions with no applied torques.

(iii) **Arc length** from Chapter 2, computed with this metric, naturally weights each DOF by its effective inertia.

Example 3.3. Inertia-Weighted Distance in the Swing

Consider two configurations that differ by either:

(a) A 10° rotation of the torso (high inertia, $I \approx 12\ \text{kg} \cdot \text{m}^2$), or

(b) A 10° rotation of the wrist (low inertia, $I \approx 0.03\ \text{kg} \cdot \text{m}^2$).

In Euclidean coordinates, both represent "10° of change." In the kinetic energy metric, the torso rotation is $\sqrt{12/0.03} = 20$ times "farther" than the wrist rotation. This correctly reflects the physical reality: rotating the torso requires 400 times more energy than rotating the wrist by the same angle.
The curvature of a trajectory measured in this metric therefore naturally captures the *physical* difficulty of the motion, not merely the angular change.

3.6.1 Geodesics as Natural Trajectories

Definition 3.8. Geodesic

A **geodesic** on a Riemannian manifold $(\mathcal{M}, \boldsymbol{G})$ is a curve that locally minimizes arc length. Geodesics satisfy the **geodesic equation**:

$$\ddot{q}^i + \sum_{j,k} \Gamma^i_{jk}(\boldsymbol{q})\, \dot{q}^j\, \dot{q}^k = 0, \tag{3.11}$$

where Γ^i_{jk} are the **Christoffel symbols** of the metric $\boldsymbol{G}$.

The remarkable fact is that the geodesic equation (3.11) is *exactly* the equation of motion for the unforced mechanical system $\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} = \boldsymbol{0}$. The Coriolis and centrifugal terms are precisely the Christoffel symbol terms. Free motions of mechanical systems are geodesics on the configuration manifold equipped with the kinetic energy metric.

Principle 3.3. Geodesics and Passive Dynamics

For trajectory design, geodesics represent the "easiest" paths—the motions the system would naturally follow with no applied forces. A well-designed trajectory stays close to geodesics where possible, deviating only where task requirements (e.g., impact conditions) demand it.
In the golf swing, the follow-through is nearly geodesic: after impact, the golfer relaxes and the body follows its natural, inertia-weighted path through configuration space. The downswing deviates significantly from the geodesic because the task (delivering maximum clubhead speed at impact) requires active torques to redirect the motion.

3.7 Curves on Manifolds Revisited

With the manifold framework in place, we can extend every concept from Chapter 2 to non-Euclidean configuration spaces.

3.7.1 Covariant Differentiation

On $\mathbb{R}^n$, we differentiate vector fields by taking component-wise derivatives. On a manifold, this does not work—the tangent spaces at different points are different vector spaces, so we cannot simply subtract vectors at different points.

Definition 3.9. Covariant Derivative

The **covariant derivative** (or **Levi-Civita connection**) ∇ is the unique way to differentiate vector fields along curves on a Riemannian manifold that:

(i) Is compatible with the metric: $\frac{\mathrm{d}}{\mathrm{d}t}\langle \boldsymbol{V}, \boldsymbol{W}\rangle = \langle \nabla_{\dot\gamma}\boldsymbol{V}, \boldsymbol{W}\rangle + \langle \boldsymbol{V}, \nabla_{\dot\gamma}\boldsymbol{W}\rangle$.

(ii) Is torsion-free: $\nabla_{\boldsymbol{V}}\boldsymbol{W} - \nabla_{\boldsymbol{W}}\boldsymbol{V} = [\boldsymbol{V}, \boldsymbol{W}]$.

In local coordinates, the covariant derivative of a vector field $\boldsymbol{V}$ along a curve $\gamma(t)$ is

$$(\nabla_{\dot\gamma}\boldsymbol{V})^i = \dot V^i + \sum_{j,k} \Gamma^i_{jk}\,\dot\gamma^j\,V^k. \tag{3.12}$$

The covariant derivative replaces the ordinary derivative in all of our Chapter 2 formulas. The generalized curvatures, the Frenet–Serret frame, and the transverse decomposition all carry over, with $\mathrm{d}/\mathrm{d}s$ replaced by $\nabla_{\dot\gamma}$.

3.7.2 Manifold Curvature of Trajectories

Definition 3.10. Geodesic Curvature

The **geodesic curvature** of a curve $\gamma(s)$ on a Riemannian manifold, parameterized by arc length, is

$$\kappa_g(s) = \left\|\nabla_{\dot\gamma}\dot\gamma\right\|. \tag{3.13}$$

This measures how much the curve deviates from a geodesic. A geodesic has $\kappa_g = 0$ everywhere.

Geodesic curvature is the manifold generalization of the curvature from Chapter 2, and it has the same control interpretation: *geodesic curvature measures the force per unit mass needed to follow the curve*, beyond what the natural dynamics would provide for free.

Proposition 3.1. *The control force required to track a trajectory $\gamma^*(s)$ at speed v on a configuration manifold with metric $\boldsymbol{M}(\boldsymbol{q})$ has magnitude*

$$\|\boldsymbol{\tau}_\perp\| = \kappa_g(s)\,v^2, \tag{3.14}$$

where $\|\cdot\|$ is measured in the dual (force) metric. Comparing with the Euclidean formula $a_\perp = \kappa v^2$ from Chapter 2: the Euclidean curvature κ is replaced by the geodesic curvature κ_g, which accounts for the curvature of the manifold itself.

This is a deeper version of the Curvature–Authority Principle. The control demand is reduced along directions where the manifold's own curvature "helps"—i.e., where the natural dynamics carry the system in the desired direction. The control demand increases where the trajectory fights the natural dynamics.

Example 3.4. Geodesic Curvature of the Downswing

During the downswing, the trajectory has two sources of curvature:

(a) **Euclidean curvature**: the path bends in coordinate space (the club changes direction).

(b) **Manifold curvature**: the Christoffel symbol terms ($\Gamma^i_{jk}\dot q^j \dot q^k$) represent Coriolis

and centrifugal effects that curve the natural trajectories.

The geodesic curvature κ_g is the *difference*. When the Coriolis/centrifugal terms push the system in the direction the trajectory wants to go (as happens during the wrist release, where centrifugal acceleration "unhinges" the club), κ_g is *less* than the Euclidean κ. The passive dynamics are doing work for free.
This is the geometric explanation for the "free speed" of the golf swing: a well-designed trajectory aligns with the manifold's geodesics during the speed-critical phase, exploiting the natural dynamics rather than fighting them.

3.8 Error Metrics on Manifolds

We now address the critical question: how do we measure tracking error when the state space is a manifold?

3.8.1 The Logarithmic Error

Definition 3.11. Configuration Error on a Lie Group

For two configurations $\boldsymbol{q}_1, \boldsymbol{q}_2 \in \mathcal{Q}$ where $\mathcal{Q}$ is a Lie group, the **error** is

$$\boldsymbol{e} = \log(\boldsymbol{q}_1^{-1}\, \boldsymbol{q}_2) \in \text{Lie algebra}. \tag{3.15}$$

For the rotation group SO(3), this gives

$$\boldsymbol{e}_R = \log(\boldsymbol{R}_1^\top \boldsymbol{R}_2)^\vee \in \mathbb{R}^3, \tag{3.16}$$

which is the rotation vector needed to go from $\boldsymbol{R}_1$ to $\boldsymbol{R}_2$. This error is zero if and only if $\boldsymbol{q}_1 = \boldsymbol{q}_2$, and it varies smoothly with both arguments (away from the cut locus at $\|\boldsymbol{e}\| = \pi$).

Remark 3.2. Why Not Just Subtract Quaternions?

Given two unit quaternions $\boldsymbol{q}_1$ and $\boldsymbol{q}_2$, a tempting error metric is $\boldsymbol{e} = \boldsymbol{q}_2 - \boldsymbol{q}_1$. But this lives in $\mathbb{R}^4$, not in the tangent space of S^3. Worse, it violates the double-cover property: $\boldsymbol{q}$ and $-\boldsymbol{q}$ represent the same rotation, so the error $\boldsymbol{q}_2 - \boldsymbol{q}_1$ and $-\boldsymbol{q}_2 - \boldsymbol{q}_1$ should be "the same," but they point in opposite directions.
The logarithmic error correctly handles the group structure and produces an error vector in the Lie algebra that has a clear physical interpretation (the angular velocity needed to correct the error in unit time).

3.8.2 The Trajectory Tube on a Manifold

The trajectory tube from Chapter 1 now generalizes naturally:

Definition 3.12. Manifold Trajectory Tube

Let $\boldsymbol{\gamma}^* : [0, T] \to \mathcal{Q}$ be a reference trajectory on a Riemannian manifold $(\mathcal{Q}, \boldsymbol{G})$ and let $\varepsilon : [0, T] \to \mathbb{R}_{>0}$ be a tube radius function. The **manifold trajectory tube** is

$$\mathcal{T}(t) = \{\, \boldsymbol{q} \in \mathcal{Q} : d_{\boldsymbol{G}}(\boldsymbol{q}, \boldsymbol{\gamma}^*(t)) \leq \varepsilon(t) \,\}, \tag{3.17}$$

where $d_{\boldsymbol{G}}$ is the **geodesic distance**—the length of the shortest geodesic connecting $\boldsymbol{q}$ and $\boldsymbol{\gamma}^*(t)$.

The geodesic distance correctly accounts for the topology of the configuration space. On SO(3), the maximum distance between any two orientations is π (a half-rotation), and the tube wraps properly around the group.

3.9 Equations of Motion on Manifolds

The Euler–Lagrange equations on a manifold take a coordinate-free form that reveals the geometric structure:

Theorem 3.1 (Lagrangian Mechanics on Manifolds)**.** *For a mechanical system on a configuration manifold $(\mathcal{Q}, \boldsymbol{M})$ with potential energy $V : \mathcal{Q} \to \mathbb{R}$ and generalized forces $\boldsymbol{\tau}$, the equations of motion are*

$$\boldsymbol{M}(\boldsymbol{q})\, \nabla_{\dot{\boldsymbol{q}}} \dot{\boldsymbol{q}} = -\operatorname{grad} V + \boldsymbol{B}\, \boldsymbol{u}, \tag{3.18}$$

where $\nabla_{\dot{\boldsymbol{q}}} \dot{\boldsymbol{q}}$ is the covariant acceleration and $\operatorname{grad} V$ is the Riemannian gradient of the potential energy.

In local coordinates, this reduces to the familiar $\boldsymbol{M}\ddot{\boldsymbol{q}} + \boldsymbol{C}\dot{\boldsymbol{q}} + \boldsymbol{g} = \boldsymbol{B}\boldsymbol{u}$, but the coordinate-free form (3.18) makes clear that:

- The Coriolis matrix $\boldsymbol{C}$ is not a separate "term"—it is the Christoffel symbol contribution to the covariant derivative.
- The equation has the same form on any manifold, regardless of coordinate choice.
- The "centrifugal and Coriolis forces" are geometric artifacts of the manifold's curvature, not real forces. On a flat manifold ($\mathbb{R}^n$), they vanish.

3.10 Practical Implications for Controller Implementation

We conclude with concrete guidance for implementing trajectory controllers on manifold-valued configuration spaces.

Principle 3.4. Manifold-Aware Controller Design

A trajectory tracking controller for a system on $\mathcal{Q}$ must:

(i) **Compute errors using the logarithmic map**, not coordinate subtraction:

$$\boldsymbol{e}_q = \log\big((\boldsymbol{q}^*)^{-1}\,\boldsymbol{q}\big)\,, \qquad \boldsymbol{e}_v = \dot{\boldsymbol{q}} - \mathrm{PT}_{q^*\to q}(\dot{\boldsymbol{q}}^*), \tag{3.19}$$

where PT denotes parallel transport of the reference velocity to the current configuration.

(ii) **Compute feedforward using the covariant derivative**, not ordinary differentiation of coordinates.

(iii) **Design feedback gains in the tangent space**, then map the resulting control back to the manifold via the exponential map.

(iv) **Monitor for cut-locus proximity**: when the error approaches π (for SO(3)), the logarithmic map becomes ill-conditioned. Switch to a recovery strategy.

Example 3.5. PD Control on SO(3)

The manifold-correct PD controller for tracking a reference orientation $\boldsymbol{R}^*(t)$ with a rigid body (moment of inertia $\boldsymbol{J}$, angular velocity $\boldsymbol{\omega}$) is:

$$\boldsymbol{\tau} = -k_p\,\log(\boldsymbol{R}^{*\top}\boldsymbol{R})^\vee - k_d\,(\boldsymbol{\omega} - \boldsymbol{R}^{*\top}\boldsymbol{R}\,\boldsymbol{\omega}^*) + \boldsymbol{J}\dot{\boldsymbol{\omega}}^* + \boldsymbol{\omega}\times\boldsymbol{J}\boldsymbol{\omega}, \tag{3.20}$$

where the first term is the manifold-correct proportional error (logarithmic map), the second is the velocity error (parallel-transported), and the last two terms are the feedforward (covariant derivative of the reference).
Compare with the naïve Euler-angle PD controller: $\tau = -k_p(\boldsymbol{\theta}-\boldsymbol{\theta}^*) - k_d(\dot{\boldsymbol{\theta}}-\dot{\boldsymbol{\theta}}^*)$. The naïve controller has singularities; Eq. (3.20) does not. The naïve controller produces torques proportional to angle differences (which can be misleading near $\pm\pi$); Eq. (3.20) produces torques proportional to the *true rotation* needed to correct the error.

3.11 Summary

Euclidean assumption	**Manifold replacement**
$\mathcal{Q} = \mathbb{R}^n$	$\mathcal{Q} = \mathrm{SO}(3)^k \times \mathbb{T}^m \times \cdots$
Error: $\boldsymbol{e} = \boldsymbol{q} - \boldsymbol{q}^*$	Error: $\boldsymbol{e} = \log((\boldsymbol{q}^*)^{-1}\boldsymbol{q})$
Derivative: $\ddot{\boldsymbol{q}}$	Covariant derivative: $\nabla_{\dot{q}}\dot{\boldsymbol{q}}$
Shortest path: straight line	Shortest path: geodesic
Distance: $\|\boldsymbol{q}_2 - \boldsymbol{q}_1\|$	Geodesic distance: $d_{\boldsymbol{G}}(\boldsymbol{q}_1, \boldsymbol{q}_2)$
Curvature: κ (Ch. 2)	Geodesic curvature: κ_g
Trajectory tube: Euclidean ball	Geodesic ball
Coriolis terms: "extra forces"	Christoffel symbols: manifold geometry
PD: $-k_p\boldsymbol{e} - k_d\dot{\boldsymbol{e}}$	$-k_p \log(\cdot)^\vee - k_d$(parallel transport)

The central message of this chapter is that the configuration spaces of mechanical systems have geometry, and this geometry matters for control. Working in $\mathbb{R}^n$ when the true space is $\mathrm{SO}(3) \times \mathbb{T}^k$ introduces artificial singularities, incorrect error metrics, and controllers that fail when the system moves far from a nominal configuration.

The trajectory-first philosophy of this book demands that we take the geometry seriously from the start. The curves we designed in Chapter 2 live on manifolds, and the stability theory we will develop in Chapter 4 must respect the manifold structure. The payoff is controllers that work globally, without singularities, and that correctly exploit the natural dynamics encoded in the manifold's Riemannian geometry.

3.12 Exercises

3.1. (Topology.) Determine the configuration manifold and its dimension for each system:

(a) A planar double pendulum (two revolute joints in 2D).

(b) A spherical pendulum (a point mass on a rigid rod, free to swing in all directions).

(c) A spacecraft with two reaction wheels (the body rotates freely in 3D; each wheel adds one internal DOF).

For each, identify whether $\mathbb{R}^n$ coordinates would have singularities.

3.2. (Euler angle singularity.) For the ZYX Euler angle representation $\boldsymbol{R} = R_z(\phi)\, R_y(\theta)\, R_x(\psi)$:

(a) Compute the Jacobian $\boldsymbol{J}$ relating angular velocity $\boldsymbol{\omega}$ to Euler angle rates $(\dot{\phi}, \dot{\theta}, \dot{\psi})$.

(b) Show that $\boldsymbol{J}$ is singular at $\theta = \pm\pi/2$.

(c) Find an orientation trajectory $\boldsymbol{R}(t)$ that passes through the singularity and compute the Euler angle rates. What happens?

3.3. (Logarithmic error.) For two rotations $\boldsymbol{R}_1 = \boldsymbol{I}$ and $\boldsymbol{R}_2 = R_z(\alpha)$ (rotation by α about z):

(a) Compute $\log(\boldsymbol{R}_1^\top \boldsymbol{R}_2)^\vee$ for $\alpha = \pi/6, \pi/2, \pi, 5\pi/3$.

(b) Compare with the "error" $\alpha - 0 = \alpha$ as a function of α. At what point do they differ significantly?

(c) What happens at $\alpha = \pi$? Explain geometrically.

3.4. (Geodesics.) For a double pendulum with masses m_1, m_2 and lengths l_1, l_2:

(a) Write the mass matrix $\boldsymbol{M}(\theta_1, \theta_2)$.

(b) Compute the Christoffel symbols (or use the Coriolis matrix formula $C_{ijk} = \frac{1}{2}(\partial M_{ij}/\partial q_k + \partial M_{ik}/\partial q_j - \partial M_{jk}/\partial q_i)$).

(c) Numerically integrate the geodesic equation from initial condition $(\theta_1, \theta_2) = (0, 0)$ with $(\dot\theta_1, \dot\theta_2) = (1, 0)$. Plot the geodesic on $\mathbb{T}^2$.

3.5. (Manifold PD control.) Implement and compare:

(a) An Euler-angle PD controller for a rigid body tracking a reference orientation $\boldsymbol{R}^*(t) = R_z(\omega t)$ (constant-speed rotation).

(b) The manifold PD controller from Eq. (3.20) for the same task.

(c) Run both controllers with the reference trajectory passing through $\theta = \pi/2$ (gimbal lock for ZYX convention). Compare the tracking errors and commanded torques.

3.6. (Golf swing configuration space.) For the 9-DOF golf swing model of Eq. (3.8):

(a) How many Euler angles would be needed to parameterize the full configuration space? How many singularity surfaces exist?

(b) Propose a quaternion-based parameterization for the two SO(3) joints. What is the total number of parameters? What constraints must be maintained?

(c) Discuss the trade-offs between the Euler angle and quaternion representations for numerical simulation of the golf swing.

The sphere cannot be flattened without tearing.
The rotation group cannot be charted without singularity.

Do not fight the topology—ride it.

The manifold is not an obstacle. It is the landscape through which every motion flows.

Chapter 4

Orbital Stability and Transverse Linearization

Stability is not the absence of motion.
It is the faithfulness of motion.

In Plain Language 4.1. Being on Time vs. Being on Track

Imagine driving a car along a winding mountain road. If you are five seconds behind schedule but perfectly in the center of your lane, you are perfectly safe. If you are exactly on time but two feet outside your lane, you might crash. Classical control theory measures "error" by comparing where you are right now to where you were "scheduled" to be right now. By that logic, being five seconds late is considered an error that the system must fight to correct. This chapter introduces "Orbital Stability," which mathematically formalizes the idea that being early or late is fine, as long as you stay on the path.

4.1 The Problem with Lyapunov

The classical Lyapunov framework asks a simple question: *does the state converge to an equilibrium?* We demonstrated in Chapter 1 that this is the wrong question for systems whose purpose is to move. But the problem goes deeper than philosophical framing—Lyapunov stability is *structurally incapable* of analyzing trajectory-tracking systems.

Consider a system perfectly tracking a reference trajectory $\boldsymbol{\gamma}^*(t)$. The state $\boldsymbol{x}(t)$ matches $\boldsymbol{\gamma}^*(t)$ at every instant. Now apply a small timing perturbation: the system executes the same motion but shifted by ε seconds. The perturbed state is $\tilde{\boldsymbol{x}}(t) = \boldsymbol{\gamma}^*(t - \varepsilon)$, and the time-indexed error is

$$\boldsymbol{e}(t) = \tilde{\boldsymbol{x}}(t) - \boldsymbol{\gamma}^*(t) = \boldsymbol{\gamma}^*(t - \varepsilon) - \boldsymbol{\gamma}^*(t) \approx -\varepsilon\, \dot{\boldsymbol{\gamma}}^*(t). \tag{4.1}$$

This error does *not* decay to zero—it persists indefinitely, oscillating with a magnitude proportional to $\varepsilon\, v(t)$ where v is the speed. By Lyapunov's definition, the trajectory is *unstable*. But the system is executing the correct motion perfectly! The only "error" is that it is slightly ahead of or behind schedule.

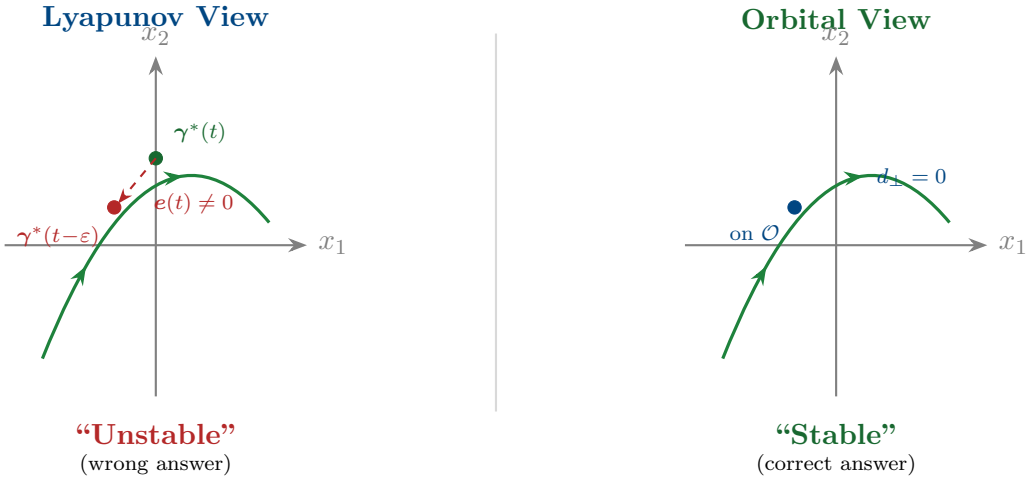
Key Idea 4.1. The Fundamental Inadequacy of Point Stability

Lyapunov stability conflates two independent phenomena:

(i) **Transverse deviations**: the system drifts *away from* the trajectory path. This is a genuine control failure.

(ii) **Tangential deviations**: the system is ahead of or behind schedule *along* the trajectory path. This is a timing difference, not a tracking failure.

Lyapunov stability rejects both. Orbital stability accepts tangential deviations and rejects only transverse ones. For systems whose purpose is motion, orbital stability is the correct notion.



4.2 Orbital Stability: Formal Definitions

We now define orbital stability rigorously. The key object is the **orbit**—the image of the trajectory as a set, stripped of all timing information.

Definition 4.1. Orbit

Given a trajectory $\boldsymbol{\gamma}^* : [0, T] \to \mathcal{X}$, the **orbit** is the set

$$\mathcal{O} = \{\boldsymbol{\gamma}^*(t) : t \in [0, T]\} \subset \mathcal{X}. \tag{4.2}$$

The orbit is a one-dimensional submanifold of $\mathcal{X}$ (a curve without parameterization). For periodic trajectories with period T, $\boldsymbol{\gamma}^*(0) = \boldsymbol{\gamma}^*(T)$ and $\mathcal{O}$ is a closed curve (a loop).

Definition 4.2. Orbital Stability

A trajectory $\boldsymbol{\gamma}^*(t)$ (or its orbit $\mathcal{O}$) is **orbitally stable** if for every $\varepsilon > 0$ there exists $\delta > 0$ such that

$$d(\boldsymbol{x}(0), \mathcal{O}) < \delta \implies d(\boldsymbol{x}(t), \mathcal{O}) < \varepsilon \quad \forall\, t \geq 0, \tag{4.3}$$

where $d(\boldsymbol{x}, \mathcal{O}) = \inf_{p \in \mathcal{O}} \|\boldsymbol{x} - p\|$ is the distance from the state to the orbit.
It is **asymptotically orbitally stable** if additionally $d(\boldsymbol{x}(t), \mathcal{O}) \to 0$ as $t \to \infty$.

Critically, orbital stability does not require $\|\boldsymbol{x}(t) - \boldsymbol{\gamma}^*(t)\| \to 0$. The system may drift along the orbit (ahead or behind schedule) while remaining perfectly stable. Only deviations *away from* the orbit matter.

Definition 4.3. Orbital Stability with Asymptotic Phase

A trajectory is orbitally stable **with asymptotic phase** if it is asymptotically orbitally stable and additionally there exists a phase shift θ^* such that

$$\|\boldsymbol{x}(t) - \boldsymbol{\gamma}^*(t + \theta^*)\| \to 0 \quad \text{as } t \to \infty. \tag{4.4}$$

This means the system not only returns to the orbit but eventually "locks in" to a definite timing, shifted by θ^* from the nominal schedule. This is the strongest practical form of trajectory stability.

Example 4.1. Limit Cycles: The Prototype

The Van der Pol oscillator $\ddot{x} - \mu(1 - x^2)\dot{x} + x = 0$ possesses a **limit cycle**—a periodic orbit that is asymptotically orbitally stable with asymptotic phase. All nearby trajectories spiral toward the limit cycle and eventually synchronize to a definite phase. The limit cycle is the simplest example of a system whose purpose is to move: the "target" is not a point but an orbit, and stability means the orbit attracts nearby trajectories. Walking, running, and heartbeats are all limit cycles. The golf swing is a transient trajectory (not periodic), but the same stability concepts apply over the finite interval $[0, T]$.

4.3 The Transverse–Tangential Decomposition

To analyze orbital stability, we must decompose the dynamics into components along and across the orbit. This is the *transverse–tangential decomposition*, the central technical tool of this chapter.

4.3.1 Projection onto the Orbit

Definition 4.4. Nearest-Point Projection

Given a state $\boldsymbol{x}$ sufficiently close to the orbit $\mathcal{O}$, the **nearest-point projection** is

$$\pi(\boldsymbol{x}) = \arg\min_{p \in \mathcal{O}} \|\boldsymbol{x} - p\|, \tag{4.5}$$

and the associated **projected phase** is the unique $s^*(\boldsymbol{x})$ satisfying $\pi(\boldsymbol{x}) = \boldsymbol{\gamma}^*(s^*)$. The projection is well-defined and smooth in a tubular neighborhood of $\mathcal{O}$ (as long as $\boldsymbol{x}$ is closer to $\mathcal{O}$ than the minimum radius of curvature of the orbit).

4.3.2 The Moving Hyperplane Decomposition

At each point $\boldsymbol{\gamma}^*(s)$ on the orbit, the state space $\mathbb{R}^n$ decomposes into a one-dimensional tangential direction and an $(n-1)$-dimensional transverse hyperplane:

Definition 4.5. Transverse Hyperplane

At the point $\boldsymbol{\gamma}^*(s)$ on the orbit, define:

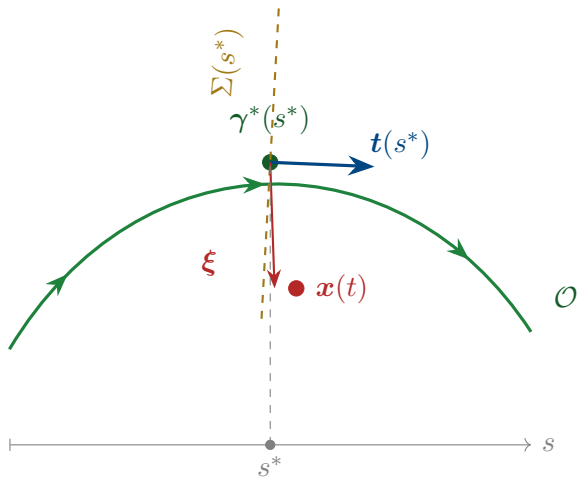
$$\text{Tangential direction:} \quad \boldsymbol{t}(s) = \frac{\dot{\boldsymbol{\gamma}}^*(s)}{\|\dot{\boldsymbol{\gamma}}^*(s)\|}, \tag{4.6}$$

$$\text{Transverse hyperplane:} \quad \Sigma(s) = \{\boldsymbol{v} \in \mathbb{R}^n : \boldsymbol{v}^\top \boldsymbol{t}(s) = 0\}. \tag{4.7}$$

A state $\boldsymbol{x}$ near $\boldsymbol{\gamma}^*(s)$ decomposes as

$$\boldsymbol{x} = \boldsymbol{\gamma}^*(s^*) + \boldsymbol{\xi}, \qquad \boldsymbol{\xi} \in \Sigma(s^*), \tag{4.8}$$

where s^* is the projected phase and $\boldsymbol{\xi}$ is the **transverse deviation**—the component of the error perpendicular to the orbit. Orbital stability is equivalent to the stability of the origin $\boldsymbol{\xi} = \mathbf{0}$ in the transverse dynamics.



4.4 Transverse Linearization

The power of the transverse decomposition is that it reduces orbital stability to the stability of a *time-varying linear system*—the transverse linearization.

4.4.1 Deriving the Transverse Dynamics

Consider the nonlinear system $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$ with a reference trajectory $\boldsymbol{\gamma}^*(t)$ satisfying $\dot{\boldsymbol{\gamma}}^* = \boldsymbol{f}(\boldsymbol{\gamma}^*, \boldsymbol{u}^*(t))$. Using the decomposition $\boldsymbol{x} = \boldsymbol{\gamma}^*(s^*) + \boldsymbol{\xi}$ and linearizing about $\boldsymbol{\xi} = \mathbf{0}$:

Theorem 4.1 (Transverse Linearization [12, 10])**.** *The linearized transverse dynamics about the orbit $\mathcal{O}$ are*

$$\dot{\boldsymbol{\xi}} = \boldsymbol{A}_\perp(s)\,\boldsymbol{\xi} + \boldsymbol{B}_\perp(s)\,\delta\boldsymbol{u}, \tag{4.9}$$

where $\delta\boldsymbol{u} = \boldsymbol{u} - \boldsymbol{u}^*$ is the feedback perturbation, and

$$\boldsymbol{A}_\perp(s) = \boldsymbol{P}(s)\left[\left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{\boldsymbol{\gamma}^*,\boldsymbol{u}^*} - \frac{\dot{\boldsymbol{\gamma}}^* \otimes (\nabla_{\boldsymbol{x}} s^*)^\top}{\|\dot{\boldsymbol{\gamma}}^*\|^2} \cdot \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{\boldsymbol{\gamma}^*,\boldsymbol{u}^*}\right]\boldsymbol{P}(s), \tag{4.10}$$

$$\boldsymbol{B}_\perp(s) = \boldsymbol{P}(s)\left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}\right|_{\boldsymbol{\gamma}^*,\boldsymbol{u}^*}, \tag{4.11}$$

and $\boldsymbol{P}(s) = \boldsymbol{I} - \boldsymbol{t}(s)\boldsymbol{t}(s)^\top$ is the orthogonal projector onto $\Sigma(s)$.

Remark 4.1. Understanding the Transverse $A_\perp$

Equation (4.10) has a clear structure:

- The first term $\boldsymbol{P}\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\boldsymbol{P}$ is the standard Jacobian projected onto the transverse space. This captures how the system dynamics affect cross-trajectory deviations.
- The second term subtracts the component of the Jacobian that acts along the tangent direction—this is exactly the tangential dynamics that orbital stability "forgets."

The resulting $\boldsymbol{A}_\perp(s)$ has dimension $(n-1) \times (n-1)$, one less than the full state Jacobian. The removed eigenvalue corresponds to motion along the orbit—the tangential direction in which we are *not* demanding stability.

Principle 4.1. The Transverse Reduction Principle

The reference trajectory $\boldsymbol{\gamma}^*(t)$ is asymptotically orbitally stable if and only if the origin of the transverse linearization (4.9) (with $\delta\boldsymbol{u} = \mathbf{0}$) is asymptotically stable. This reduces the orbital stability question from a nonlinear problem on the full n-dimensional state space to a *linear* problem on the $(n-1)$-dimensional transverse space. For controller design: if we can stabilize the linear system (4.9), we achieve orbital stability of the nonlinear system. All the tools of linear time-varying (LTV) control

theory apply.

Remark 4.2. Adjoint Systems and Costate Evolution

The transverse linearization and feedback gain computation via time-varying LQR are intimately connected to the adjoint system and costate dynamics from the Pontryagin Maximum Principle, as developed in Agrachev and Sachkov [1]. When solving trajectory optimization (Chapter 6), Pontryagin introduces costate variables $\boldsymbol{\lambda}(t)$ evolving backward according to adjoint dynamics. The Riccati matrix $\boldsymbol{P}(t)$ in the transverse LQR feedback law connects directly to these costates. Thus orbital stabilization (this chapter) and trajectory optimization (Chapter 6) spring from the same Hamiltonian geometric principle underlying optimal control.

4.4.2 The Transverse Jacobian: A Practical Formula

For implementation, the key quantities are computed as follows. Let $\boldsymbol{J}(t) = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\big|_{\boldsymbol{\gamma}^*, \boldsymbol{u}^*}$ be the Jacobian along the reference. Choose any orthonormal basis $\{\boldsymbol{n}_1(s), \dots, \boldsymbol{n}_{n-1}(s)\}$ for $\Sigma(s)$—the Frenet–Serret frame from Chapter 2 provides a natural choice. Collect these into the matrix $\boldsymbol{N}(s) = [\boldsymbol{n}_1 \mid \cdots \mid \boldsymbol{n}_{n-1}] \in \mathbb{R}^{n\times(n-1)}$. Then:

$$\boldsymbol{A}_\perp(s) = \boldsymbol{N}(s)^\top \left[\boldsymbol{J}(s) - \frac{\boldsymbol{f}^* \cdot \boldsymbol{J}(s)^\top \boldsymbol{t}(s)}{\|\boldsymbol{f}^*\|^2} \boldsymbol{f}^* \boldsymbol{t}(s)^\top - \dot{\boldsymbol{N}}(s)\, \boldsymbol{N}(s)^\top \right] \boldsymbol{N}(s), \tag{4.12}$$

where $\boldsymbol{f}^* = \boldsymbol{f}(\boldsymbol{\gamma}^*, \boldsymbol{u}^*)$ and $\dot{\boldsymbol{N}}$ accounts for the rotation of the transverse frame along the orbit (the Frenet–Serret equations).

Example 4.2. Transverse Linearization of a Planar System

Consider the system $\dot{x}_1 = x_2$, $\dot{x}_2 = -x_1 + u$ tracking the circular orbit $\boldsymbol{\gamma}^*(t) = (\cos t, -\sin t)$ with $u^* = 0$.
The Jacobian is $\boldsymbol{J} = \left(\begin{smallmatrix} 0 & 1 \\ -1 & 0 \end{smallmatrix}\right)$, the tangent vector is $\boldsymbol{t} = (-\sin t, -\cos t)^\top / 1 = (-\sin t, -\cos t)^\top$, and the normal is $\boldsymbol{n} = (-\cos t, \sin t)^\top$.
Computing $\boldsymbol{A}_\perp = \boldsymbol{n}^\top [\boldsymbol{J} - \text{tangential terms}] \boldsymbol{n}$, we find:

$$\boldsymbol{A}_\perp(t) = 0, \qquad \boldsymbol{B}_\perp(t) = \boldsymbol{n}^\top \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \sin t. \tag{4.13}$$

The open-loop transverse dynamics are *neutrally stable* ($\boldsymbol{A}_\perp = 0$). Feedback $\delta u = -k\,\xi$ gives $\dot{\xi} = -k \sin^2 t\, \xi$ (after averaging), which is stable for $k > 0$. The orbit is stabilizable.

4.5 Floquet Theory: Stability of Periodic Orbits

For periodic trajectories (limit cycles, walking gaits, repetitive manufacturing), the transverse linearization is a *periodic* linear system. Floquet theory provides the definitive stability analysis tool.

4.5.1 The State Transition Matrix

Definition 4.6. State Transition Matrix

For the linear time-varying system $\dot{\boldsymbol{\xi}} = \boldsymbol{A}_\perp(t)\,\boldsymbol{\xi}$, the **state transition matrix** $\boldsymbol{\Phi}(t, t_0)$ satisfies

$$\dot{\Phi}(t, t_0) = \boldsymbol{A}_\perp(t)\,\boldsymbol{\Phi}(t, t_0), \qquad \boldsymbol{\Phi}(t_0, t_0) = \boldsymbol{I}. \tag{4.14}$$

The solution is $\boldsymbol{\xi}(t) = \boldsymbol{\Phi}(t, t_0)\,\boldsymbol{\xi}(t_0)$.

4.5.2 The Monodromy Matrix

Definition 4.7. Monodromy Matrix

For a periodic orbit with period T, the **monodromy matrix** is the state transition matrix over one complete period:

$$\mathcal{M} = \boldsymbol{\Phi}(T, 0). \tag{4.15}$$

The eigenvalues $\mu_1, \ldots, \mu_{n-1}$ of $\mathcal{M}$ are the **Floquet multipliers** (also called **characteristic multipliers**).

Theorem 4.2 (Floquet Stability Criterion)**.** *A periodic orbit is:*

(i) ***Orbitally stable*** *if all Floquet multipliers satisfy $|\mu_k| \leq 1$, with multipliers on the unit circle being semisimple.*

(ii) ***Asymptotically orbitally stable*** *if all Floquet multipliers satisfy $|\mu_k| < 1$ (all strictly inside the unit circle).*

(iii) ***Orbitally unstable*** *if any Floquet multiplier satisfies $|\mu_k| > 1$.*

Remark 4.3. The Missing Multiplier

If we had computed the monodromy matrix for the *full* n-dimensional system (not the transverse reduction), there would be n Floquet multipliers, with one of them exactly equal to $+1$. This "trivial" multiplier corresponds to the tangential direction—perturbations along the orbit neither grow nor decay. The transverse reduction removes this trivial multiplier, leaving only the $n-1$ multipliers that matter for orbital

stability.
This is why the transverse reduction is essential: without it, the monodromy matrix *always* has an eigenvalue at $+1$, making the orbit appear only marginally stable even when it is asymptotically orbitally stable.

Example 4.3. Floquet Analysis of a Walking Gait

Consider a simplified compass-gait biped with 4-dimensional state space $(\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2)$. The walking gait is a periodic orbit with period T (one stride). The transverse linearization has dimension $n - 1 = 3$.
After numerically integrating $\mathbf{\Phi}(T, 0)$ for the transverse dynamics, suppose we find the monodromy matrix has eigenvalues $\mu_1 = 0.7$, $\mu_2 = 0.4$, $\mu_3 = -0.3$. Since all $|\mu_k| < 1$, the gait is asymptotically orbitally stable.
The physical interpretation: $\mu_1 = 0.7$ means that after one stride, transverse deviations in the first mode are reduced to 70% of their initial value. After 10 strides, they are reduced to $0.7^{10} \approx 3\%$. The negative multiplier $\mu_3 = -0.3$ indicates an alternating (flip-flop) convergence pattern in the third mode.
For the golf swing (a non-periodic motion), we cannot use the monodromy matrix directly, but the state transition matrix $\mathbf{\Phi}(T, 0)$ plays an analogous role—its singular values measure how much transverse deviations amplify or attenuate over the course of the motion.

4.6 Moving Poincaré Sections

A complementary tool for analyzing orbital stability is the **Poincaré section**—a codimension-1 surface that the orbit crosses transversally. While Floquet theory gives continuous-time information, Poincaré sections give *discrete-time* (strobed) information.

Definition 4.8. Poincaré Section

A **Poincaré section** for an orbit $\mathcal{O}$ is a smooth $(n-1)$-dimensional surface $\mathcal{P} \subset \mathcal{X}$ such that:

(i) $\mathcal{O}$ intersects $\mathcal{P}$ transversally (not tangentially).

(ii) In a neighborhood of the intersection, every orbit near $\mathcal{O}$ also crosses $\mathcal{P}$.

The **Poincaré return map** $\boldsymbol{P} : \mathcal{P} \to \mathcal{P}$ sends each point on $\mathcal{P}$ to the next intersection with $\mathcal{P}$ under the flow.

For periodic orbits, the orbit crosses $\mathcal{P}$ once per period, and the Poincaré return map has a fixed point at the intersection $\boldsymbol{x}^* = \mathcal{O} \cap \mathcal{P}$. Orbital stability of $\mathcal{O}$ is equivalent to stability of this fixed point under the return map.

Theorem 4.3 (Poincaré–Floquet Connection)**.** *The Jacobian of the Poincaré return map at the fixed point equals the transverse monodromy matrix:*

$$D\boldsymbol{P}(\boldsymbol{x}^*) = \mathcal{M}_\perp. \tag{4.16}$$

The Floquet multipliers and the eigenvalues of the Poincaré map are identical.

4.6.1 Moving Poincaré Sections for Non-Periodic Trajectories

The golf swing is not periodic—it is a *finite-time* trajectory. The classical Poincaré section (which relies on periodicity) does not directly apply. However, we can generalize to **moving Poincaré sections**: a continuous family of transverse hyperplanes, one at each point of the orbit.

Definition 4.9. Moving Poincaré Section

A **moving Poincaré section** along a trajectory $\boldsymbol{\gamma}^*(s)$, $s \in [0, L]$, is the family of hyperplanes

$$\mathcal{P}(s) = \boldsymbol{\gamma}^*(s) + \Sigma(s), \tag{4.17}$$

where $\Sigma(s)$ is the transverse hyperplane at s. The **section-to-section map** from $\mathcal{P}(s_1)$ to $\mathcal{P}(s_2)$ is defined by following the flow from s_1 to s_2:

$$\boldsymbol{P}_{s_1 \to s_2} : \mathcal{P}(s_1) \to \mathcal{P}(s_2), \qquad \boldsymbol{\xi}(s_1) \mapsto \boldsymbol{\xi}(s_2). \tag{4.18}$$

The linearized section-to-section map is the state transition matrix $\boldsymbol{\Phi}_\perp(s_2, s_1)$ of the transverse dynamics.

For the golf swing, this gives us a powerful analysis tool:

Principle 4.2. Finite-Time Orbital Stability

For a finite-time trajectory $\boldsymbol{\gamma}^* : [0, T] \to \mathcal{X}$, define the **transverse amplification ratio** from phase s_1 to s_2:

$$\rho(s_1, s_2) = \|\boldsymbol{\Phi}_\perp(s_2, s_1)\|_2 = \sigma_{\max}(\boldsymbol{\Phi}_\perp(s_2, s_1)), \tag{4.19}$$

where $\sigma_{\max}$ is the largest singular value. If $\rho(s_1, s_2) < 1$, then transverse deviations shrink between s_1 and s_2. If $\rho(s_1, s_2) > 1$, they grow.
For trajectory design, we require:

$$\rho(0, s_{\text{impact}}) \ll 1 \qquad \text{(deviations shrink toward impact)}. \tag{4.20}$$

This is a precise formulation of the funnel concept from Chapter 1: the tube narrows because the transverse dynamics are contracting.

Example 4.4. Amplification Profile of the Golf Swing

Consider the 8-dimensional state space of a 4-DOF golf swing model. The transverse dynamics have dimension 7. The transverse state transition matrix $\boldsymbol{\Phi}_\perp(s, 0)$ maps initial deviations to deviations at phase s.
Numerical computation reveals the amplification ratio $\rho(0, s)$ as a function of s:

- $s \in [0, 0.3]$ (backswing): ρ is approximately 1—deviations neither grow nor shrink. The motion is kinematically slow and nearly linear.
- $s \in [0.3, 0.5]$ (transition): ρ *increases* past 1. This is the unstable phase—the high curvature and speed change amplify deviations. Without feedback, errors grow here.
- $s \in [0.5, 0.7]$ (late downswing): ρ *decreases*. The passive dynamics of the double-pendulum release are self-stabilizing in the transverse directions. The wrist release "funnels" the club toward the correct impact state.
- $s = 0.75$ (impact): $\rho(0, 0.75) < 1$. The net effect over the entire swing is transverse contraction—deviations are smaller at impact than at address.

This profile explains why skilled golfers are accurate despite the extreme nonlinearity of the swing: the trajectory is designed so that its *passive transverse dynamics* are contracting near impact, creating a natural funnel that corrects errors without active feedback.

4.7 Controller Design via Transverse Stabilization

We now use the transverse linearization to design tracking controllers. The approach is systematic: stabilize the transverse linear system (4.9) using standard LTV techniques, then map the result back to the nonlinear system.

4.7.1 Time-Varying LQR for Transverse Stabilization

Principle 4.3. Transverse LQR

The trajectory tracking problem reduces to solving the *differential Riccati equation* for the transverse dynamics:

$$-\dot{\boldsymbol{S}}(t) = \boldsymbol{A}_\perp^\top \boldsymbol{S} + \boldsymbol{S}\,\boldsymbol{A}_\perp - \boldsymbol{S}\,\boldsymbol{B}_\perp \boldsymbol{R}^{-1} \boldsymbol{B}_\perp^\top \boldsymbol{S} + \boldsymbol{Q}(t), \tag{4.21}$$

with terminal condition $\boldsymbol{S}(T) = \boldsymbol{S}_T$. The optimal transverse feedback gain is

$$\boldsymbol{K}_\perp(t) = \boldsymbol{R}^{-1}\,\boldsymbol{B}_\perp(t)^\top\,\boldsymbol{S}(t). \tag{4.22}$$

The full control law is

$$\boldsymbol{u}(t) = \underbrace{\boldsymbol{u}^*(t)}_{\text{feedforward}} + \underbrace{\boldsymbol{N}(s^*)\,\boldsymbol{K}_\perp(t)\,\boldsymbol{N}(s^*)^\top(\boldsymbol{x} - \boldsymbol{\gamma}^*(s^*))}_{\text{transverse feedback}}, \tag{4.23}$$

where $\boldsymbol{N}(s^*)$ maps between the transverse coordinates $\boldsymbol{\xi}$ and the full state coordinates.

The weighting matrices $\boldsymbol{Q}(t)$ and $\boldsymbol{R}$ have clear physical interpretations:

- $\boldsymbol{Q}(t)$ penalizes transverse deviation. Making $\boldsymbol{Q}$ large near impact tightens the funnel where precision matters.
- $\boldsymbol{R}$ penalizes control effort. Making $\boldsymbol{R}$ small allows aggressive correction but may demand unrealistic actuator authority.
- The *phase-dependent* nature of $\boldsymbol{Q}(t)$ is essential: we want tight tracking near impact and loose tracking during the backswing. This is the time-varying tube from Chapter 1, now realized through the Riccati equation.

4.7.2 Phase-Varying Gains and the Funnel Connection

The terminal-value Riccati equation (4.21) naturally produces gains that tighten as the terminal time approaches. This is exactly the funnel behavior we designed geometrically in Chapter 1:

Theorem 4.4 (Riccati Funnel)**.** *Let $\boldsymbol{S}(t)$ be the solution of (4.21) with terminal penalty $\boldsymbol{S}_T \succ 0$. The ellipsoidal set*

$$\mathcal{E}(t) = \left\{\boldsymbol{\xi} \in \Sigma(s) : \boldsymbol{\xi}^\top \boldsymbol{S}(t)\,\boldsymbol{\xi} \le c\right\} \tag{4.24}$$

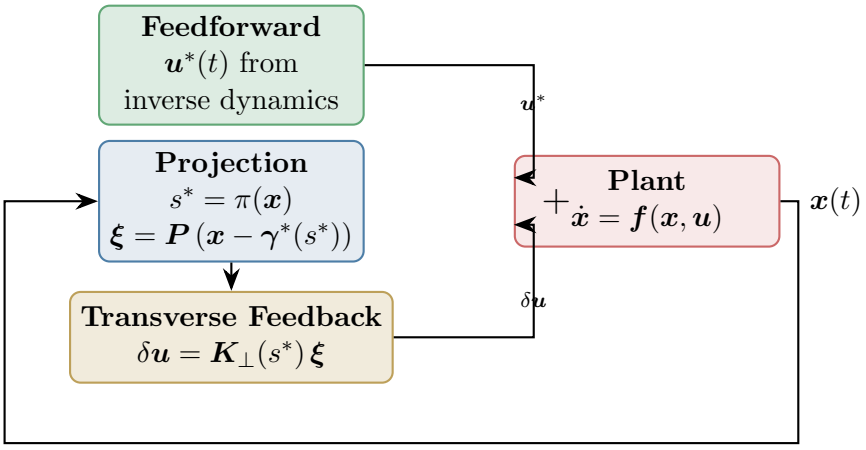
defines a time-varying tube whose width (in the direction of eigenvector $\boldsymbol{v}_k$ of $\boldsymbol{S}$) is proportional to $1/\sqrt{\lambda_k(t)}$, where $\lambda_k(t)$ is the corresponding eigenvalue of $\boldsymbol{S}(t)$.

As $t \to T$ and $\boldsymbol{S}(t) \to \boldsymbol{S}_T$, the tube tightens. The closed-loop transverse dynamics ensure that any trajectory starting inside the tube at $t = 0$ remains inside for all $t \in [0, T]$, provided the control authority is sufficient.

This is the formal connection between the geometric funnel of Chapter 1 and the algebraic Riccati equation: **the funnel is the level set of the Riccati cost-to-go.**

4.7.3 The Complete Tracking Controller Architecture

Assembling all the pieces, the trajectory tracking controller has three layers:



Layer 1. Feedforward $\boldsymbol{u}^*(t)$: computed offline from inverse dynamics, provides ∼90% of the total control effort.

Layer 2. Projection: measures the current state $\boldsymbol{x}(t)$, projects onto the orbit to find s^*, and computes the transverse deviation $\boldsymbol{\xi}$.

Layer 3. Transverse feedback $\delta\boldsymbol{u} = \boldsymbol{K}_\perp(s^*)\,\boldsymbol{\xi}$: stabilizes the transverse dynamics using phase-varying gains from the Riccati equation.

Remark 4.4. Why Not Just Use Standard LQR?

One could linearize the *full* dynamics about $\boldsymbol{\gamma}^*(t)$ and apply standard time-varying LQR to the n-dimensional error $\boldsymbol{e}(t) = \boldsymbol{x}(t) - \boldsymbol{\gamma}^*(t)$. This works, but:

(a) It "wastes" control effort trying to correct tangential deviations (timing errors), which are benign.

(b) It fights the inherent marginal stability in the tangential direction, leading to unnecessarily high gains.

(c) It does not produce the correct funnel shape—it penalizes all errors equally, including timing errors.

The transverse approach avoids all three problems. It is geometrically correct, computationally smaller (dimension $n - 1$ vs. n), and physically meaningful.

4.8 Robustness of Orbital Stability

A practical controller must be robust to model uncertainty and external disturbances. The transverse framework provides natural robustness margins.

Definition 4.10. Transverse Gain and Phase Margins

For the closed-loop transverse system $\dot{\boldsymbol{\xi}} = [\boldsymbol{A}_\perp - \boldsymbol{B}_\perp \boldsymbol{K}_\perp]\,\boldsymbol{\xi}$, the **transverse gain margin** and **transverse phase margin** are the gain and phase margins of the transverse loop transfer function. These are the trajectory-tracking analogues of the classical setpoint stability margins.

Proposition 4.1 (Robustness of Transverse LQR)**.** *If the transverse LQR is designed with* $\boldsymbol{R} = r\boldsymbol{I}$*, then the closed-loop transverse system has guaranteed gain margin* $[1/2, \infty)$ *and phase margin* $\pm 60°$*, inheriting the classical LQR robustness guarantees in each transverse channel.*

For the golf swing, robustness translates directly to *repeatability*: a swing with high transverse robustness margins produces consistent results even when the golfer's motor commands are noisy.

4.9 Summary

Concept	Significance
Orbital stability	The correct stability notion for moving systems: only cross-orbit deviations matter
Transverse linearization	Reduces orbital stability to $(n-1)$-dim LTV stability; removes the tangential "nuisance" direction
Floquet multipliers	Eigenvalues of the transverse monodromy matrix; determine stability of periodic orbits
Moving Poincaré sections	Extend Poincaré analysis to non-periodic trajectories; connect to the state transition matrix
Amplification ratio ρ	Maximum singular value of $\boldsymbol{\Phi}_\perp$; quantifies transverse error growth or contraction
Transverse LQR	Phase-varying feedback from the Riccati equation; the optimal funnel controller
Riccati funnel	The LQR cost-to-go defines the funnel boundary; connects geometric and algebraic views

This chapter has provided the mathematical backbone for everything that follows. The transverse linearization converts the nonlinear trajectory-tracking problem into a linear, lower-dimensional problem that inherits all the powerful tools of LTV control theory. The Floquet/Poincaré framework provides both analytical insight and numerical algorithms for assessing stability.

The key insight for the golf swing is that orbital stability—not Lyapunov stability—is the correct framework. The swing is designed so that its passive transverse dynamics create a natural funnel toward impact, and the transverse LQR controller tightens this funnel using phase-varying gains that match the demands of each phase of the motion.

In Chapter 5, we will see how underactuation constrains the transverse dynamics, creating geometric structure that can be *exploited* rather than merely tolerated.

4.10 Exercises

4.1. (Conceptual.) Explain why a pendulum swinging on its limit cycle (pumped by a periodic torque) is orbitally stable but *not* Lyapunov stable. Sketch the phase portrait and identify the tangential and transverse directions.

4.2. (Computation.) For the Van der Pol oscillator $\ddot{x} - \mu(1 - x^2)\dot{x} + x = 0$ with $\mu = 1$:

(a) Numerically find the limit cycle by simulating from $(x, \dot{x}) = (3, 0)$ until transient effects decay.

(b) Compute the transverse linearization along the limit cycle.

(c) Compute the monodromy matrix and Floquet multipliers numerically.

(d) Verify that one multiplier of the full (2D) system is exactly 1.

4.3. (Transverse linearization.) For the system $\dot{x}_1 = x_2 + u$, $\dot{x}_2 = -x_1^3$, consider the trajectory $\boldsymbol{\gamma}^*(t) = (\cos t, -\sin t)$ with appropriate u^*.

(a) Find $u^*(t)$ such that $\boldsymbol{\gamma}^*$ is a solution.

(b) Compute the full Jacobian $\boldsymbol{J}(t)$ along $\boldsymbol{\gamma}^*$.

(c) Compute the transverse Jacobian $\boldsymbol{A}_\perp(t)$.

(d) Is the orbit stable open-loop? Design stabilizing transverse feedback.

4.4. (Numerical project: Transverse LQR.) For the double pendulum from Exercise 3.4:

(a) Compute a reference swing trajectory using direct collocation (or load from a data file).

(b) Compute the transverse linearization along the trajectory.

(c) Solve the differential Riccati equation with $\boldsymbol{Q}(t)$ that increases 10-fold near the terminal phase ("impact").

(d) Simulate the closed-loop system with random initial perturbations and plot the transverse deviation $\|\boldsymbol{\xi}(t)\|$ over time.

(e) Compute the amplification ratio profile $\rho(0, s)$ and identify the "natural funnel" phase.

4.5. (Design.) A walking robot with period-1 gait has Floquet multipliers $\mu_1 = 0.9$, $\mu_2 = 1.1$, $\mu_3 = 0.5$.

(a) Is the gait orbitally stable? Which mode is problematic?

(b) Design a transverse feedback controller that places the unstable multiplier μ_2 at 0.6. What gain is required?

(c) After stabilization, what is the worst-case convergence rate per stride?

(d) How many strides does it take to reduce a 10% transverse deviation to 1%?

4.6. (Philosophical.) Compare the transverse LQR approach (this chapter) with the standard trajectory-tracking LQR (linearize about $\boldsymbol{\gamma}^*(t)$, regulate $\boldsymbol{e} = \boldsymbol{x} - \boldsymbol{\gamma}^*(t)$ to zero).

(a) Under what conditions do the two approaches give identical feedback gains? (Hint: consider straight-line trajectories.)

(b) Construct a numerical example where standard LQR uses significantly more control effort than transverse LQR.

(c) Discuss which approach is preferable for the golf swing and why.

The pendulum does not care if you are watching at noon or midnight.
Its orbit knows no clock.

Stability is not arrival. It is the promise
that the motion, once begun, will keep its shape—
even when the world pushes back.

Chapter 5

Underactuation and Passive Dynamics

Do not force the motion. Let the physics do the work.

In Plain Language 5.1. Working with Physics, Not Against It

Imagine trying to push a child on a playground swing. If you hold their back and try to physically muscle them through the exact shape of the arc at every single moment, you'll be exhausted and the ride will be jerky. Classical robotic control often tries to do just this: put a powerful motor on every joint and force the robot along a path, overpowering momentum and gravity. But "underactuated" systems, like a person walking or a golfer swinging a club, don't have enough motors (or muscles) to micromanage every joint at all times. In a golf swing, the wrists actually swing freely for much of the downswing. The club whips through the ball not because the wrists flexed forcefully, but because the speed and sudden braking of the arms created a physical whipping effect. This chapter looks at how to use these "passive" physics to our advantage, rather than seeing them as a lack of control.

5.1 The Loss of Control Authority

In classical robotics, the goal is often full actuation: one motor for every degree of freedom. If a robot has six joints, it has six motors. This fully actuated paradigm allows the system to follow essentially any smooth trajectory in the configuration space, subject only to actuator saturation limits. The control problem is algebraically simple because the mapping from torques to joint accelerations is invertible.

But nature does not build fully actuated systems, and neither do athletes.

When a gymnast swings on a high bar, their center of mass rotates around the bar, but they have no motor at the bar—that "joint" is a passive pin constraint. When a human walks, the foot acting as the pivot is unactuated during the single-support phase; we cannot create torque from a point contact. And crucially for our running example, when a golfer releases the club during the downswing, the wrist joint acts as a free hinge. The club rapidly accelerates to over 100 miles per hour, *driven entirely by the passive physics* of the arms pulling the grip, not by muscular torque applied at the wrist.

Definition 5.1. Underactuated System

Consider a mechanical system with n degrees of freedom (DOF) and configuration coordinates $\boldsymbol{q} \in \mathcal{Q} \subseteq \mathbb{R}^n$. The equations of motion take the standard manipulator form:

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) = \boldsymbol{B}(\boldsymbol{q})\boldsymbol{u} \tag{5.1}$$

where $\boldsymbol{M}$ is the inertia matrix, $\boldsymbol{C}$ contains Coriolis and centrifugal terms, $\boldsymbol{g}$ is the gravity vector, $\boldsymbol{u} \in \mathbb{R}^m$ is the vector of control inputs, and $\boldsymbol{B}(\boldsymbol{q}) \in \mathbb{R}^{n \times m}$ maps inputs into generalized forces.

The system is **underactuated** if $m < n$, meaning there are fewer actuators than degrees of freedom. In this case, the matrix $\boldsymbol{B}$ is not full rank (it maps a lower-dimensional control space into the n-dimensional configuration space).

If we lack the authority to command arbitrary accelerations, the classical setpoint perspective sees this as a profound failing. A setpoint controller wants to cancel the nonlinearities, feedback-linearize the system, and dictate the exact motion. Underactuation prevents this brute-force cancellation.

However, from the trajectory-first perspective developed in Chapter 1, underactuation is not merely a restriction—it is a geometric feature. The *passive dynamics* (the unactuated portions of the dynamics) define a manifold of feasible trajectories. We don't fight the physics; we ride it.

5.2 The Double Pendulum: A Window into the Golf Swing

To understand how passive dynamics can be exploited, we study the simplest model that captures the physics of the golf downswing: the planar underactuated double pendulum.

Let the first link (the arms) have angle θ_1, length l_1, mass m_1, and inertia I_1. Let the second link (the club) have angle θ_2, length l_2, mass m_2, and inertia I_2. The configuration is $\boldsymbol{q} = (\theta_1, \theta_2)^\top$.

Crucially, we assume there is a motor at the shoulder (controlling torque $u_1 = \tau_{\text{shoulder}}$) but *no motor* at the wrist ($\tau_{\text{wrist}} = 0$).

The input matrix is therefore:

$$\boldsymbol{B} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad \boldsymbol{u} = [u_1]. \tag{5.2}$$

The dynamics expand into two coupled equations:

$$M_{11}\ddot{\theta}_1 + M_{12}\ddot{\theta}_2 + C_1 + g_1 = u_1, \tag{5.3}$$

$$M_{21}\ddot{\theta}_1 + M_{22}\ddot{\theta}_2 + C_2 + g_2 = 0. \tag{5.4}$$

Equation (5.4) is the **unactuated dynamics** or **passive constraint**. It dictates the motion of the club. Notice that we cannot directly command $\ddot{\theta}_2$. However, we *can* influence it through the coupling terms:

(i) **Inertial coupling** ($M_{21}\ddot{\theta}_1$): Accelerating or decelerating the arms immediately exerts a force on the club.

(ii) **Velocity coupling** (C_2): This term contains the centrifugal forces. As the arms swing rapidly, centrifugal force naturally pulls the club outward.

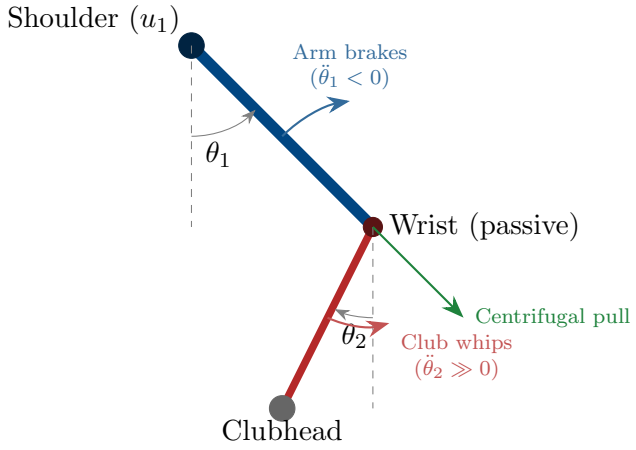
Key Idea 5.1. The Physics of the Wrist Release

In a golf swing, the huge clubhead speed at impact is generated by exploiting the C_2 and M_{21} terms. Early in the downswing, the golfer maintains an acute angle between the arms and the club ($\theta_2 - \theta_1 \approx 90°$). The arms accelerate ($\ddot{\theta}_1 > 0$).
As the hands approach the bottom of the swing arc, the arms decelerate ($\ddot{\theta}_1 < 0$). This negative acceleration, transferred through the M_{21} coupling, causes the club to rapidly whip forward. Simultaneously, the massive velocity of the arms ($\dot{\theta}_1$) creates a large centrifugal force (C_2) that drives the club in-line with the arms ($\theta_2 \to \theta_1$). The golfer does not "unhinge" their wrists with muscular effort; the physics forces the release.

Let's look at the unactuated row algebraically to see this whipping action:

$$\ddot{\theta}_2 = -M_{22}^{-1}\big(M_{21}\ddot{\theta}_1 + C_2(\boldsymbol{q}, \dot{\boldsymbol{q}}) + g_2(\boldsymbol{q})\big). \tag{5.5}$$

Since M_{22} is scalar and positive, a large negative $\ddot{\theta}_1$ (braking the arms) produces a large positive $\ddot{\theta}_2$ (accelerating the club). This is the parametric excitation that drives the moving target control problem. It is an optimal transfer of momentum.



5.3 The Annihilator and the Null Space

The double pendulum example shows that the passive dynamics constrain the allowable states and accelerations. We generalize this using the mathematics of the **null space**.

Recall the dynamics:

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) = \boldsymbol{B}(\boldsymbol{q})\boldsymbol{u}. \tag{5.6}$$

Definition 5.2. Left Annihilator

We define a full-rank matrix $\boldsymbol{B}^{\perp}(\boldsymbol{q}) \in \mathbb{R}^{(n-m)\times n}$, called the **left annihilator** of $\boldsymbol{B}$, such that:

$$\boldsymbol{B}^{\perp}(\boldsymbol{q})\boldsymbol{B}(\boldsymbol{q}) = \boldsymbol{0} \quad \text{for all } \boldsymbol{q}. \tag{5.7}$$

The rows of $\boldsymbol{B}^{\perp}$ span the left null space of $\boldsymbol{B}$.

In our double pendulum where $\boldsymbol{B} = [1, 0]^{\top}$, the annihilator is trivially $\boldsymbol{B}^{\perp} = [0, 1]$.

By multiplying the equations of motion by the annihilator $\boldsymbol{B}^{\perp}$, we eliminate the control input $\boldsymbol{u}$, revealing the intrinsic constraints on the system's acceleration:

$$\boldsymbol{B}^{\perp}\boldsymbol{M}\ddot{\boldsymbol{q}} + \boldsymbol{B}^{\perp}(\boldsymbol{C}\dot{\boldsymbol{q}} + \boldsymbol{g}) = \boldsymbol{0}. \tag{5.8}$$

Principle 5.1. The Theorem of Feasible Trajectories

A smooth curve $\boldsymbol{\gamma}^*(t)$ is a dynamically feasible trajectory for the underactuated system if and only if it satisfies the unactuated dynamics:

$$\boldsymbol{B}^{\perp}(\boldsymbol{\gamma}^*)\left[\boldsymbol{M}(\boldsymbol{\gamma}^*)\ddot{\boldsymbol{\gamma}}^* + \boldsymbol{C}(\boldsymbol{\gamma}^*, \dot{\boldsymbol{\gamma}}^*)\dot{\boldsymbol{\gamma}}^* + \boldsymbol{g}(\boldsymbol{\gamma}^*)\right] = \boldsymbol{0} \tag{5.9}$$

for all $t \in [0, T]$. If a curve satisfies this condition, one can always find the necessary control input $\boldsymbol{u}^*(t)$ by projecting the dynamics onto the range of $\boldsymbol{B}$.

For control design, this means we cannot arbitrarily select a path in $\mathcal{X}$ and expect a controller to track it. The reference trajectory *must* live in the restricted subspace defined by (5.8). The trajectory optimization algorithms in Chapter 6 will explicitly enforce this constraint.

5.4 Partial Feedback Linearization

While we cannot feedback-linearize the entire underactuated system, we can feedback-linearize the *actuated* degrees of freedom. This technique is called **partial feedback linearization** or **collocated feedback linearization**.

By cleverly substituting coordinates, we can partition the system into a linear, fully actuated subsystem (say, the golfer's arms) and a nonlinear, passive subsystem (the club) that is driven by the state of the first.

Partition the coordinates as $\boldsymbol{q} = (\boldsymbol{q}_a, \boldsymbol{q}_u)$, representing actuated and unactuated DOFs. We can rewrite the dynamics and choose $\boldsymbol{u}$ to exactly cancel the nonlinearities of the actuated part:

$$\ddot{\boldsymbol{q}}_a = \boldsymbol{v}_a \tag{5.10}$$

where $\boldsymbol{v}_a$ is our new synthetic control input. The unactuated dynamics become:

$$\boldsymbol{M}_{uu}(\boldsymbol{q})\ddot{\boldsymbol{q}}_u + \boldsymbol{M}_{ua}(\boldsymbol{q})\boldsymbol{v}_a + \text{nonlinear terms} = 0. \tag{5.11}$$

If we force the actuated coordinates to track a reference trajectory (e.g., using a high-gain PD controller, $\boldsymbol{v}_a = \ddot{\boldsymbol{q}}_a^* - K_d\dot{\tilde{\boldsymbol{q}}}_a - K_p\tilde{\boldsymbol{q}}_a$), the unactuated coordinates are left to evolve according to (5.11). The stability of the resulting motion depends entirely on whether (5.11) is a stable differential equation when forced by $\boldsymbol{q}_a^*$. This leads us to the concept of the *zero dynamics*.

5.5 Accessibility, Chow's Theorem, and the Orbit Theorem

For underactuated systems with fewer inputs than degrees of freedom, a fundamental question arises: which configurations are reachable? The answer comes from **Chow's Theorem**, the centerpiece of geometric accessibility theory developed rigorously in Agrachev and Sachkov [1].

Theorem 5.1 (Chow's Theorem / Orbit Theorem)**.** *For a control system $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}) + \sum_{i=1}^{m} u_i(\boldsymbol{x})\boldsymbol{g}_i(\boldsymbol{x})$, a point $\boldsymbol{x}_f$ is accessible from $\boldsymbol{x}_0$ if and only if $\boldsymbol{x}_f$ lies in the orbit generated by the Lie algebra $\mathfrak{g} = Lie\{\boldsymbol{f}, \boldsymbol{g}_1, \ldots, \boldsymbol{g}_m\}$ and their Lie brackets evaluated at $\boldsymbol{x}_0$.*

For the double pendulum of the golf swing, even though we cannot directly command the unactuated wrist ($\ddot{\theta}_2 = 0$ alone), the Lie bracket [shoulder control, inertial coupling] allows us to achieve motions of the club. Higher-order Lie brackets $[[\cdot,\cdot],\cdot]$ allow still more complex maneuvers. The dimension of the orbit directly determines the size of the reachable set. If the Lie algebra is full-rank everywhere, the system is "fully controllable"—every configuration is reachable. If the Lie algebra has lower rank, only a lower-dimensional submanifold of configurations is reachable, and this submanifold structure directly appears as the nonholonomic constraint in trajectory optimization.

Sub-Riemannian Geometry: When only a subset of directions are controllable (non-involutive distribution), the system naturally inherits a sub-Riemannian metric. Optimal trajectories in underactuated systems are geodesics with respect to this metric, and abnormal extremals (singular arcs) emerge precisely at points where the control becomes singular. The golf swing wrist release is an example: the optimal trajectory cannot be achieved with constant-effort controls; it requires a singular arc (bang-bang structure) where the control switches instantaneously to exploit the Lie bracket structure maximally.

5.6 Zero Dynamics and the Golf Swing Funnel

If we perfectly stabilize the actuated tracking errors to zero ($\boldsymbol{q}_a \equiv \boldsymbol{q}_a^*$), the remaining dynamics governing $\boldsymbol{q}_u$ are called the **zero dynamics**.

For the golf swing, the zero dynamics describe how the club moves if the golfer's arms perfectly follow the nominal swing path. Are the zero dynamics of the golf swing stable?

Revisiting the concept of Transverse Linearization (Chapter 4), if we linearize the unactuated dynamics around the trajectory $\boldsymbol{\gamma}^*(t)$, we analyze the transverse divergence of the club from its optimal path.

We find a beautiful physical phenomenon:

- During the early downswing, the club's zero dynamics are *unstable*. If the club gets slightly out of position, the centrifugal force tends to pull it further off the optimal plane.

- During the late downswing (the release phase), the club's zero dynamics become *strongly stable*. The massive acceleration $\ddot{\theta}_2 \gg 0$ acts like a stiff restoring spring. The centrifugal force aligns the club perfectly with the arms.

This is the manifestation of the time-varying tube (the Funnel) from Chapter 1! The passive physics naturally form a wide, loose tube that tightens into an incredibly precise singularity exactly at the moment of impact. The golfer does not need to use wrist torque to steer the clubhead to the ball; if the shoulder inputs are grossly correct, the physics of the double pendulum *pulls* the clubhead into the ball.

Remark 5.1. Why Good Swings Feel Effortless

Novice golfers attempt to fully actuate the swing. They use their wrists to steer the club, treating it as a fully actuated robotic arm. This fights the passive dynamics, resulting in slower club speeds and higher variability. Pro golfers use their arms and torso to set up the boundary conditions for the passive wrist release. By acting primarily in the null space (letting the unactuated dynamics dominate), the swing maximizes speed and consistency. The physics does the work.

5.7 Abnormal Extremals and Singular Arcs

A remarkable feature of underactuated optimal control is the existence of **abnormal extremals**—optimal trajectories where the maximum principle is satisfied in a degenerate way (the costate is proportional to the velocity). These do not appear in standard optimal control with full actuation, but are pervasive in underactuated systems. As explained by Agrachev and Sachkov [1], abnormal extremals correspond precisely to trajectories where the control must be "singular"—switching instantaneously or taking on a specific form not derivable from the regular Hamiltonian maximization.

In the golf swing, the club's dramatic acceleration during the wrist release is achieved partly through an abnormal arc. The control does not smoothly follow from the Hamiltonian; instead, it must switch or saturate to exploit the maximum amount of inertial coupling available. The geometry of the Lie bracket generates this singular structure automatically. Recognition of abnormal extremals is crucial for trajectory optimization: standard numerical solvers may miss them unless the problem is explicitly formulated to allow singular controls.

5.8 Summary

Concept	**Significance**
Underactuation	Fewer actuators (m) than coordinates (n). Cannot arbitrarily command accelerations in configuration space.
Passive Dynamics	The natural, unforced motion of the unactuated DOFs.
Coupling	Inertial (M_{21}) and velocity (C_2) terms transmit energy from actuated to unactuated DOFs.
Left Annihilator	A matrix $\boldsymbol{B}^{\perp}$ that strips away the control input to reveal the intrinsic geometric constraints of the motion.
Partial Feedback Lin.	Cancelling nonlinearities only on the actuated joints to simplify control.
Zero Dynamics	The residual dynamics of the unactuated subsystem when the actuated subsystem perfectly tracks its reference.

Underactuation forces a shift from "forcing" a system to an equilibrium, to "orchestrating" the natural physics of the system. The trajectory is a choreography between the motors and gravity/inertia. In Chapter 6, we will see how numerical trajectory optimization can synthesize these choreographies automatically by solving optimal control problems that explicitly respect the $\boldsymbol{B}^{\perp}$ constraint.

5.9 Exercises

5.1. (Annihilator construction.) A cart-pole system has state $\boldsymbol{q} = (x, \theta)^{\top}$, where x is the cart position and θ is the pole angle. The motor applies horizontal force to the cart.

(a) What is the input matrix $\boldsymbol{B}$?

(b) Construct a left annihilator $\boldsymbol{B}^{\perp}$.

(c) Derive the unactuated dynamics constraint. What physical conservation law does this represent when gravity is ignored?

5.2. (Passive pumping.) In a playground swing, the person swinging cannot exert an external torque on the swing setup (there is no motor at the top pivot). By what mechanism (which dynamics term) does the swinger add energy to the system by leaning back and forth? Relate this to the golf wrist release.

5.3. (Zero Dynamics Stability.) Consider a generic system partitioned into $\ddot{q}_a = v_a$ and $\ddot{q}_u + cq_u = q_a$. Let the reference trajectory be $q_a^*(t) = \sin(t)$.

(a) Assume we enforce $q_a = q_a^*$ perfectly. Write the zero dynamics for q_u.

(b) Under what conditions on the parameter c are the zero dynamics stable? Is it possible to have a trajectory that passes through both stable and unstable regimes of zero dynamics?

Control is not dominion over nature;
it is an intelligent negotiation with it.

Chapter 6

Trajectory Optimization

We do not merely seek a path that obeys the laws of physics.
We seek the path that bends them to our advantage.

In Plain Language 6.1. Discovering the Best Motion

How does an athlete instinctively know exactly how to move their arms to swing a bat as fast as possible? Up until now, we've talked about how to follow a path, but where does the path come from? This chapter is about how computers can simulate and "discover" these optimal paths. By giving the computer a goal—like "maximize the speed of the clubhead exactly when it hits the ball"—it can test thousands of different ways to swing, constrained by the laws of physics, until it discovers the perfect motion. You don't have to tell it how to swing; you just tell it what the goal is, and the math figures out the rest.

6.1 The Moving Target Problem

In Chapter 1, we fundamentally reframed our control objective from "reaching a destination" to "tracking a trajectory." Up to this point, we have assumed that this optimal reference trajectory $\boldsymbol{\gamma}^*(t)$ is given to us by an oracle. This chapter answers the obvious next question: *Where does this reference come from?*

For systems whose purpose is to move—like our running example of the golf swing—the optimal trajectory is not arbitrary. We are not just trying to go from point A to point B while minimizing energy. We are trying to fulfill what we term the **Moving Target Objective**: optimizing the kinematics (velocity, pose, acceleration) of a system precisely at the moment it intersects a target manifold (e.g., the golf ball).

Crucially, *the exact clock time that impact occurs does not matter.* A golfer has the freedom to start their backswing slowly and pause at the top. The optimization is entirely concerned with the *shape* of the motion and its terminal kinematics. Time is a resource, not a constraint.

Definition 6.1. The Moving Target Optimal Control Problem (OCP)

Find the state trajectory $\boldsymbol{x}(t)$, control input $\boldsymbol{u}(t)$, and terminal time t_f that minimize the cost functional:

$$J = \phi(\boldsymbol{x}(t_f)) + \int_0^{t_f} L(\boldsymbol{x}(t), \boldsymbol{u}(t))\, \mathrm{d}t \tag{6.1}$$

subject to the dynamics and constraints:

$$\dot{\boldsymbol{x}}(t) = \boldsymbol{f}(\boldsymbol{x}(t), \boldsymbol{u}(t)) \qquad \text{(Dynamics)} \tag{6.2}$$

$$\boldsymbol{h}(\boldsymbol{x}(t), \boldsymbol{u}(t)) \leq \boldsymbol{0} \qquad \text{(Path constraints)} \tag{6.3}$$

$$\psi(\boldsymbol{x}(t_f)) = \boldsymbol{0} \qquad \text{(Terminal manifold/Moving goal)} \tag{6.4}$$

$$\boldsymbol{x}(0) = \boldsymbol{x}_0 \qquad \text{(Initial state)} \tag{6.5}$$

In the golf downswing, the terminal cost $\phi(\boldsymbol{x}(t_f))$ might be negative clubhead speed (to maximize speed), the integral cost L might penalize excessive joint torques to ensure repeatability, and the terminal manifold constraint $\psi(\boldsymbol{x}(t_f)) = \boldsymbol{0}$ forces the clubhead to be geometrically located at the ball with a squared face. Notice that t_f is a *free parameter* in the optimization.

6.2 Transcribing the Continuous Problem

The OCP described above occurs in infinite-dimensional function space. To solve it on a computer, we must transcribe it into a finite-dimensional Nonlinear Programming problem (NLP):

$$\begin{aligned} \min_{\boldsymbol{w}} \quad & f(\boldsymbol{w}) \\ \text{subject to} \quad & \boldsymbol{c}(\boldsymbol{w}) \leq \boldsymbol{0}, \end{aligned}$$

where $\boldsymbol{w}$ is a large vector of decision variables. How we discretize the integrals and differential equations defines the transcription method. We will focus on two modern approaches: **Direct Shooting** and **Direct Collocation**.

6.2.1 Direct Multi-Shooting

In direct multi-shooting, we break the time interval $[0, t_f]$ into N segments. The decision variables are the control parameters $\boldsymbol{u}_k$ on each interval and the state values $\boldsymbol{x}_k$ at the beginning of each interval.

The dynamic constraint $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$ is enforced by numerical integration (e.g., Runge-Kutta). We require that the integrated state at the end of interval k exactly matches the assumed starting state of interval $k+1$:

$$\boldsymbol{x}_{k+1} - \boldsymbol{\Phi}(\boldsymbol{x}_k, \boldsymbol{u}_k, \Delta t) = \boldsymbol{0} \qquad \text{(Defect constraint)} \tag{6.6}$$

where $\boldsymbol{\Phi}$ represents the numerical integrator step. Multi-shooting is excellent for highly nonlinear dynamics because it keeps the integration intervals short, preventing the massive nonlinear sensitivities that plague single-shooting methods.

6.2.2 Direct Collocation

Direct Collocation takes this a step further. We represent both the state trajectory and the control signal as piecewise polynomials (e.g., cubic splines for state, linear splines for control). The decision variables are the coefficients of these polynomials, typically located at discrete **collocation points**.

Instead of using a numerical integrator, we enforce the differential equation algebraicaly at the collocation points:

$$\dot{\tilde{\boldsymbol{x}}}(t_c) - \boldsymbol{f}(\tilde{\boldsymbol{x}}(t_c), \tilde{\boldsymbol{u}}(t_c)) = \boldsymbol{0} \qquad \text{(Collocation constraint)} \tag{6.7}$$

where $\tilde{\boldsymbol{x}}$ and $\tilde{\boldsymbol{u}}$ are the polynomial approximations.

Remark 6.1. Pontryagin Maximum Principle and Optimal Control

While direct collocation provides a practical numerical algorithm, the theoretical foundation for trajectory optimization comes from the **Pontryagin Maximum Principle** [1]. This principle states that along an optimal trajectory, a Hamiltonian function $\mathcal{H}(\boldsymbol{x}, \boldsymbol{u}, \boldsymbol{\lambda}) = L(\boldsymbol{x}, \boldsymbol{u}) + \boldsymbol{\lambda}^\top \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$ must be maximized with respect to $\boldsymbol{u}$ at each instant, where the costate $\boldsymbol{\lambda}(t)$ evolves backward according to adjoint dynamics. For underactuated systems, **bang-bang** structures emerge (control saturates at bounds) when the Hamiltonian is linear in $\boldsymbol{u}$. **Singular arcs** occur when the Hamiltonian is affine but not linear, requiring higher-order conditions to determine $\boldsymbol{u}$. The golf swing's explosive wrist release is precisely such a singular arc: the optimal control structure is predicted by Agrachev and Sachkov's geometric analysis of the control Lie algebra and its role in the maximization condition.

where $\tilde{\boldsymbol{x}}$ and $\tilde{\boldsymbol{u}}$ are the polynomial approximations.

Key Idea 6.1. Fidelity vs. Flexibility

Direct collocation results in a massive, highly sparse NLP. Because the solver has access to the state at every grid point simultaneously, it can easily manipulate the physical trajectory to satisfy difficult path constraints without waiting to "simulate" the result. Collocation is widely considered the state-of-the-art for generating complex dynamic motions like walking, jumping, and swinging.

6.3 Optimizing Underactuated Geometries

Recall from Chapter 5 that the underactuated dynamics enforce a strict algebraic constraint on the system:

$$\boldsymbol{B}^\perp(\boldsymbol{x})\Big[\boldsymbol{M}(\boldsymbol{x})\ddot{\boldsymbol{x}} + \boldsymbol{C}(\boldsymbol{x}, \dot{\boldsymbol{x}})\dot{\boldsymbol{x}} + \boldsymbol{g}(\boldsymbol{x})\Big] = \boldsymbol{0}. \tag{6.8}$$

When optimizing for a golf swing, direct collocation handles this underactuation organically. We do not need to explicitly partition the variables into actuated and unactuated sets for the solver. The collocation constraint $\dot{\boldsymbol{x}} - \boldsymbol{f} = \boldsymbol{0}$ already implicitly contains the passive dynamic constraints.

Example 6.1. The Golf Swing OCP

Consider our planar double pendulum model from Chapter 5. The solver is given the task to maximize $\dot{\theta}_2(t_f)$ (club speed).
Initial Guess: If we seed the NLP solver with a naive guess (e.g., constant shoulder torque), the club speed will be pitifully low.
The Optimized Result: The solver discovers the optimal physical orchestration entirely on its own. It will apply a massive positive u_1 to accelerate the arms, and then immediately reverse u_1 to negative maximum just prior to t_f. The optimizer mathematicaly "discovers" the necessary inertial coupling (the parametric excitation of $M_{21}\ddot{\theta}_1$) to whip the unactuated club link through the terminal manifold constraint! The passive dynamics are beautifully exploited to satisfy the objective.

6.4 Repeatability: The Stochastic OCP

The previous sections frame a deterministic problem: "Find the swing that maximizes terminal speed."

But as we argued in Chapter 1, biological systems and athletes do not optimize for peak deterministic performance. If you instruct an NLP solver to maximize golf club speed without regard for motor noise, the solver will ride the very edge of the constraint boundaries. It will produce a swing so delicately timed that a 5-millisecond execution error would result in a complete whiff.

Principle 6.1. The Repeatability Addendum

The theoretically optimal trajectory is useless if its local topology amplifies small execution errors into catastrophic terminal misses. The true objective must account for *variance*.

We modify our deterministic cost $J = \phi(\boldsymbol{x}(t_f)) + \int L$ into a stochastic expectation:

$$\min_{\boldsymbol{\gamma}^*} \quad \mathbb{E}\left[\phi(\boldsymbol{x}(t_f, \boldsymbol{w}))\right] + \lambda \cdot \mathrm{Var}\left[\psi(\boldsymbol{x}(t_f, \boldsymbol{w}))\right] \tag{6.9}$$

where $\boldsymbol{w} \sim \mathcal{N}(0, \Sigma_w)$ represents signal-dependent motor noise (a known property of human muscle actuation, where noise scales proportionally to the applied torque).

To solve this in a collocation framework without massive Monte Carlo simulations, we use **Covariance Steering** or **Linear Covariance Analysis**. We augment the state vector $\boldsymbol{x}(t)$ with its local covariance matrix $\boldsymbol{\Sigma}(t)$. The dynamics of $\boldsymbol{\Sigma}(t)$ are propagated alongside the nominal trajectory using the linearized Jacobian matrices along the path.

The optimizer is now forced to "widen the funnel" as it approaches the moving goal. It sacrifices a small amount of peak clubhead speed to select a trajectory geometry where the passive dynamics are heavily stabilizing in the transverse direction. The result is a trajectory that a human can actually *repeat*.

6.5 Summary

Concept	**Significance**
Moving Target OCP	Finding a trajectory that optimizes terminal kinematics when intersecting a spatial manifold, without prescribing the clock time.
Multi-Shooting	Discretizing the time domain into segments, using numerical integrators to enforce dynamics constraints between nodes.
Direct Collocation	Representing the entire trajectory as splines and enforcing dynamics algebraically at discrete points arrayed along the trajectory.
Passive Exploitation	Solvers naturally optimize underactuated topologies by weaponizing coupling matrices (M_{21}, C_2).
Stochastic Robustness	Augmenting the OCP with expected variance to ensure the resulting trajectory can be consistently executed by noisy nonlinear plants.

Determining the shape of the reference trajectory is the cornerstone of Control Is Motion. With our passive geometry understood (Chap 5) and our optimal reference trajectory computed (Chap 6), we must now surround that reference trajectory with a guaranteed, robust control tube to reject disturbances online. We turn to Funnel Synthesis in Chapter 7.

6.6 Exercises

6.1. (Free Terminal Time Formulation.) Explain how one sets up an OCP where final time t_f is a decision variable in a fixed-grid Direct Collocation solver. (Hint: consider scaling time $t = \tau \cdot t_f$, where $\tau \in [0, 1]$). Show how the dynamic constraint $\dot{\boldsymbol{x}} = \boldsymbol{f}$ changes under this substitution.

6.2. (Kinematic Goals vs Setpoints.) You are optimizing a baseball bat swing. Your state is $(\theta, \dot{\theta})$. Assume the ball arrives at the plate between $t = 0.5$ and $t = 0.6$ seconds. Formulate the path constraint and the terminal manifold constraint ψ assuming you want maximum $\dot{\theta}$ when $\theta = \pi/2$, but the exact time of impact within that window does not matter.

6.3. (Signal-Dependent Noise Penalty.) Human muscle torque output τ has variance $\sigma^2 = k|\tau|^2$. Write a modified integral cost function $L(\boldsymbol{x}, \tau)$ that penalizes the introduction of variance into the system. Interpret what this does to the optimal trajectory shape.

The perfect motion is rarely the fastest.
It is the one that survives reality.

Chapter 7

Funnel Synthesis

You cannot control the exact trajectory a stone will take down a mountain.
But you can build a trough to ensure it reaches the bottom.

In Plain Language 7.1. Building a Mathematical Trough

Even if you know the mathematically perfect path for a golf swing, no human can execute it perfectly twice. Muscles twitch, timing fluctuates, and wind blows. If we want to guarantee success, we cannot just plan a single path; we need to build a "funnel" or a "tube" around that path. As long as the system stays anywhere inside this tube, it is mathematically guaranteed to reach the target. This chapter explains how to use powerful optimization algorithms to calculate the exact shape and size of this tube. We compute a boundary that the system is literally forbidden from crossing, ensuring that minor errors in the backswing are naturally corrected by the time the club hits the ball.

7.1 Beyond LQR: The Need for Guaranteed Invariance

In Chapter 4, we showed that applying time-varying Linear Quadratic Regulation (LQR) to the transverse dynamics creates a contracting envelope of stability. The Riccati equation naturally shapes this envelope into a "funnel" that narrows as the terminal constraint approaches.

However, LQR guarantees are inherently *local*. They rely on the linear approximation of the transverse dynamics $\dot{\boldsymbol{\xi}} = \boldsymbol{A}_\perp(t)\boldsymbol{\xi} + \boldsymbol{B}_\perp(t)\delta\boldsymbol{u}$. For actual physical systems traversing highly nonlinear state spaces—like a golf club accelerating to 50 m/s while subject to massive centrifugal and Coriolis forces—this linearization quickly breaks down. If a disturbance pushes the state too far from the reference trajectory $\boldsymbol{\gamma}^*(t)$, the nonlinearities will dominate, the LQR restoring forces will compute the wrong corrective action, and the clubhead will completely miss the ball.

We need a mathematically rigorous way to define the *exact boundary* of our funnel. We must prove that for the fully nonlinear equations of motion, any state starting within the

mouth of the funnel at time $t = 0$ will remain inside the funnel for all $t \in [0, t_f]$.

Definition 7.1. Dynamic Funnel

Given a reference trajectory $\boldsymbol{\gamma}^*(t)$, a feedback controller $\boldsymbol{u} = \boldsymbol{k}(\boldsymbol{x}, t)$, and a scalar boundary function $V(\boldsymbol{x}, t)$, the set:

$$\mathcal{F}(t) = \{\boldsymbol{x} \in \mathcal{X} \mid V(\boldsymbol{x}, t) \leq 1\} \tag{7.1}$$

constitutes a valid **dynamic funnel** if the set is positively invariant under the closed-loop dynamics. That is, if $\boldsymbol{x}(0) \in \mathcal{F}(0)$, then $\boldsymbol{x}(t) \in \mathcal{F}(t)$ for all $t > 0$.

Remark 7.1. Funnels as Attainable Sets and Symplectic Structure

The dynamic funnel can be reinterpreted through the framework of **attainable sets** developed in Agrachev and Sachkov [1]. At time t, the funnel boundary $V(\boldsymbol{x}, t) = 1$ defines the set of states that are on the margin of being able to reach the terminal manifold $\psi(\boldsymbol{x}(t_f)) = \mathbf{0}$ at time t_f. States strictly inside the funnel ($V < 1$) can reach the target with robustness to spare; states outside cannot reach it at all. As $t \to t_f$, the attainable set shrinks to a single point (or lower-dimensional manifold if the terminal constraint has slack). The Lyapunov function $V(\boldsymbol{x}, t)$ encodes this shrinking geometry. Furthermore, the Hamiltonian structure from the Pontryagin framework implies that the level sets $V = \text{const}$ are transverse to the true optimal trajectories in the sense of symplectic geometry. The funnel is thus not merely a Euclidean tube, but a geometric object reflecting the Hamiltonian symplectic structure of optimal motion.

7.2 Time-Varying Lyapunov Functions for Trajectories

To verify invariance, we use the machinery of Lyapunov theory, adapted for trajectories rather than equilibria. We require our boundary function $V(\boldsymbol{x}, t)$ to act as a time-varying Lyapunov function.

Theorem 7.1 (Funnel Invariance)**.** *The set $\mathcal{F}(t) = \{\boldsymbol{x} \mid V(\boldsymbol{x}, t) \leq 1\}$ is invariant if, on the boundary where $V(\boldsymbol{x}, t) = 1$, the function is strictly decreasing along the system trajectories:*

$$\dot{V}(\boldsymbol{x}, t) = \frac{\partial V}{\partial t} + \nabla_{\boldsymbol{x}} V(\boldsymbol{x}, t) \cdot \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{k}(\boldsymbol{x}, t)) < 0 \quad \textit{for all } \boldsymbol{x} \textit{ s.t. } V(\boldsymbol{x}, t) = 1. \tag{7.2}$$

If the derivative of the boundary function points inward everywhere on the boundary, trajectories may circulate inside the funnel, but they can never escape it. For a moving target problem, we want the spatial volume of $\mathcal{F}(t)$ to shrink as $t \to t_f$, terminating in a tight bounded region at impact.

Key Idea 7.1. Funnels Define the Usable State Space

A computed funnel is not just an analysis tool; it is the ultimate judge of repeatability. It explicitly defines the exact set of initial conditions (e.g., the kinematic state at the top of the backswing) that are mathematically guaranteed to successfully strike the ball despite the presence of extreme nonlinearities.

7.3 Sums-of-Squares (SOS) Programming

Proving that $\dot{V} < 0$ on the boundary $V = 1$ for a general nonlinear function $\boldsymbol{f}$ is famously difficult. We have an infinite number of states on the boundary to check! We circumvent this by restricting our dynamics $\boldsymbol{f}$ algorithms to polynomials (via Taylor expansion or substitution) and leveraging **Sums-of-Squares (SOS) Programming** [2, 9].

A polynomial $p(x)$ is a Sum of Squares if it can be written as $p(x) = \sum_i q_i(x)^2$ for some polynomials $q_i(x)$. If a polynomial is SOS, it is globally non-negative. Testing if a polynomial is SOS turns out to be equivalent to solving a Semidefinite Program (SDP)—a convex optimization problem that can be solved very efficiently.

We can reframe Theorem 7.1 using the S-procedure. To guarantee that $\dot{V}(\boldsymbol{x}, t) < 0$ whenever $V(\boldsymbol{x}, t) = 1$, we require:

$$-\dot{V}(\boldsymbol{x}, t) - L(\boldsymbol{x}, t)(1 - V(\boldsymbol{x}, t)) \quad \text{is SOS} \tag{7.3}$$

where $L(\boldsymbol{x}, t)$ is a strictly positive polynomial multiplier. If $V(\boldsymbol{x}, t) = 1$, the second term vanishes, and the SOS condition forces $-\dot{V} > 0$, achieving our invariance proof!

7.4 Computing the Funnel for the Golf Swing

Let us return to the double pendulum of the golf swing. Our reference trajectory $\boldsymbol{\gamma}^*(t)$ from Chapter 6 gives us the nominal kinematics. Our transverse LQR from Chapter 4 gives us a feedback matrix $\boldsymbol{K}(t)$ designed to stabilize deviations. We wish to compute the largest possible funnel around this sequence that the closed-loop system is mathematically guaranteed to stay inside.

We define our candidate Lyapunov function as a time-varying quadratic form:

$$V(\boldsymbol{x}, t) = (\boldsymbol{x} - \boldsymbol{\gamma}^*(t))^\top \boldsymbol{P}(t)(\boldsymbol{x} - \boldsymbol{\gamma}^*(t)) \tag{7.4}$$

where $\boldsymbol{P}(t)$ is a positive definite matrix that changes continuously along the trajectory. The funnel boundary is the time-varying ellipsoid where $V = 1$. By altering $\boldsymbol{P}(t)$, we alter the shape and orientation of the funnel.

Remark 7.2. Value Functions and Hamilton-Jacobi-Bellman Theory

The Lyapunov function $V(\boldsymbol{x}, t)$ in funnel synthesis is deeply connected to the value function from dynamic programming, and more broadly to the **Hamilton-Jacobi-**

Bellman (HJB) equation in optimal control. The value function $J^*(\boldsymbol{x}, t)$ represents the optimal cost to reach the terminal constraint from state $\boldsymbol{x}$ at time t. In the deterministic setting of Agrachev and Sachkov's framework [1], the value function satisfies the HJB partial differential equation:

$$\frac{\partial J^*}{\partial t} + \min_{\boldsymbol{u}} \left[L(\boldsymbol{x}, \boldsymbol{u}) + \nabla_{\boldsymbol{x}} J^* \cdot \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u}) \right] = 0. \tag{7.5}$$

The level sets of $J^*(\boldsymbol{x}, t) = c$ define the attainable set from which cost c can still be incurred to reach the target. The funnel synthesized via Lyapunov methods is a conservative approximation of this optimal value function: it guarantees invariance via the $\dot{V} < 0$ condition, making it a certificate of feasibility. Computing the exact value function would require solving the HJB equation, which is generally difficult; the Lyapunov/SOS approach trades exactness for verifiability.

Example 7.1. Volume Maximization via SDP

We transcribe the continuous time t into discrete nodes t_k, just as we did in direct collocation. At each node, we set up an SDP that seeks to *maximize* the volume of the ellipsoid defined by $\boldsymbol{P}(t_k)$, subject to the SOS invariance constraints connecting it to adjacent nodes.

The optimizer actively computes the exact structural limits of the golf swing's passive dynamics. It discovers that early in the downswing, the funnel can be incredibly "wide" (forgiving), but as centrifugal forces build, the SOS constraints force $\boldsymbol{P}(t_k)$ to become highly elongated along the unactuated dimensions in order to maintain the invariance guarantee. The resulting geometry is a true numerical portrait of athletic consistency.

In Chapter 8, we will explore how to make these funnels robust to temporal variation by migrating out of the time domain entirely and into the phase domain.

Chapter 8

Phase-Variable Control

Time is a river that carries us forward. Phase is the boat we build to navigate it.

In Plain Language 8.1. Ditching the Clock

If a robot is programmed to walk across a room in exactly 10 seconds, and you push it backwards halfway through, the robot will try to instantly teleport to where it was "supposed" to be at second 6. It will fall over. A human, on the other hand, doesn't walk by the clock. A human walks by physical milestones: "When my left foot hits the ground, swing my right leg forward." This is called using a "phase variable" instead of time. A phase variable is a physical marker (like the angle of the left leg) that tells the system where it is in the motion. By throwing away the clock and linking all actions to physical progress, the motion becomes incredibly resilient to being slowed down, sped up, or paused.

8.1 Spatial Paths vs. Temporal Execution

The most glaring flaw in classical control architecture, when applied to biomechanics or advanced robotics, is its rigid adherence to the clock. If you construct a time-varying trajectory $\boldsymbol{\gamma}^*(t)$ for a robotic arm and command a tracking controller to follow it, the controller will evaluate the error as $\boldsymbol{e}(t) = \boldsymbol{x}(t) - \boldsymbol{\gamma}^*(t)$.

If the robot hits an unmodeled friction patch and slows down, the reference trajectory $\boldsymbol{\gamma}^*(t)$ continues advancing relentlessly. The controller calculates a massive position error not because the robot deviated physically from the path, but simply because it is *behind schedule*. In a desperate attempt to catch up, the actuators max out, applying violent torques that throw the system entirely off the stable path.

Key Idea 8.1. The Autonomy of Time

A system executing a motion should dictate its own progress. If the system is impeded, the reference should slow down. We must decouple the spatial geometry of the motion from its temporal execution. We achieve this by replacing chronological time t with a

monotonically increasing **phase variable** s.

Instead of parameterizing our optimal trajectory as $\boldsymbol{\gamma}^*(t)$, we parameterize it as a function of the phase: $\boldsymbol{\gamma}^*(s)$. The phase $s \in [0, 1]$ represents the percentage of motion completed. For a walking robot, s could be the forward position of the hip. For a golf swing, s could be the angular position of the shoulder relative to the ball. The reference trajectory is now a purely spatial curve.

Definition 8.1. Phase Variable

A phase variable $s(\boldsymbol{x})$ is a continuously differentiable scalar function of the state $\boldsymbol{x}$ such that its time derivative is strictly positive along all feasible trajectories inside the nominal funnel:

$$\dot{s}(\boldsymbol{x}) > 0 \quad \text{for all } \boldsymbol{x} \in \mathcal{F}. \tag{8.1}$$

8.2 Virtual Constraints

If the goal is no longer to match a time sequence but rather to conform to a spatial path defined by $s(\boldsymbol{x})$, we can express the control objective as a set of holonomic constraints on the state. However, because these constraints are not mechanical linkages but rather enforced by software, they are termed **Virtual Constraints** [17]. The theory of virtual constraints and hybrid zero dynamics was developed primarily in the context of bipedal locomotion by Westervelt, Grizzle, and collaborators.

Remark 8.1. Phase Variables and the Orbit Theorem

The phase variable $s(\boldsymbol{x})$ that decouples spatial path from temporal execution is intimately related to the orbits generated by control vector fields via the Orbit Theorem (Chow's Theorem) of Agrachev and Sachkov [1]. Specifically, the phase variable defines a *reduced system* where the high-dimensional configuration manifold is projected onto a one-dimensional phase axis. For underactuated systems, the Orbit Theorem guarantees that if the control Lie algebra is sufficiently rich (certain rank conditions on Lie brackets), then all states reachable on the manifold defined by fixed s are also reachable by varying s. The phase variable construction thus implements a reduction of the control system in the sense of Agrachev and Sachkov's theory of systems with symmetries. The virtual constraints are enforced in the Lie algebra sense: they represent the use of feedback to "steer" the system's evolution within the control algebra to remain on a prescribed lower-dimensional submanifold parameterized by s.

If the goal is no longer to match a time sequence but rather to conform to a spatial path defined by $s(\boldsymbol{x})$, we can express the control objective as a set of holonomic constraints on the state. However, because these constraints are not mechanical linkages but rather enforced by software, they are termed **Virtual Constraints**.

Let our control objective be to drive the state $\boldsymbol{x}$ to the invariant manifold $\mathcal{Z}$ defined by:

$$\mathcal{Z} = \{\boldsymbol{x} \mid \boldsymbol{x} - \boldsymbol{\gamma}^*(s(\boldsymbol{x})) = \mathbf{0}\}. \tag{8.2}$$

In Plain Language 8.2. Mathematical Reminder: The Invariant Manifold

Recall our manifold definition from Volume 0: a curved geometric surface. The **Virtual Constraints** physically force the system to stick to a very specific, lower-dimensional "submanifold" embedded within the full State Space. We call this an **Invariant Manifold** because once the feedback controller forces the system onto this surface, the system's own natural physics (combined with the controller) ensures it never leaves it. The system "slides" perfectly along this curved surface toward the goal.

By using partial feedback linearization (Chapter 5), we can design a synthetic input $\boldsymbol{v}_a$ that drives the actuated degrees of freedom $\boldsymbol{q}_a$ onto their nominal surfaces proportional to the phase:

$$y = \boldsymbol{q}_a - \boldsymbol{\gamma}_a^*(s(\boldsymbol{x})) \to 0. \tag{8.3}$$

As $y \to 0$, the system converges toward the virtual constraints. Crucially, the rate at which it progresses along the path is entirely governed by $\dot{s}$, which is itself determined by the current momentum and state of the system, not a stopwatch.

8.3 Hybrid Zero Dynamics

When the virtual constraints are perfectly satisfied ($y = 0$), the entire complexity of the n-dimensional state space collapses down into a highly reduced dynamical system that evolves strictly along the manifold $\mathcal{Z}$. This is the **Zero Dynamics** of the system.

For systems that undergo discrete impacts (a foot hitting the ground, or a golf club striking a ball), the system evolves via continuous differential equations until an impact surface is triggered, followed by an instantaneous jump map (a reset) dictated by conservation of momentum.

Principle 8.1. Hybrid Zero Dynamics (HZD)

A control architecture achieves Hybrid Zero Dynamics if the invariant manifold $\mathcal{Z}$ (the zero dynamics surface) is invariant not just under the continuous flow of the differential equations, but also invariant under the discrete impact map. That is, an impact starting on the manifold $\mathcal{Z}$ must immediately map to a new state that is also on $\mathcal{Z}$.

By enforcing HZD, we can design rhythmic walking gaits that are astonishingly robust. If the robot trips, its phase slows down, but the virtual constraints remain active, keeping the limbs on the correct spatial path until the next footfall resets the sequence. The temporal sequence arises naturally from the interplay of gravity and momentum.

8.4 Application: The Golf Downswing

While the golf swing is not a periodic walking gait, the phase-variable approach perfectly encapsulates the athletic reality of the swing. The golfer's shoulder angle defines the phase

s. The target manifold is $s = 1.0$ (the impact location).

By structuring the control scheme computationally through Virtual Constraints, the golfer is impervious to slight hesitations at the top of the backswing. The transverse feedback LQR (Chapter 4) corrects spatial deviations from the unactuated club (θ_2), while the phase variable ensures the arm actuation (u_1) applies the necessary parametric braking entirely in synchrony with the physical location of the arms, leading to perfectly timed passive impact dynamics regardless of chronological speed.

Chapter 9

Stochastic Trajectories & Motor Variability

Perfection is inherently fragile. Biological optimization seeks the strongest balance between intent and reality.

In Plain Language 9.1. The Harder You Try, The More You Miss

If you program a robot motor to apply 50 units of force, it applies exactly 50 units of force, every single time. Human muscles don't work like that. The harder you try to contract a muscle, the more "noisy" and jittery the actual force becomes. This is a massive problem for trying to swing a golf club as fast as humanly possible: the closer you get to your maximum physical effort, the more unpredictable your motion becomes. This chapter explains the math behind why smooth, controlled movements are more accurate than jerky, maximum-effort ones. We'll see that a truly "optimal" golf swing doesn't use 100% of your strength, because dialing it back slightly eliminates a huge amount of physical chaos, making the swing actually repeatable.

9.1 Signal-Dependent Noise

When a roboticist specifies a torque of 50 Nm, the motor delivers 50 Nm with a tiny, constant variance. Biological actuators do not work this way. Human muscles generate force by recruiting discrete motor units. As higher forces are required, larger, more fast-twitch oriented motor units are recruited. This physiological reality manifests as a profound control challenge: **motor noise is proportional to the command signal**.

Definition 9.1. Signal-Dependent Noise (SDN)

A control input $\boldsymbol{u}(t)$ applied by a biological system is actually realized as a stochastic variable $\tilde{\boldsymbol{u}}(t)$:

$$\tilde{\boldsymbol{u}}(t) = \boldsymbol{u}(t) + \boldsymbol{W}\boldsymbol{u}(t) \cdot \boldsymbol{\epsilon}(t) \tag{9.1}$$

where $\boldsymbol{W}$ is a scaling matrix representing the coefficient of variation, and $\boldsymbol{\epsilon}(t) \sim \mathcal{N}(0, I)$ is white noise. That is, the variance of the applied force grows squarely with the magnitude of the ordered force: $\text{Var}(\tilde{u}) = cu^2$.

This simple reality crumbles deterministic optimal control. If you compute an optimal trajectory that requires maximum muscular exertion, the trajectory will mathematically possess minimum variance only if noise is constant. Under SDN, maximum exertion guarantees maximum chaotic noise.

9.2 Minimum Variance Theory of Human Movement

Why do humans move in smooth, bell-shaped velocity profiles rather than jerky "bang-bang" optimal control solutions when reaching for coffee?

Classical control attempts to explain this via "minimum jerk" theories, positing that the nervous system explicitly encodes an optimization functional to minimize $\int \dddot{x}^2 \, dt$. The trajectory-first perspective offers a much deeper, physically grounded explanation championed by Harris and Wolpert: the **Minimum Variance Theory**.

Key Idea 9.1. Optimization for Repeatability

The human nervous system seeks to move in a way that minimizes the final endpoint variance evaluated across multiple trials.

Under SDN, the only way to minimize endpoint variance (ensure you actually grasp the coffee cup smoothly) is to minimize sudden spikes in required actuator forces, because large forces inject huge amounts of uncertainty into the dynamics. Smooth trajectories inherently possess lower peak torque commands than rapid accelerations and decelerations. Smoothness is not the explicitly programmed objective; it is an emergent property of attempting to be consistently accurate under the unavoidable reality of signal-dependent noise.

9.3 The Speed-Accuracy Tradeoff in the Golf Swing

Fitts's Law states that the faster you try to move to a target, the less accurate you become. In golf, we see the supreme manifestation of this. The deterministic optimal control solution (Chapter 6) tells the solver to maximize input torque to achieve theoretically maximal clubhead speed.

However, if we introduce the Stochastic OCP (via Covariance Steering) from Chapter 6 and apply the realistic premise of SDN, the optimizer faces a profound tradeoff.

Applying maximum u_1 torque during the downswing injects enormous variance into the kinematic state. The unactuated club (θ_2) begins whipping through its zero dynamics, but its exact phase progression is now highly uncertain due to the noise flooding the arms.

To ensure the clubface squares up at the moving goal, the stochastic optimizer will *deliberately back off* the peak requested torque. It trades a deterministic 5 mph gain in

clubhead speed for a massive reduction in angular variance at the target manifold. The optimal biological swing is mathematically proven to be a sub-maximal physical exertion.

The extension of the deterministic Pontryagin framework to stochastic systems is developed rigorously in Agrachev and Sachkov [1]. When the system is subject to signal-dependent noise (SDN), the traditional Hamiltonian maximization is supplemented by terms encoding risk: the optimizer must maximize expected performance while managing the variance of terminal conditions under noise propagation. This leads to **risk-sensitive control** problems where the cost includes an entropy-like term penalizing uncertainty.

For the golf swing under SDN, the stochastic extension of the attainable set framework tells us that not all nominally reachable configurations are equally *robustly* reachable. A configuration in the interior of the deterministic attainable set may lie on the boundary of the stochastic attainable set if noise-driven perturbations commonly push trajectories away from it. By contrast, configurations that are "stable" with respect to noise (in the direction of the control Lie algebra) remain deeply inside the stochastic attainable set. The funnel computed in Chapter 7 with stochastic cost weighting thus encodes not just reachability, but **robust attainment**—guarantee of success under unavoidable noise.

9.4 Summary

Concept	**Implication for Control**
Signal-Dependent Noise (SDN)	Muscle noise variance proportional to command: $\mathrm{Var}(\tilde{u}) = cu^2$. Requires bounded control effort for repeatability.
Minimum Variance Theory	Explains smooth, controlled velocity profiles as solutions to endpoint error variance minimization under SDN.
Stochastic OCP	Optimal control formulation that minimizes $\mathbb{E}[\phi(\boldsymbol{x}(t_f))] + \lambda \cdot \mathrm{Var}[\psi(\boldsymbol{x}(t_f))]$.
Covariance Steering	Technique to propagate the covariance matrix $\boldsymbol{\Sigma}(t)$ alongside the nominal trajectory during optimization.
Speed-Accuracy Tradeoff	Formal manifestation of Fitts's Law: higher peak forces inject noise proportional to force squared.

Chapter 10

Learning to Move

The first swing establishes the manifold. The ten-thousandth swing perfects the funnel.

In Plain Language 10.1. Practice Makes Funnels

When you swing a golf club for the first time, your brain is guessing how much force to use, guessing the weight of the club, and guessing how your muscles will respond. The mathematical models we've built so far are just like that first swing—a really smart guess, but still a guess. Real physics (wind, friction, fatigue) is too messy to model perfectly. This chapter is about how systems *learn*. Instead of trying to calculate the perfect math equations from scratch every time, the system tries a movement, measures how badly it missed the target, and updates its "muscle memory" for the next try. Over thousands of repetitions, the brain (or the computer) learns the true physics of the motion without ever needing the exact equations. It builds a whole library of these learned motions, letting it instantly adjust to new situations without thinking.

10.1 Iterative Learning Control (ILC)

Trajectory optimization (Chapter 6) computes a reference motion strictly from a mathematical model. However, dynamic models of humans—and even robots—are invariably flawed. Frictional forces are unmodeled, inertias are estimated, and actuators possess hidden dynamics. If you execute a computed reference $\boldsymbol{\gamma}^*(t)$ open-loop, the physical system will likely fail to strike the moving target due to these discrepancies.

Iterative Learning Control (ILC) resolves this by leveraging the repetitive nature of tasks like a golf swing. ILC exploits the premise that while models are inaccurate, *the unmodeled dynamics are highly deterministic across repeated trials.*

Remark 10.1. ILC and the Orbit of Feasible Trajectories

From the perspective of geometric control theory as presented by Agrachev and Sachkov [1], each iterative trial of ILC represents an exploration within the orbit of

feasible trajectories generated by the control Lie algebra. The true plant's dynamics define an actual (unknown) orbit in state space; the nominal model defines an approximate orbit. ILC learns the discrepancy between these orbits by iteratively modulating the feedforward command. The convergence of ILC is thus a convergence toward the true orbit boundary—the set of trajectories reachable on the actual (unknown) plant. Persistent excitation conditions required for identifiability of model parameters correspond to activating the control Lie algebra sufficiently to explore all dimensions of the reachable subspace. In other words, ILC's success in learning motion requires that the control inputs, applied across trials, persistently excite the full Lie algebra of admissible directions.

Iterative Learning Control (ILC) resolves this by leveraging the repetitive nature of tasks like a golf swing. ILC exploits the premise that while models are inaccurate, *the unmodeled dynamics are highly deterministic across repeated trials.*

If a robot under-swings by 5 cm on the first attempt, the controller records this terminal error and updates the feedforward command $\boldsymbol{u}_{\text{ff}}$ for the *second* attempt. Over repeated trials, the system learns the inverse dynamics of the true physical plant without ever explicitly identifying the model parameters.

Definition 10.1. ILC Update Law

Let j denote the trial index. The feedforward command for trial $j+1$ is updated based on the command and the *transverse error* $\boldsymbol{\xi}$ from trial j:

$$\boldsymbol{u}_j(t) = \boldsymbol{u}_{\text{ff},j}(t) + \boldsymbol{K}_\perp(t)\boldsymbol{\xi}_j(t) \tag{10.1}$$

$$\boldsymbol{u}_{\text{ff},j+1}(t) = \boldsymbol{u}_{\text{ff},j}(t) + \boldsymbol{L}(t)\boldsymbol{\xi}_j(t) \tag{10.2}$$

where $\boldsymbol{L}(t)$ is a learning filter. This updates the nominal trajectory to iteratively cancel out deterministic disturbances.

10.2 Policy Search and Reinforcement Learning

While ILC updates the feedforward commands to zero-out deterministic tracking errors, **Policy Search** algorithms optimize the *shape of the funnel* itself.

Instead of computing the transverse LQR gains $\boldsymbol{K}_\perp(t)$ from an analytical Riccati equation (Chapter 4), we parameterize the controller as a policy $\pi_{\boldsymbol{\theta}}(\boldsymbol{x})$ and search the parameter space $\boldsymbol{\theta}$ to minimize the variance at the moving goal under realistic noise.

Reinforcement Learning (RL), specifically policy gradient methods, allows the agent to sample thousands of golf swings in simulation, continuously updating its policy parameters to maximize the expected reward (clubhead speed and squareness at impact). A heavily trained RL agent organically converges on the very principles we mathematically derived in earlier chapters:

1. It abandons rigid time-tracking and autonomously discovers phase-variable mapping.

2. It exploits the underactuated double-pendulum whipping motion to gain speed.

3. It artificially widens its "funnel" during the transition phase, realizing that overly tight feedback control here merely amplifies motor noise and spoils the impact constraints.

10.3 Trajectory Libraries

A singular optimal trajectory $\boldsymbol{\gamma}^*$ and its corresponding funnel $\mathcal{F}$ are highly specific to a single objective. What if the golfer wishes to fade the ball, hit a draw, or execute a punch shot out of the rough?

Instead of re-running a massive NLP optimization for every scenario, biological systems and advanced controllers maintain **Trajectory Libraries**. A library consists of several distinct, pre-computed optimal trajectories and their corresponding local funnels.

When faced with a new task (e.g., striking the ball on an uneven lie), the system searches its library for the closest matching funnel. By interpolating between established funnels using techniques like LQR-Trees [15] or simple spatial blending, the system can instantly generate a stable, feasible motion without any online optimization.

Chapter 11

Case Study: The Complete Golf Swing

The physics are invariant. What separates the professional is their mastery over the geometry of the resultant funnel.

In Plain Language 11.1. Putting It All Together

Everything we've learned so far comes together in the golf swing. We started by throwing away the idea of a fixed target and instead focusing on the path (Chapter 1). We looked at the geometry of that path (Chapters 2 and 3), how to stay on it regardless of timing (Chapter 4), and how to let momentum do the heavy lifting for the wrists (Chapter 5). Then we let a computer discover the optimal path (Chapter 6) and built a mathematical funnel around it to guarantee it works every time (Chapter 7), throwing away the clock entirely (Chapter 8) because trying too hard just makes things worse (Chapter 9). Finally, we saw how the human body learns these funnels through repetition (Chapter 10). This final chapter is the grand finale, tying all these concepts together to explain, in pure mathematical terms, what makes a professional golf swing so effortlessly powerful and wildly consistent.

11.1 Unifying the "Control is Motion" Framework

Throughout this text, we have systematically deconstructed the classical control paradigm and rebuilt it to handle tasks defined uniquely by motion. We conclude by applying our holistic framework to a 15-DOF musculoskeletal model of the human golf swing in order to formally synthesize the mathematical structure of athletic perfection.

The objective is clear: maximize negative clubhead velocity at the precise spatial intersection with the golf ball, while maintaining face angle orthogonality. The exact time interval t_f in which this occurs is irrelevant; only the geometric state at impact matters. Furthermore, the commanded motion must not exceed the structural limits of biological torque, and the trajectory must be highly robust to physiological signal-dependent noise (SDN).

11.1.1 Phase 1: Defining the Geometry and Optimization

The system is heavily underactuated. The wrists are treated as passive pin joints, delegating the final propulsive phase entirely to inertial coupling.

We formulate a **Stochastic Moving Target Optimal Control Problem** (Chapter 6). We deploy Direct Collocation to optimize the trajectory $\boldsymbol{\gamma}^*(s)$ parameterized over a spatial phase variable s representing the shoulder angle (Chapter 8). The integral cost function strictly penalizes signal variance (Chapter 9) by incorporating local covariance propagation.

The collocation solver successfully navigates the $\boldsymbol{B}^\perp$ annihilator constraint (Chapter 5) and produces an optimal, smooth backswing. It discovers that a massive torque inversion (braking the torso) right before impact causes the zero dynamics of the passive wrist to wildly accelerate, perfectly squaring the face precisely as the clubhead spatial coordinate intersects the ball's location.

11.1.2 Phase 2: Funnel Synthesis and Robustness

With $\boldsymbol{\gamma}^*(s)$ computed, we lock down the trajectory using **Transverse Linearization** (Chapter 4). We ignore errors along the tangent of the path (since chronological timing is irrelevant) and construct a time-varying LQR feedback matrix to regulate only the transverse deviations.

Finally, we compute the verified bounds of this stability region using **Sums-of-Squares Programming** (Chapter 7). The SDP calculates a rigorous boundary function $V(\boldsymbol{x}, t) = 1$ that proves the system is confined to an invariant funnel.

Key Idea 11.1. The Architecture of the Perfect Swing

The final computed funnel reveals the mathematical secret of the professional golfer. During the backswing, the funnel is immensely wide; the motion is forgiving, and vast transverse deviations are easily tolerated without violating the invariant boundary.
As the swing approaches the transition and the explosive downswing, the funnel aggressively tightens. The stochastic optimization has selected a trajectory whose passive unactuated dynamics are so profoundly stable in the final 0.15 seconds that the funnel diameter at impact shrinks to virtually zero. Any small deviation injected by muscular noise is passively swallowed by the immense centrifugal geometry of the zero dynamics. The golfer does not aggressively steer the club to the ball; they simply set the initial conditions, and the physics irresistibly pull the clubface through the target manifold.

Through this book, we hope we have proven that the thermostat has its place, but when the objective is a sweeping, dynamic motion cutting through the physical world, "Control is Motion" provides the only mathematically truthful language.

Bibliography

[1] Andrei A. Agrachev and Yuri L. Sachkov. *Control Theory from the Geometric Viewpoint*, volume 87 of *Encyclopaedia of Mathematical Sciences*. Springer-Verlag, Berlin, 2004. Comprehensive treatment of geometric methods in control theory including Lie brackets, controllability via the Orbit theorem and Rashevskii–Chow theorem, the Pontryagin Maximum Principle from the symplectic viewpoint, sub-Riemannian geometry, and second-order optimality conditions. A foundational reference for the geometric approach to nonlinear control adopted throughout this series.

[2] S. Boyd, L. El Ghaoui, E. Feron, and V. Balakrishnan. *Linear Matrix Inequalities in System and Control Theory*. SIAM, 1994.

[3] F. Bullo. *Contraction Theory for Dynamical Systems*. Kindle Direct Publishing, 2024.

[4] F. Bullo and A. D. Lewis. *Geometric Control of Mechanical Systems*. Springer, 2004.

[5] R. Featherstone. *Rigid Body Dynamics Algorithms*. Springer, 2008.

[6] T. Flash and N. Hogan. The coordination of arm movements: an experimentally confirmed mathematical model. *Journal of Neuroscience*, 5(7):1688–1703, 1985.

[7] O. Khatib. A unified approach for motion and force control of robot manipulators: The operational space formulation. *IEEE Journal on Robotics and Automation*, 3(1):43–53, 1987.

[8] W. Lohmiller and J.-J. E. Slotine. On contraction analysis for non-linear systems. *Automatica*, 34(6):683–696, 1998.

[9] A. Majumdar and R. Tedrake. Funnel libraries for real-time robust feedback motion planning. *The International Journal of Robotics Research*, 36(8):947–982, 2017.

[10] I. R. Manchester and J.-J. E. Slotine. Control contraction metrics: Convex and intrinsic criteria for nonlinear feedback design. *IEEE Transactions on Automatic Control*, 62(6):3046–3053, 2017.

[11] R. M. Murray, Z. Li, and S. S. Sastry. *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994.

[12] A. S. Shiriaev, L. B. Freidovich, and S. V. Gusev. Transverse linearization for controlled mechanical systems with several passive degrees of freedom. *IEEE Transactions on Automatic Control*, 55(12):2802–2814, 2010.

[13] J.-J. E. Slotine and W. Li. *Applied Nonlinear Control*. Prentice Hall, 1991.

[14] M. W. Spong, S. Hutchinson, and M. Vidyasagar. *Robot Modeling and Control*. Wiley, 2006.

[15] R. Tedrake. Lqr-trees: Feedback motion planning on sparse randomized trees. In *Robotics: Science and Systems*, 2009.

[16] E. Todorov and M. I. Jordan. Optimal feedback control as a theory of motor coordination. *Nature Neuroscience*, 5(11):1226–1235, 2002.

[17] E. R. Westervelt, J. W. Grizzle, C. Chevallereau, J. H. Choi, and B. Morris. *Feedback Control of Dynamic Bipedal Robot Locomotion*. CRC Press, 2007.

The entire pipeline—from identifying the control-affine structure of the musculoskeletal system, through characterizing controllability via the Orbit Theorem, to optimal control via Pontryagin, to funnel synthesis via Lyapunov and SOS verification, to phase-variable phase-reduction and virtual constraint enforcement, to stochastic robustness and learning—is a comprehensive realization of the geometric control framework developed by Agrachev and Sachkov [1].

The key insight is that a professional golfer does not "control" the club in the classical sense. Instead, the golfer understands, through years of practice, the natural geometry of reachable trajectories (the orbit generated by their control Lie algebra), and designs inputs that exploit this geometry to their maximum. The passive dynamics of the wrists and the inertial couplings are not obstacles to be overcome; they are features to be orchestrated. The result is motion that appears effortless and infinitely repeatable—because it is aligned with the underlying geometric structure of the system.

11.2 Conclusion and Future Directions

Through this book, we have aimed to reconstruct control theory from first principles, replacing the setpoint paradigm with a trajectory-centric view. We have shown how the geometric structure of nonlinear systems—revealed through Lie algebras, Lie brackets, sub-Riemannian metrics, and attainable sets—is not merely of mathematical interest, but is central to designing robust, efficient, and ultimately repeatable motion.

The thermostat ha

Glossary of Important Concepts

Articulated Body Algorithm A recursive $O(N)$ algorithm for computing the forward dynamics of a kinematic chain, widely used in physics engines.

Configuration Space The space (often a manifold) of all possible positions of a system.

Contraction Analysis A framework for analyzing the stability of trajectories with respect to each other, rather than with respect to an equilibrium point.

Control Contraction Metric (CCM) A Riemannian metric that can be used to synthesize stabilizing feedback controllers for nonlinear systems along arbitrary feasible trajectories.

Degrees of Freedom (DOF) The minimum number of independent coordinates required to completely specify the configuration of a system.

Exponential Map The operation that relates a Lie Algebra (representing velocity) to its corresponding Lie Group (representing position/orientation).

Euler Angles A 3-parameter representation of orientation that is subject to gimbal lock.

Flow The solution map of a differential equation smoothly moving points along vector fields in state space.

Gimbal Lock A coordinate singularity where a 3D rotation parameterization loses a degree of freedom.

Jacobian The matrix of partial derivatives relating joint velocities to task-space velocities.

Kinematics The study of motion geometry without considering the forces that cause it.

Lagrangian Mechanics An energy-based formulation of mechanics utilizing kinetic and potential energy to strictly determine the equations of motion.

Lie Algebra The mathematical space of velocities associated with a Lie Group, corresponding to its tangent space at the identity.

Lie Group A smooth manifold that is also a mathematical group, typically describing continuous symmetries or rigid body transformations (e.g., $SO(3)$, $SE(3)$).

Lyapunov Stability The classical definition of stability describing whether a system returns to an equilibrium point when perturbed.

Manifold A topological space that locally resembles Euclidean space, crucial for rigorous formulation of non-trivial state spaces.

State Space The space of all possible states of a dynamical system, usually comprising both configuration and momentum/velocity.

Screw Axis A geometric representation of spatial motion combining a rotation axis and a translation along that axis, foundational for modern rigid body mathematics.

Tangent Space The vector space of all possible velocities at a specific point on a manifold.

Tangent Bundle The manifold consisting of all tangent spaces glued together, typically representing the full state space of mechanical systems.

Twist The spatial velocity of a rigid body, composed of angular and linear velocity components, residing in the Lie Algebra.

Wrench The spatial force acting on a rigid body, combining torque and linear force.

Further Reading and References

This text is a primer and introductory guide designed to construct an intuitive and unified framework. The formal depths of modern geometric mechanics and nonlinear control are actively pioneered and formalized across several seminal texts. For the reader eager to pursue more mathematically rigorous treatments, we strongly recommend the following resources:

Geometric Control and Robotic Manipulation

- **A Mathematical Introduction to Robotic Manipulation** [11]. The definitive textbook bridging Lie group theory with classical robotics, establishing the Product of Exponentials (PoE) standard.
- **Geometric Control of Mechanical Systems** [4]. An authoritative resource on the differential geometry of mechanical control systems, including connections and geodesics.
- **Robot Modeling and Control** [14]. A highly accessible and rigorous text covering everything from basic Euler-Lagrange forms to robust robotic control.

Advanced Rigid Body Dynamics

- **Rigid Body Dynamics Algorithms** [5]. To truly implement physics engines, Featherstone's Spatial Vector notation and the derivation of $O(N)$ ABA are unmatched.

Nonlinear Systems and Contraction

- **Applied Nonlinear Control** [13]. Slotine's classic text providing the most intuitive treatment of sliding mode and robust nonlinear stability.
- **Contraction Theory for Dynamical Systems** [3]. The modern codification of contraction bounding and metrics for trajectory stability.